Progetto di Ingegneria del Software
Parte A

Raffaele Principe (749438) – Riccardo Marini (746230)

Anno Accademico 2025/2026

Indice

1 Versione 1 **4**
Casi d'uso testuali 4
Casi d'uso grafici 14
Diagramma delle classi 15

2 Versione 2 **17**
Casi d'uso testuali modificati 17
Casi d'uso testuali nuovi 18
Casi d'uso grafici 22
Diagramma delle classi 24

3 Versione 3 **26**
Casi d'uso testuali modificati 26
Casi d'uso testuali nuovi 29
Casi d'uso grafici 36
Diagramma delle classi 38

4 Versione 4 **41**
Casi d'uso testuali nuovi 41
Casi d'uso grafici 43
Diagramma delle classi 46

5 Versione 5 **49**
Casi d'uso testuali nuovi 49
Casi d'uso grafici 52
Diagramma delle classi 54

6 Versione Finale **57**
Casi d'uso testuali 57
Casi d'uso grafici 83
Diagramma delle classi 85

Sommario **2**

1 Installazione **5**
1.1 Requisiti di sistema 5
1.1.1 Requisiti hardware 5
1.1.2 Requisiti software 5
1.1.3 Verifica della versione Java installata 5

1.2 Ottenere i file del progetto . . . 6
1.3 Compilazione . . . 6
1.3.1 Linux e macOS — script automatico . . . 6
1.3.2 Windows — script automatico . . . 6
1.3.3 Compilazione manuale (qualsiasi sistema operativo) . . . 6
1.4 Prima esecuzione . . . 7

2 Avvio e navigazione generale 8
2.1 Menu di selezione del ruolo . . . 8
2.2 Convenzioni di interfaccia . . . 8
2.3 Transizioni automatiche di stato . . . 9

3 Guida al back-end del configuratore 10
3.1 Accesso e gestione delle credenziali . . . 10
3.1.1 Primo accesso . . . 10
3.1.2 Accessi successivi . . . 10
3.1.3 Multi-configuratore . . . 10
3.2 Menu principale del configuratore . . . 10
3.3 Gestione dei campi . . . 11
3.3.1 Campi base (voce 1) . . . 11
3.3.2 Campi comuni (voce 2) . . . 11
3.3.3 Campi specifici (voce 3 — sotto-menu campi specifici) . . . 12
3.4 Gestione delle categorie . . . 12
3.4.1 Aggiunta di una categoria (voce 3 – a) . . . 12
3.4.2 Gestione dei campi specifici (voce 3 – c) . . . 12
3.4.3 Rimozione di una categoria (voce 3 – r) . . . 12
3.4.4 Visualizzazione (voce 4) . . . 12
3.5 Gestione delle proposte . . . 12
3.5.1 Ciclo di vita di una proposta . . . 13
3.5.2 Creazione di una proposta (voce 5 – n) . . . 13
3.5.3 Continuare la compilazione (voce 5 – c) . . . 13
3.5.4 Pubblicazione (voce 5 – p) . . . 14
3.5.5 Eliminazione dalla sessione (voce 5 – e) . . . 14
3.6 Visualizzazione della bacheca (voce 6) . . . 14
3.7 Visualizzazione dell'archivio (voce 7) . . . 14
3.8 Ritiro di una proposta (voce 8) . . . 14

4 Guida al front-end del fruitore 15
4.1 Registrazione e accesso . . . 15
4.1.1 Registrazione (primo accesso) . . . 15
4.1.2 Login . . . 15
4.2 Menu principale del fruitore . . . 15
4.3 Visualizzazione della bacheca (voce 1) . . . 15
4.4 Adesione a una proposta (voce 2) . . . 16
4.5 Disdetta dell'iscrizione (voce 3) . . . 16
4.6 Spazio personale e notifiche (voce 4) . . . 16

5 Importazione batch NUOVO V5 17
5.1 Panoramica . . . 17

5.2 Accesso alla funzionalità 17
5.3 Formato dei file batch 17
5.3.1 Commenti e righe vuote 17
5.3.2 Comando CAMPO_COMUNE 18
5.3.3 Comando CATEGORIA 18
5.3.4 Comando PROPOSTA 18
5.4 Importazione di un singolo file 19
5.5 Importazione di più file in sequenza 19
5.6 Il resoconto di importazione 19
5.7 File batch di esempio 20

6 Formato dei dati e persistenza 21
6.1 File di persistenza 21
6.2 Cosa viene salvato 21
6.3 Cosa *non* viene salvato 21
6.4 Compatibilità retroattiva 21
6.5 Backup e ripristino 22

7 Messaggi di errore e risoluzione dei problemi 23
7.1 Messaggi dell'applicazione 23
7.2 Messaggi dell'importazione batch 24
7.3 Problemi di avvio 25

A Credenziali e dati demo precaricati 26
A.1 Credenziali 26
A.2 Proposte precaricate 26

B File batch di esempio completo 27

CAPITOLO 1

Versione 1

Casi d'uso testuali

Nome	**UC01 - Avvio e Login**
Attore	Configuratore
Scenario principale	Precondizione: L'applicazione è avviata. Trigger: L'utente seleziona "Configuratore" dal menu iniziale. 1. Il sistema mostra la schermata di login dedicata. 2. Il configuratore inserisce username e password. 3. Il sistema verifica la corrispondenza delle credenziali. 4. Il sistema autentica il configuratore. 5. Il sistema verifica se si tratta del primissimo accesso assoluto (credenziali di default). 6. «include» "UC02 - Imposta credenziali personali" (se primo accesso). 7. Il sistema mostra il menu principale. Postcondizione: Il configuratore è autenticato nel back-end. Fine
Scenario alternativo	2a. Precondizione: il configuratore digita "esci" come username. Il sistema interrompe l'operazione e torna al menu di selezione ruolo. Fine
Scenario alternativo	3a. Precondizione: le credenziali non sono valide. Il sistema mostra un errore e indica i tentativi rimasti. Torna al punto 2 (se tentativi < 3), altrimenti il sistema blocca l'accesso alla sessione. Fine
Scenario alternativo	3b. Precondizione: nessun configuratore registrato nel sistema. Il sistema accetta le credenziali di default (admin/admin123), crea il profilo iniziale, imposta la necessità di cambio credenziali e prosegue al punto 6.

Nome	**UC02 - Imposta credenziali personali**
Attore	Configuratore
Scenario principale	Precondizione: Login effettuato con credenziali di default (primo accesso). Trigger: Il sistema rileva la necessità di cambiare le credenziali. 1. Il sistema richiede di scegliere le credenziali personali. 2. Il configuratore inserisce il nuovo username. 3. Il sistema verifica che lo username non coincida con la parola riservata "esci". 4. Il configuratore inserisce la nuova password. 5. Il configuratore inserisce la password per conferma. 6. Il sistema verifica che le due password coincidano. 7. Il sistema verifica l'unicità globale dello username tra Configuratori. 8. Il sistema aggiorna le credenziali in memoria. 9. Il sistema salva i dati in modo persistente. 10. Il sistema mostra la conferma e accede al menu. Postcondizione: Le credenziali di default sono sostituite e salvate in modo permanente. Fine
Scenario alternativo	2a. Precondizione: il configuratore digita "annulla" nel campo username. Il sistema annulla l'operazione ed esegue il logout automatico. Fine
Scenario alternativo	3a. Precondizione: lo username inserito è "esci". Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	4a. Precondizione: il configuratore digita "annulla" nel campo password. Il sistema annulla l'operazione ed esegue il logout automatico. Fine
Scenario alternativo	6a. Precondizione: le password non coincidono. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	7a. Precondizione: campi lasciati vuoti. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	8a. Precondizione: username già in uso nel sistema. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	9a. Precondizione: Errore di salvataggio (I/O). Il sistema ripristina le credenziali in memoria, mostra un errore ed esegue il logout. Fine

Nome	**UC03 - Logout**
Attore	Configuratore
Scenario principale	Precondizione: Configuratore autenticato. Trigger: Selezione "Logout". 1. Il configuratore seleziona Logout. 2. Il sistema termina la sessione corrente. 3. Il sistema chiede se accedere con un altro account. 4. Sì: ritorna a UC01. 5. No: termina l'applicazione. Fine
Scenario alternativo	3a. Precondizione: Il configuratore inserisce una risposta non valida (diversa da "s" o "n"). Il sistema mostra un messaggio di errore testuale e ripropone la domanda.

Nome	**UC04 - Visualizza campi base**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. I campi base sono stati inizializzati. Trigger: Il configuratore seleziona "Gestione campi base" dal menu principale. 1. Il configuratore seleziona la voce "Gestione campi base". 2. Il sistema mostra l'elenco dei campi base con nome e tipo, evidenziando che sono tutti obbligatori (Titolo, Numero partecipanti, ecc.). 3. Il sistema indica che i campi base sono immutabili. 4. Il configuratore preme INVIO per tornare al menu principale. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	2a. Precondizione: Campi base non inizializzati. Il sistema li inizializza automaticamente al primo avvio e poi mostra l'elenco. Fine

Nome	**UC05 - Aggiungi campo comune**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Il configuratore seleziona "Aggiungi campo comune" nel menu "Gestione campi comuni". 1. Il configuratore seleziona "Aggiungi campo comune". 2. Il sistema chiede il nome del nuovo campo. 3. Il configuratore inserisce il nome. 4. Il sistema chiede se il campo è obbligatorio o facoltativo. 5. Il configuratore risponde (s/n). 6. Il sistema verifica che il nome non coincida con un campo base o comune esistente. 7. Il sistema aggiunge il campo comune e salva in modo persistente. 8. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Nome vuoto. Il sistema mostra errore e richiede nuovamente il nome.
Scenario alternativo	6a. Precondizione: Conflitto con campo base. Il sistema mostra errore e non aggiunge il campo. Fine
Scenario alternativo	6b. Precondizione: Conflitto con campo comune esistente. Il sistema mostra errore e non aggiunge il campo. Fine
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiunge il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC06 - Rimuovi campo comune**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno un campo comune. Trigger: Il configuratore seleziona "Rimuovi campo comune". 1. Il configuratore seleziona "Rimuovi campo comune". 2. Il sistema mostra l'elenco dei campi comuni esistenti. 3. Il configuratore inserisce il nome del campo da rimuovere. 4. Il sistema rimuove il campo comune e salva in modo persistente. 5. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra errore e non esegue la rimozione. Fine
Scenario alternativo	4a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC07 - Modifica obbligatorietà campo comune**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno un campo comune. Trigger: Il configuratore seleziona "Modifica obbligatorietà campo comune". 1. Il configuratore seleziona "Modifica obbligatorietà campo comune". 2. Il sistema mostra l'elenco dei campi comuni con l'obbligatorietà attuale. 3. Il configuratore inserisce il nome del campo da modificare. 4. Il configuratore indica il nuovo valore di obbligatorietà (s/n). 5. Il sistema aggiorna il campo e salva in modo persistente. 6. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra errore e non applica modifiche. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	5a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiorna il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC08 - Aggiungi categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. I campi base sono inizializzati. Trigger: Il configuratore seleziona "Aggiungi categoria". 1. Il configuratore seleziona "Aggiungi categoria". 2. Il sistema chiede il nome della nuova categoria. 3. Il configuratore inserisce il nome. 4. Il sistema verifica che il nome non sia già in uso. 5. Il sistema crea la categoria e salva in modo persistente. 6. Il sistema chiede se aggiungere subito campi specifici. 7. Sì: il caso d'uso estende UC10 (aggiunta campi specifici). 8. No: il sistema torna al menu categorie. 9. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Nome vuoto. Errore e richiesta nuovo inserimento.
Scenario alternativo	4a. Precondizione: Categoria duplicata. Errore e nessuna creazione. Fine
Scenario alternativo	5a. Precondizione: Errore di salvataggio (I/O). Il sistema crea la categoria in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC09 - Rimuovi categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una categoria. Trigger: Il configuratore seleziona "Rimuovi categoria". 1. Il configuratore seleziona "Rimuovi categoria". 2. Il sistema mostra l'elenco categorie. 3. Il configuratore inserisce il nome della categoria da rimuovere. 4. Il sistema chiede conferma esplicita prima della rimozione. 5. Il configuratore conferma. 6. Il sistema rimuove la categoria e tutti i suoi campi specifici. 7. Il sistema salva i dati in modo persistente. 8. Il sistema mostra conferma. Fine
Scenario alternativo	3a. Precondizione: Categoria non trovata. Errore, nessuna rimozione. Fine
Scenario alternativo	5a. Precondizione: Annulla. Il configuratore non conferma; operazione interrotta. Fine
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove la categoria in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC10 - Aggiungi campo specifico a categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una categoria. Trigger: Il configuratore sceglie di aggiungere un campo specifico a una categoria. 1. Il configuratore seleziona "Gestisci campi specifici" e indica la categoria. 2. Il sistema verifica che la categoria esista. 3. Il configuratore seleziona "Aggiungi campo specifico". 4. Il sistema chiede il nome del nuovo campo. 5. Il configuratore inserisce il nome. 6. Il sistema chiede se il campo è obbligatorio o facoltativo. 7. Il configuratore risponde (s/n). 8. Il sistema verifica che il nome non coincida con un campo base, comune o specifico già presente. 9. Il sistema aggiunge il campo specifico alla categoria e salva. 10. Il sistema mostra conferma e permette inserimenti successivi (INVIO per terminare). Fine
Scenario alternativo	2a. Precondizione: Categoria non esiste. Il sistema mostra un messaggio di errore. Fine
Scenario alternativo	5a. Precondizione: Nome vuoto. Il sistema mostra un errore e richiede un nuovo inserimento.
Scenario alternativo	8a. Precondizione: Nome duplicato. Il sistema mostra un errore e non procede con l'aggiunta. Fine
Scenario alternativo	9a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiunge il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC11 - Rimuovi campo specifico da categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste una categoria con almeno un campo specifico. Trigger: Il configuratore seleziona "Rimuovi campo specifico". 1. Il configuratore seleziona la categoria. 2. Il sistema mostra i campi specifici della categoria. 3. Il configuratore inserisce il nome del campo da rimuovere. 4. Il sistema rimuove il campo specifico e salva in modo persistente. 5. Il sistema mostra conferma. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra un messaggio di errore. Fine
Scenario alternativo	4a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC12 - Modifica obbligatorietà campo specifico**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste una categoria con almeno un campo specifico. Trigger: Il configuratore seleziona "Modifica obbligatorietà campo specifico". 1. Il configuratore seleziona la categoria. 2. Il sistema mostra i campi specifici con obbligatorietà attuale. 3. Il configuratore inserisce il nome del campo da modificare. 4. Il configuratore indica il nuovo valore (s/n). 5. Il sistema aggiorna il campo specifico e salva in modo persistente. 6. Il sistema mostra conferma. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra un messaggio di errore. Fine

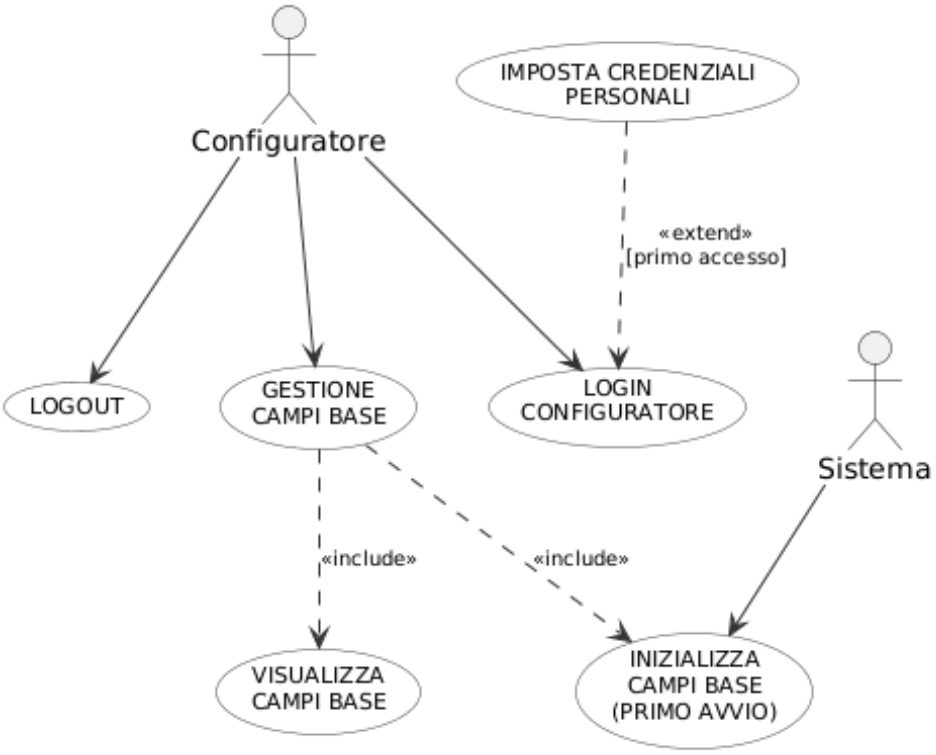
Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	5a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiorna il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC13 - Visualizza categorie e campi**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Il configuratore seleziona "Visualizza riepilogo categorie e campi". 1. Il configuratore seleziona "Visualizza riepilogo". 2. Il sistema mostra i campi BASE. 3. Il sistema mostra i campi COMUNI (con indicazione di obbligatorio/facoltativo). 4. Per ogni categoria, il sistema mostra: nome categoria e campi specifici (con indicazione di obbligatorio/facoltativo). 5. Il configuratore preme INVIO per tornare al menu. Fine
Scenario alternativo	3a. Precondizione: Nessun campo comune definito. Il sistema mostra la dicitura "(nessuno definito)".
Scenario alternativo	4a. Precondizione: Nessuna categoria definita nel sistema. Il sistema mostra la dicitura "(nessuna categoria definita)".
Scenario alternativo	4b. Precondizione: Una categoria non possiede campi specifici. Il sistema mostra la dicitura "(nessun campo specifico)" sotto il nome della categoria interessata.

Nome	**UC14 - Visualizza dettaglio categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una categoria. Trigger: Il configuratore seleziona "Visualizza dettaglio categoria". 1. Il configuratore seleziona "Visualizza dettaglio categoria". 2. Il sistema richiede l'inserimento del nome della categoria. 3. Il configuratore inserisce il nome richiesto. 4. Il sistema mostra, per la categoria selezionata, il set completo dei dati: tutti i campi BASE, i campi COMUNI e i campi SPECIFICI della categoria stessa. 5. Il configuratore preme INVIO per terminare la visualizzazione e tornare al menu. Fine
Scenario alternativo	3a. Precondizione: La categoria inserita non è presente nell'archivio. Il sistema mostra un messaggio di errore e l'operazione si interrompe. Fine

Casi d'uso grafici



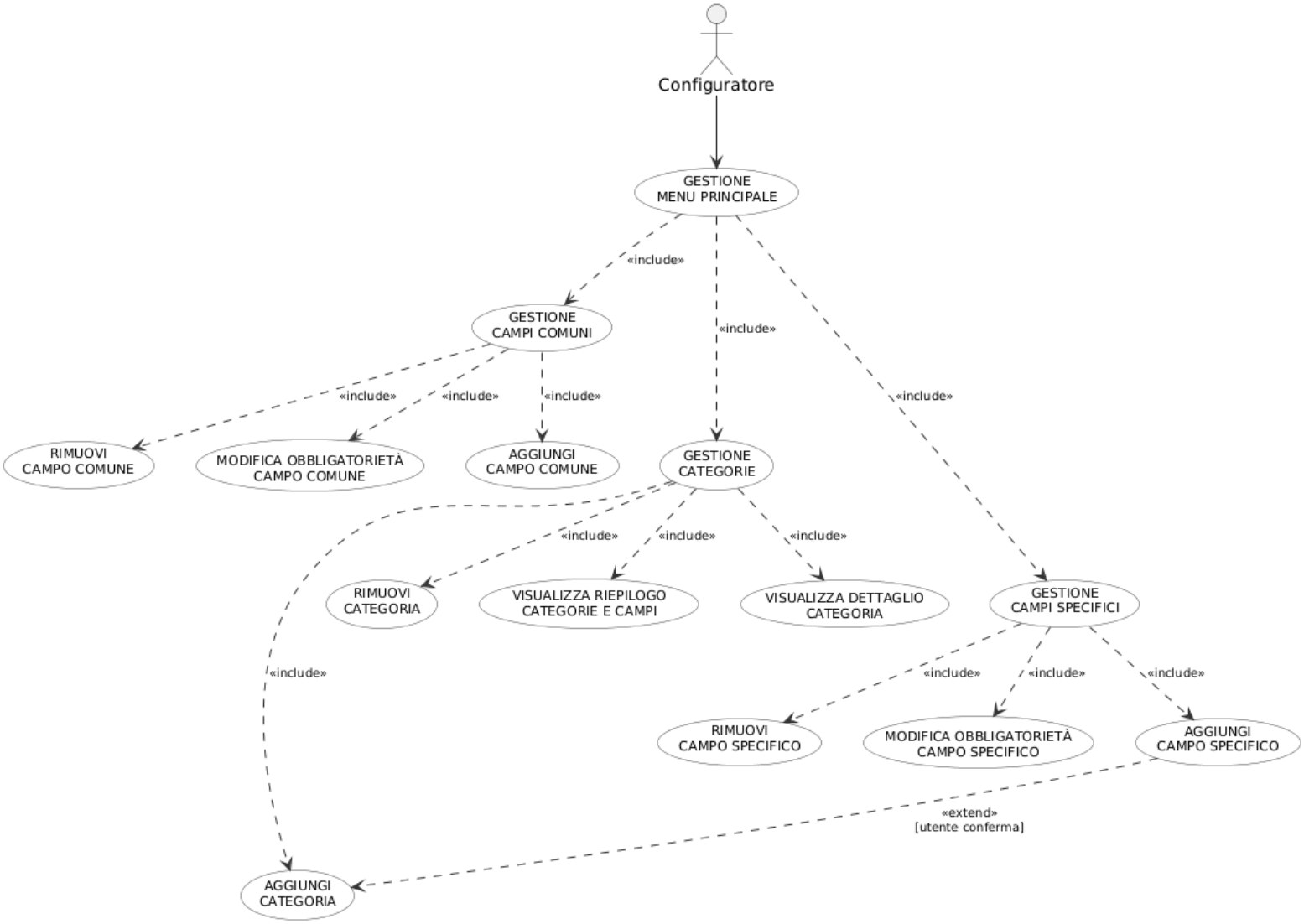
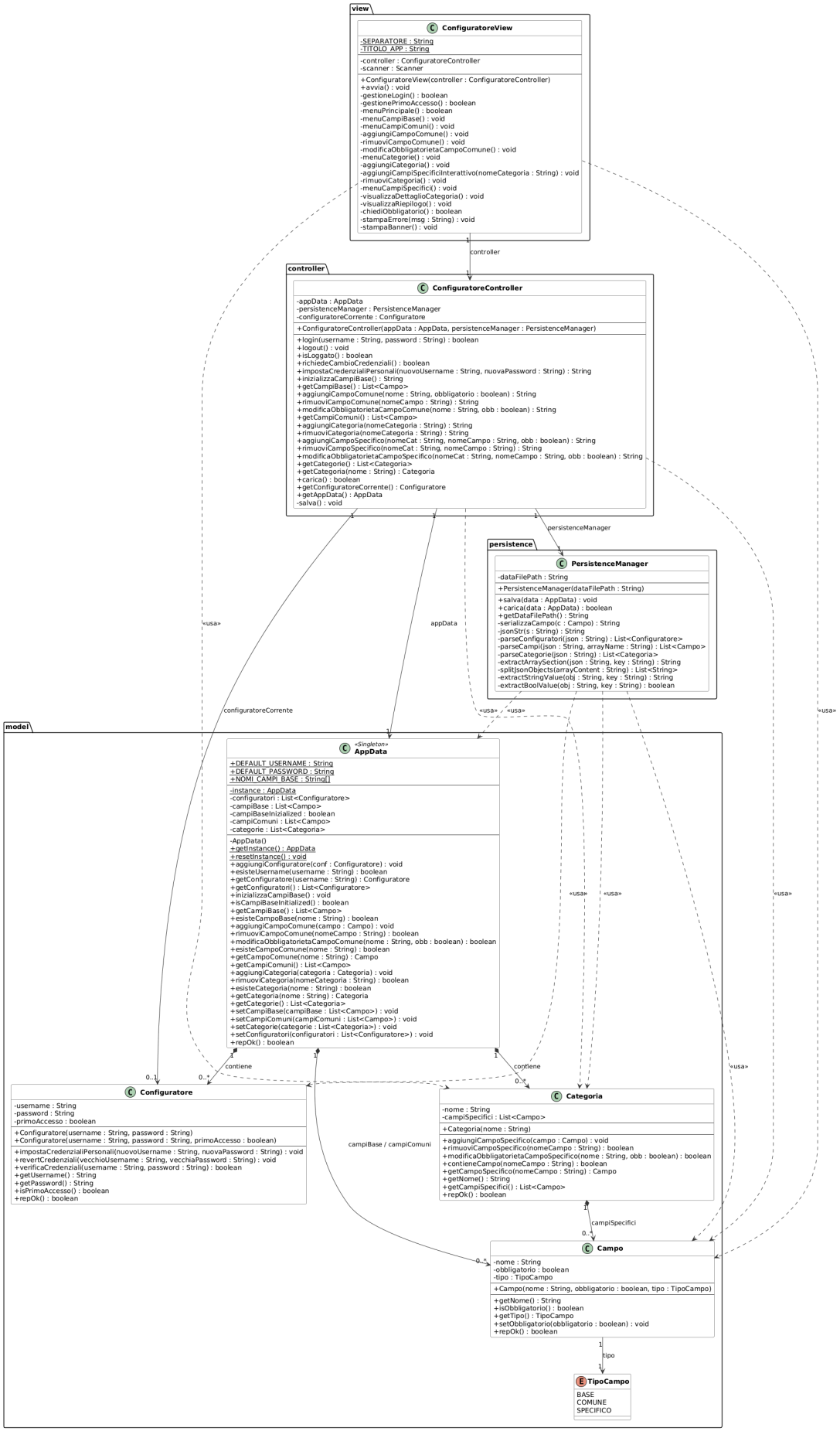


Diagramma delle classi


view
ConfiguratoreView
-SEPARATORE : String
-TITOLO_APP : String
-controller : ConfiguratoreController
-scanner : Scanner
+ConfiguratoreView(controller : ConfiguratoreController)
+avvia() : void
-gestioneLogin() : boolean
-gestionePrimoAccesso() : boolean
-menuPrincipale() : boolean
-menuCampiBase() : void
-menuCampiComuni() : void
-aggiungiCampoComune() : void
-rimuoviCampoComune() : void
-modificaObbligatorietaCampoComune() : void
-menuCategorie() : void
-aggiungiCategoria() : void
-aggiungiCampiSpecificiInterattivo(nomeCategoria : String) : void
-rimuoviCategoria() : void
-menuCampiSpecifici() : void
-visualizzaDettaglioCategoria() : void
-visualizzaRiepilogo() : void
-chiediObbligatorio() : boolean
-stampaErrore(msg : String) : void
-stampaBanner() : void
controller
ConfiguratoreController
-appData : AppData
-persistenceManager : PersistenceManager
-configuratoreCorrente : Configuratore
+ConfiguratoreController(appData : AppData, persistenceManager : PersistenceManager)
+login(username : String, password : String) : boolean
+logout() : void
+isLoggato() : boolean
+richiedeCambioCredenziali() : boolean
+impostaCredenzialiPersonali(nuovoUsername : String, nuovaPassword : String) : String
+inizializzaCampiBase() : String
+getCampiBase() : List<Campo>
+aggiungiCampoComune(nome : String, obbligatorio : boolean) : String
+rimuoviCampoComune(nomeCampo : String) : String
+modificaObbligatorietaCampoComune(nome : String, obb : boolean) : String
+getCampiComuni() : List<Campo>
+aggiungiCategoria(nomeCategoria : String) : String
+rimuoviCategoria(nomeCategoria : String) : String
+aggiungiCampoSpecifico(nomeCat : String, nomeCampo : String, obb : boolean) : String
+rimuoviCampoSpecifico(nomeCat : String, nomeCampo : String) : String
+modificaObbligatorietaCampoSpecifico(nomeCat : String, nomeCampo : String, obb : boolean) : String
+getCategorie() : List<Categoria>
+getCategoria(nome : String) : Categoria
+carica() : boolean
+getConfiguratoreCorrente() : Configuratore
+getAppData() : AppData
-salva() : void
persistenceManager
persistence
PersistenceManager
-dataFilePath : String
+PersistenceManager(dataFilePath : String)
+salva(data : AppData) : void
+carica(data : AppData) : boolean
+getDataFilePath() : String
-serializzaCampo(c : Campo) : String
-jsonStr(s : String) : String
-parseConfiguratori(json : String) : List<Configuratore>
-parseCampi(json : String, arrayName : String) : List<Campo>
-parseCategorie(json : String) : List<Categoria>
-extractArraySection(json : String, key : String) : String
-splitJsonObjects(arrayContent : String) : List<String>
-extractStringValue(obj : String, key : String) : String
-extractBoolValue(obj : String, key : String) : boolean
«usa»
appData
configuratoreCorrente
model
«Singleton»
AppData
+DEFAULT_USERNAME : String
+DEFAULT_PASSWORD : String
+NOMI_CAMPI_BASE : String[]
-instance : AppData
-configuratori : List<Configuratore>
-campiBase : List<Campo>
-campiBaseInizialized : boolean
-campiComuni : List<Campo>
-categorie : List<Categoria>
-AppData()
+getInstance() : AppData
+resetInstance() : void
+aggiungiConfiguratore(conf : Configuratore) : void
+esisteUsername(username : String) : boolean
+getConfiguratore(username : String) : Configuratore
+getConfiguratori() : List<Configuratore>
+inizializzaCampiBase() : void
+isCampiBaseInitialized() : boolean
+getCampiBase() : List<Campo>
+esisteCampoBase(nome : String) : boolean
+aggiungiCampoComune(campo : Campo) : void
+rimuoviCampoComune(nomeCampo : String) : boolean
+modificaObbligatorietaCampoComune(nome : String, obb : boolean) : boolean
+esisteCampoComune(nome : String) : boolean
+getCampoComune(nome : String) : Campo
+getCampiComuni() : List<Campo>
+aggiungiCategoria(categoria : Categoria) : void
+rimuoviCategoria(nomeCategoria : String) : boolean
+esisteCategoria(nome : String) : boolean
+getCategoria(nome : String) : Categoria
+getCategorie() : List<Categoria>
+setCampiBase(campiBase : List<Campo>) : void
+setCampiComuni(campiComuni : List<Campo>) : void
+setCategorie(categorie : List<Categoria>) : void
+setConfiguratori(configuratori : List<Configuratore>) : void
+repOk() : boolean
contiene
contiene
«usa»
«usa»
«usa»
«usa»
«usa»
«usa»
Configuratore
-username : String
-password : String
-primoAccesso : boolean
+Configuratore(username : String, password : String)
+Configuratore(username : String, password : String, primoAccesso : boolean)
+impostaCredenzialiPersonali(nuovoUsername : String, nuovaPassword : String) : void
+revertCredenziali(vecchioUsername : String, vecchiaPassword : String) : void
+verificaCredenziali(username : String, password : String) : boolean
+getUsername() : String
+getPassword() : String
+isPrimoAccesso() : boolean
+repOk() : boolean
campiBase / campiComuni
Categoria
-nome : String
-campiSpecifici : List<Campo>
+Categoria(nome : String)
+aggiungiCampoSpecifico(campo : Campo) : void
+rimuoviCampoSpecifico(nomeCampo : String) : boolean
+modificaObbligatorietaCampoSpecifico(nome : String, obb : boolean) : boolean
+contieneCampo(nomeCampo : String) : boolean
+getCampoSpecifico(nomeCampo : String) : Campo
+getNome() : String
+getCampiSpecifici() : List<Campo>
+repOk() : boolean
campiSpecifici
Campo
-nome : String
-obbligatorio : boolean
-tipo : TipoCampo
+Campo(nome : String, obbligatorio : boolean, tipo : TipoCampo)
+getNome() : String
+isObbligatorio() : boolean
+getTipo() : TipoCampo
+setObbligatorio(obbligatorio : boolean) : void
+repOk() : boolean
tipo
TipoCampo
BASE
COMUNE
SPECIFICO

CAPITOLO 2

Versione 2

Casi d'uso testuali modificati

Nome	**UC01 - Avvio e Login**
Attore	Configuratore
Scenario principale	Precondizione: L'applicazione è avviata. Trigger: L'utente seleziona "Configuratore" dal menu iniziale. 1. Il sistema mostra la schermata di login dedicata. 2. Il configuratore inserisce username e password. 3. Il sistema verifica la corrispondenza delle credenziali. 4. Il sistema autentica il configuratore. 5. Il sistema verifica se si tratta del primissimo accesso assoluto (credenziali di default). 6. «include» "UC02 - Imposta credenziali personali" (se primo accesso). 7. Il sistema mostra il menu principale con i contatori aggiornati: numero di proposte in sessione e numero di proposte aperte in bacheca. Postcondizione: Il configuratore è autenticato nel back-end. Fine
Scenario alternativo	2a. Precondizione: il configuratore digita "esci" come username. Il sistema interrompe l'operazione e torna al menu di selezione ruolo. Fine
Scenario alternativo	3a. Precondizione: le credenziali non sono valide. Il sistema mostra un errore e indica i tentativi rimasti. Torna al punto 2 (se tentativi < 3), altrimenti il sistema blocca l'accesso alla sessione. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	3b. Precondizione: nessun configuratore registrato nel sistema. Il sistema accetta le credenziali di default (admin/admin123), crea il profilo iniziale, imposta la necessità di cambio credenziali e prosegue al punto 6.

Nome	**UC03 - Logout**
Attore	Configuratore
Scenario principale	Precondizione: Configuratore autenticato. Trigger: Selezione "Logout". 1. Il configuratore seleziona Logout. 2. Il sistema scarta le proposte non pubblicate della sessione corrente (stato BOZZA o VALIDA). 3. Se erano presenti proposte non pubblicate, il sistema avvisa il configuratore. 4. Il sistema termina la sessione corrente. 5. Il sistema chiede se accedere con un altro account. 6. Sì: ritorna a UC01. 7. No: termina l'applicazione. Fine
Scenario alternativo	5a. Precondizione: Il configuratore inserisce una risposta non valida (diversa da "s" o "n"). Il sistema mostra un messaggio di errore testuale e ripropone la domanda.

Casi d'uso testuali nuovi

Nome	**UC15 - Crea proposta**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato ed esiste almeno una categoria. Trigger: Selezione "Nuova proposta" dal menu gestionale. 1. Il sistema mostra l'elenco delle categorie disponibili. 2. Il configuratore inserisce il nome della categoria desiderata. 3. Il sistema verifica l'esistenza della categoria. 4. Il sistema cattura lo snapshot dei campi (base, comuni e specifici) rendendoli indipendenti da future modifiche alla configurazione. 5. Il sistema assegna un ID univoco progressivo estratto dal contatore globale. 6. Il sistema aggiunge la proposta alla sessione corrente in stato BOZZA. 7. «include» "UC16 - Compila campi proposta". 8. Il sistema mostra il riepilogo e, se in stato VALIDA, offre la pubblicazione immediata (estende UC17). Fine
Scenario alternativo	1a. Precondizione: l'archivio categorie è vuoto. Il sistema mostra l'errore "Nessuna categoria disponibile" e impedisce la creazione. Fine
Scenario alternativo	3a. Precondizione: categoria non trovata in base all'input dell'utente. Il sistema mostra un errore e torna al menu proposte. Fine

Nome	**UC16 - Compila / Modifica campi proposta**
Attore	Configuratore
Scenario principale	Precondizione: Esiste almeno una proposta in sessione. Trigger: Estensione da UC15 o selezione "Continua compilazione". 1. Il sistema presenta i campi dello snapshot uno alla volta. 2. Per ogni campo, il sistema mostra l'etichetta [OBB/FAC], il formato richiesto e il valore attuale. 3. Il configuratore inserisce un valore o preme INVIO per mantenere il valore attuale. 4. Il sistema valida l'input e ricalcola automaticamente lo stato (BOZZA/VALIDA). 5. Il sistema verifica i vincoli temporali: Termine iscrizione $>$ oggi; Data inizio $\geq$ Termine + 2gg; Data fine $\geq$ Data inizio; Formato Ora (HH:MM). 6. Il sistema mostra il riepilogo finale e l'elenco degli errori se lo stato è BOZZA. Fine
Scenario alternativo	4a. Precondizione: formato data (gg/mm/aaaa) o ora (HH:MM) errato. Il sistema mostra un errore specifico e non aggiorna il valore del campo. Prosegue con il campo successivo.

Nome	**UC17 - Pubblica proposta in bacheca**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato e c'è almeno una proposta VALIDA in sessione. Trigger: Selezione "Pubblica proposta in bacheca". 1. Il sistema mostra l'elenco delle proposte in sessione attualmente VALIDE. 2. Il configuratore inserisce l'ID della proposta da pubblicare. 3. Il sistema cattura la data odierna. 4. Il sistema imposta lo stato della proposta su APERTA e registra la data nello storico. 5. Il sistema sposta la proposta dalla sessione all'Archivio del configuratore. 6. Il sistema salva le modifiche in modo persistente. 7. Il sistema mostra conferma dell'operazione. Postcondizione: La proposta ha stato APERTA, è salvata nell'archivio ed è visibile ai fruitori. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	1a. Precondizione: nessuna proposta in sessione è in stato VALIDA. Il sistema mostra un messaggio di errore e l'operazione termina. Fine
Scenario alternativo	2a. Precondizione: l'input non è numerico o non corrisponde a un ID presente in elenco. Il sistema mostra un errore e l'operazione termina. Fine

Nome	**UC18 - Elimina proposta dalla sessione**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una proposta in sessione (stato BOZZA o VALIDA). Trigger: Il configuratore seleziona "Elimina proposta" nel menu proposte. 1. Il sistema mostra l'elenco delle proposte in sessione. 2. Il configuratore inserisce l'ID della proposta da eliminare. 3. Il sistema chiede conferma esplicita prima dell'eliminazione. 4. Il configuratore conferma. 5. Il sistema rimuove definitivamente la proposta dalla sessione. 6. Il sistema mostra conferma dell'avvenuta operazione. Fine
Scenario alternativo	2a. Precondizione: l'ID inserito non è valido o non corrisponde a una proposta presente in sessione. Il sistema mostra un messaggio di errore e interrompe l'operazione. Fine
Scenario alternativo	4a. Precondizione: il configuratore annulla l'operazione (nega la conferma). Il sistema interrompe l'operazione senza apportare modifiche e torna al menu. Fine

Nome	**UC19 - Visualizza bacheca**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Selezione "Visualizza bacheca". 1. Il configuratore richiede la visualizzazione della bacheca. 2. Il sistema recupera tutte le proposte con stato APERTA. 3. Il sistema raggruppa le proposte per categoria esistente. 4. Il sistema mostra per ogni proposta: ID, Data pubblicazione, Creatore e i valori di tutti i campi compilati (omettendo i campi facoltativi lasciati vuoti). 5. Il configuratore preme INVIO per terminare la visualizzazione. Postcondizione: Nessuna modifica allo stato del sistema. Fine
Scenario alternativo	2a. Precondizione: non ci sono proposte in stato APERTA nell'archivio. Il sistema mostra il messaggio "(Nessuna proposta aperta in bacheca)". Fine
Scenario alternativo	3a. Precondizione: la categoria originaria di una proposta è stata rimossa dopo la pubblicazione. Il sistema visualizza la proposta raggruppandola sotto l'etichetta "(categoria rimossa)". Fine

Casi d'uso grafici

Colore	Significato
Verde	Novità
Giallo	Modifica
Bianco	Invariato

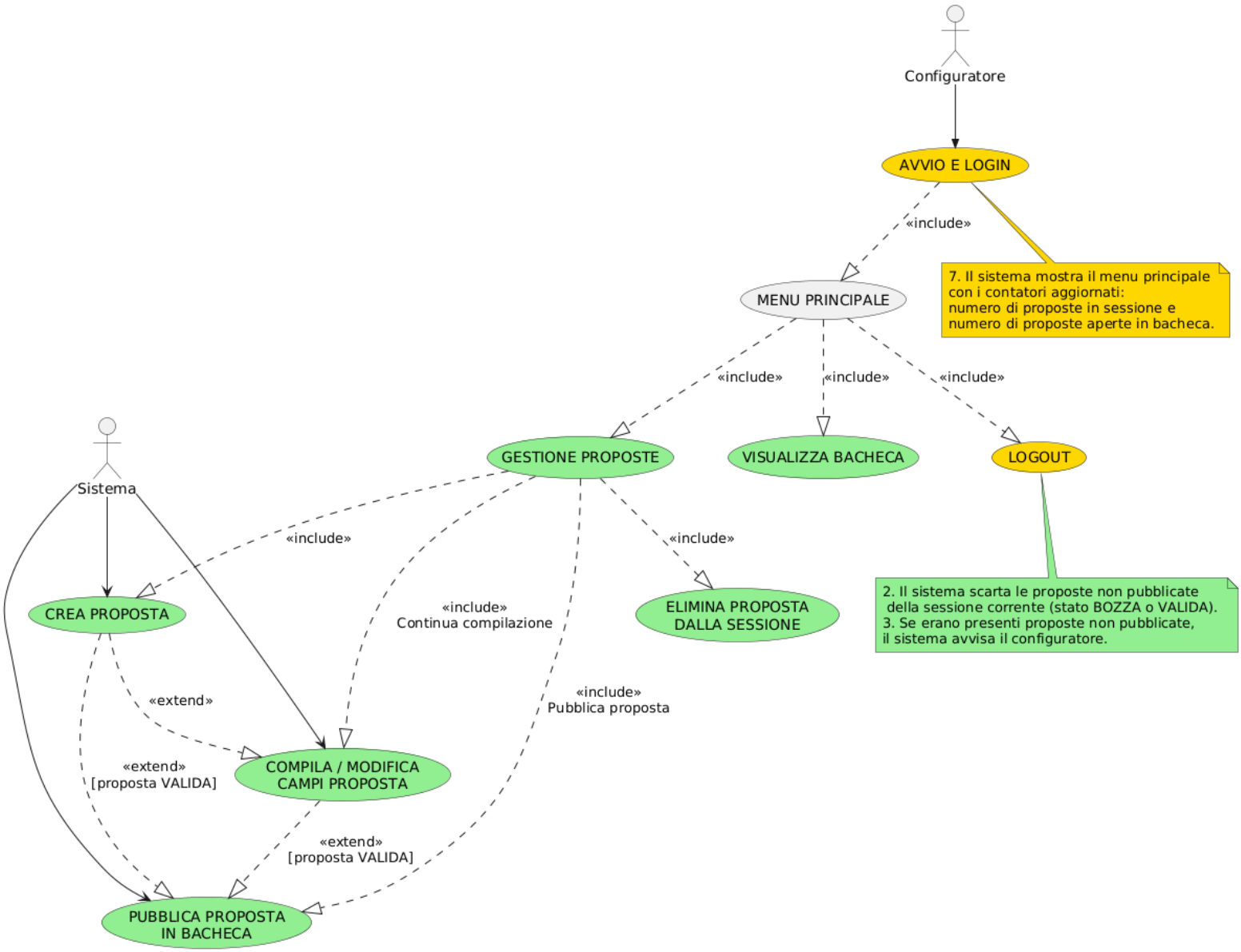
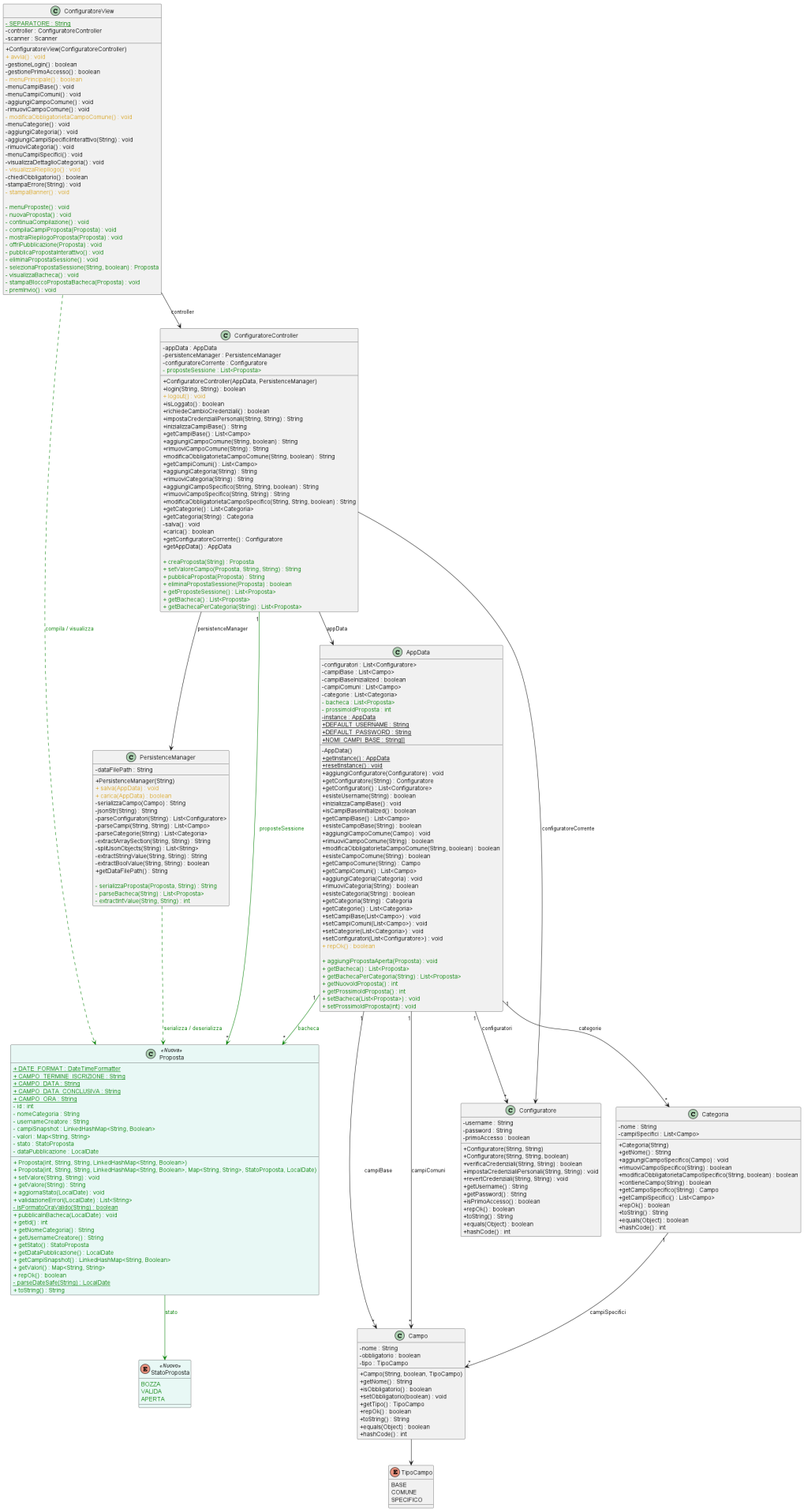


Diagramma delle classi


ConfiguratoreView
- SEPARATORE : String
-controller : ConfiguratoreController
-scanner : Scanner
+ConfiguratoreView(ConfiguratoreController)
+ avvia() : void
-gestioneLogin() : boolean
-gestionePrimoAccesso() : boolean
- menuPrincipale() : boolean
-menuCampiBase() : void
-menuCampiComuni() : void
-aggiungiCampoComune() : void
-rimuoviCampoComune() : void
- modificaObbligatorietaCampoComune() : void
-menuCategorie() : void
-aggiungiCategoria() : void
-aggiungiCampiSpecificiInterattivo(String) : void
-rimuoviCategoria() : void
-menuCampiSpecifici() : void
-visualizzaDettaglioCategoria() : void
- visualizzaPropriEq() : void
-chiediObbligatorio() : boolean
-stampaErrore(String) : void
- stampaBanner() : void
- menuProposte() : void
- nuovaProposta() : void
- continuaCompilazione() : void
- compilaCampiProposta(Proposta) : void
- mostraRiepilogoProposta(Proposta) : void
- offriPubblicazione(Proposta) : void
- pubblicaPropostaInterattivo() : void
- eliminaPropostaSessione() : void
- selezionaPropostaSessione(String, boolean) : Proposta
- visualizzaBacheca() : void
- stampaBloccoPropostaBacheca(Proposta) : void
- premiInvio() : void
controller
ConfiguratoreController
-appData : AppData
-persistenceManager : PersistenceManager
-configuratoreCorrente : Configuratore
- proposteSessione : List<Proposta>
+ConfiguratoreController(AppData, PersistenceManager)
+login(String, String) : boolean
+ logout() : void
+isLoggato() : boolean
+richiedeCambioCredenziali() : boolean
+impostaCredenzialiPersonali(String, String) : String
+inizializzaCampiBase() : String
+getCampiBase() : List<Campo>
+aggiungiCampoComune(String, boolean) : String
+rimuoviCampoComune(String) : String
+modificaObbligatorietaCampoComune(String, boolean) : String
+getCampiComuni() : List<Campo>
+aggiungiCategoria(String) : String
+rimuoviCategoria(String) : String
+aggiungiCampoSpecifico(String, String, boolean) : String
+rimuoviCampoSpecifico(String, String) : String
+modificaObbligatorietaCampoSpecifico(String, String, boolean) : String
+getCategorie() : List<Categoria>
+getCategoria(String) : Categoria
-salva() : void
+carica() : boolean
+getConfiguratoreCorrente() : Configuratore
+getAppData() : AppData
+ creaProposta(String) : Proposta
+ setValoreCampo(Proposta, String, String) : String
+ pubblicaPropostaProposta() : String
+ eliminaPropostaSessione(Proposta) : boolean
+ getProposteSessione() : List<Proposta>
+ getBacheca() : List<Proposta>
+ getBachecaPerCategoria(String) : List<Proposta>
compila / visualizza
persistenceManager
1
appData
configuratoreCorrente
AppData
-configuratori : List<Configuratore>
-campiBase : List<Campo>
-campiBaseInitialized : boolean
-campiComuni : List<Campo>
-categorie : List<Categoria>
- bacheca : List<Proposta>
- prossimoIdProposta : int
-instance : AppData
+DEFAULT_USERNAME : String
+DEFAULT_PASSWORD : String
+NOMI_CAMPI_BASE : String[]
-AppData()
+getInstance() : AppData
+resetInstance() : void
+aggiungiConfiguratore(Configuratore) : void
+getConfiguratore(String) : Configuratore
+getConfiguratori() : List<Configuratore>
+esisteUsername(String) : boolean
+inizializzaCampiBase() : void
+isCampiBaseInitializzo() : boolean
+getCampiBase() : List<Campo>
+esisteCampoBase(String) : boolean
+aggiungiCampoComune(Campo) : void
+rimuoviCampoComune(String) : boolean
+modificaObbligatorietaCampoComune(String, boolean) : boolean
+esisteCampoComune(String) : boolean
+getCampoComune(String) : Campo
+getCampiComuni() : List<Campo>
+aggiungiCategoria(Categoria) : void
+rimuoviCategoria(String) : boolean
+esisteCategoria(String) : boolean
+getCategoria(String) : Categoria
+getCategorie() : List<Categoria>
+setCampiBase(List<Campo>) : void
+setCampiComuni(List<Campo>) : void
+setCategorie(List<Categoria>) : void
+setConfiguratori(List<Configuratore>) : void
+ repOk() : boolean
+ aggiungiPropostaAperta(Proposta) : void
+ getBacheca() : List<Proposta>
+ getBachecaPerCategoria(String) : List<Proposta>
+ getNuovoIdProposta() : int
+ getProssimoIdProposta() : int
+ setBacheca(List<Proposta>) : void
+ setProssimoIdProposta(int) : void
PersistenceManager
-dataFilePath : String
+PersistenceManager(String)
+ salvaAppData() : void
+ carica(AppData) : boolean
-serializzaCampo(Campo) : String
-jsonStr(String) : String
-parseConfiguratori(String) : List<Configuratore>
-parseCampi(String, String) : List<Campo>
-parseCategorie(String) : List<Categoria>
-extractArraySection(String, String) : String
-splitJsonObjects(String) : List<String>
-extractStringValue(String, String) : String
-extractBoolValue(String, String) : boolean
+getDataFilePath() : String
- serializzaProposta(Proposta, String) : String
- parseBacheca(String) : List<Proposta>
- extractIntValue(String, String) : int
proposteSessione
serializza / deserializza
bacheca
configuratori
categorie
«Nuova»
Proposta
+ DATE_FORMAT : DateTimeFormatter
+ CAMPO_TERMINE_ISCRIZIONE : String
+ CAMPO_DATA : String
+ CAMPO_DATA_CONCLUSIVA : String
+ CAMPO_ORA : String
- id : int
- nomeCategoria : String
- usernameCreatore : String
- campiSnapshot : LinkedHashMap<String, Boolean>
- valori : Map<String, String>
- stato : StatoProposta
- dataPubblicazione : LocalDate
+ Proposta(int, String, String, LinkedHashMap<String, Boolean>)
+ Proposta(int, String, String, LinkedHashMap<String, Boolean>, Map<String, String>, StatoProposta, LocalDate)
+ setValore(String, String) : void
+ getValore(String) : String
+ aggiornaStato(LocalDate) : void
+ validazioneErrori(LocalDate) : List<String>
- isFormatoOraValido(String) : boolean
+ pubblicaInBacheca(LocalDate) : void
+ getId() : int
+ getNomeCategoria() : String
+ getUsernameCreatore() : String
+ getStato() : StatoProposta
+ getDataPubblicazione() : LocalDate
+ getCampiSnapshot() : LinkedHashMap<String, Boolean>
+ getValori() : Map<String, String>
+ repOk() : boolean
- parseDateSafe(String) : LocalDate
+ toString() : String
stato
«Nuova»
StatoProposta
BOZZA
VALIDA
APERTA
Configuratore
-username : String
-password : String
-primoAccesso : boolean
+Configuratore(String, String)
+Configuratore(String, String, boolean)
+verificaCredenziali(String, String) : boolean
+impostaCredenzialiPersonali(String, String) : void
+revertCredenziali(String, String) : void
+getUsername() : String
+getPassword() : String
+isPrimoAccesso() : boolean
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
Categoria
-nome : String
-campiSpecifici : List<Campo>
+Categoria(String)
+getNome() : String
+aggiungiCampoSpecifico(Campo) : void
+rimuoviCampoSpecifico(String) : boolean
+modificaObbligatorietaCampoSpecifico(String, boolean) : boolean
+contieneCampo(String) : boolean
+getCampoSpecifico(String) : Campo
+getCampiSpecifici() : List<Campo>
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
campiBase
campiComuni
campiSpecifici
Campo
-nome : String
-obbligatorio : boolean
-tipo : TipoCampo
+Campo(String, boolean, TipoCampo)
+getNome() : String
+isObbligatorio() : boolean
+setObbligatorio(boolean) : void
+getTipo() : TipoCampo
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
TipoCampo
BASE
COMUNE
SPECIFICO

CAPITOLO 3

Versione 3

Casi d'uso testuali modificati

Nome	**UC01 - Avvio e Login**
Attore	Configuratore
Scenario principale	Precondizione: L'applicazione è avviata. Transizioni automatiche (UC20) completate dal sistema. Trigger: L'utente seleziona "Configuratore" dal menu iniziale. 1. Il sistema mostra la schermata di login dedicata. 2. Il configuratore inserisce username e password. 3. Il sistema verifica la corrispondenza delle credenziali. 4. Il sistema autentica il configuratore. 5. Il sistema verifica se si tratta del primissimo accesso assoluto (credenziali di default). 6. «include» "UC02 - Imposta credenziali personali" (se primo accesso). 7. Il sistema mostra il menu principale aggiornato coi contatori in tempo reale. Postcondizione: Il configuratore è autenticato nel back-end. Fine
Scenario alternativo	2a. Precondizione: il configuratore digita "esci" come username. Il sistema interrompe l'operazione e torna al menu di selezione ruolo. Fine
Scenario alternativo	3a. Precondizione: le credenziali non sono valide. Il sistema mostra un errore e indica i tentativi rimasti. Torna al punto 2 (se tentativi < 3), altrimenti il sistema blocca l'accesso alla sessione. Fine

Nome	**UC02 - Imposta credenziali personali**
Attore	Configuratore
Scenario principale	Precondizione: Login effettuato con credenziali di default (primo accesso). Trigger: Il sistema rileva la necessità di cambiare le credenziali. 1. Il sistema richiede di scegliere le credenziali personali. 2. Il configuratore inserisce il nuovo username. 3. Il sistema verifica che lo username non coincida con la parola riservata "esci". 4. Il configuratore inserisce la nuova password. 5. Il configuratore inserisce la password per conferma. 6. Il sistema verifica che le due password coincidano. 7. Il sistema verifica che username e password non siano stringhe vuote/blank. 8. Il sistema verifica l'unicità globale dello username (tra Configuratori e Fruitori). 9. Il sistema aggiorna le credenziali in memoria. 10. Il sistema salva i dati in modo persistente. 11. Il sistema mostra la conferma e accede al menu. Postcondizione: Le credenziali di default sono sostituite e salvate in modo permanente. Fine
Scenario alternativo	2a. Precondizione: il configuratore digita "annulla" nel campo username. Il sistema annulla l'operazione ed esegue il logout automatico. Fine
Scenario alternativo	3a. Precondizione: lo username inserito è "esci". Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	4a. Precondizione: il configuratore digita "annulla" nel campo password. Il sistema annulla l'operazione ed esegue il logout automatico. Fine
Scenario alternativo	6a. Precondizione: le password non coincidono. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	7a. Precondizione: campi lasciati vuoti. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	8a. Precondizione: username già in uso nel sistema. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	10a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback in memoria, mostra un errore ed esegue il logout. Fine

Nome	**UC17 - Pubblica proposta in bacheca**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato e c'è almeno una proposta VALIDA in sessione. Trigger: Selezione "Pubblica proposta in bacheca". 1. Il sistema mostra l'elenco delle proposte in sessione attualmente VALIDE. 2. Il configuratore inserisce l'ID della proposta da pubblicare. 3. Il sistema cattura la data odierna. 4. Il sistema ricalcola la validità della proposta in base alla data odierna. 5. Il sistema imposta lo stato su APERTA e registra la data nello storico. 6. Il sistema sposta la proposta dalla sessione all'Archivio globale. 7. Il sistema salva le modifiche in modo persistente. 8. Il sistema mostra conferma dell'operazione. Postcondizione: La proposta ha stato APERTA, è salvata nell'archivio ed è visibile ai fruitori. Fine
Scenario alternativo	1a. Precondizione: nessuna proposta in sessione è in stato VALIDA. Il sistema mostra un errore. Fine
Scenario alternativo	2a. Precondizione: l'input non è numerico o non corrisponde a una proposta in elenco. Il sistema mostra un errore. Fine
Scenario alternativo	4a. Precondizione: la validità è scaduta a causa del cambio di data (es. superata mezzanotte). Il sistema nega la pubblicazione e mostra l'elenco degli errori. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback completo riportando la proposta allo stato VALIDA in sessione. Fine

Nome	**UC19 - Visualizza bacheca Configuratore**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Selezione "Visualizza bacheca". 1. Il configuratore richiede la visualizzazione della bacheca. 2. Il sistema recupera tutte le proposte con stato APERTA. 3. Il sistema raggruppa dinamicamente le proposte per categoria (inclusa la voce "(categoria rimossa)" per le eventuali proposte orfane). 4. Il sistema mostra per ogni proposta: ID, dettagli, valori dei campi e il numero di iscritti attuali rispetto al massimo consentito. 5. Il configuratore preme INVIO per terminare. Postcondizione: Nessuna modifica allo stato del sistema. Fine
Scenario alternativo	2a. Precondizione: non ci sono proposte APERTE in archivio. Il sistema mostra il messaggio "(Nessuna proposta aperta in bacheca)". Fine
Scenario alternativo	3a. Precondizione: categoria di una proposta rimossa dopo la pubblicazione. Il sistema visualizza la proposta sotto l'etichetta "(categoria rimossa)". Fine

Casi d'uso testuali nuovi

Nome	**UC20 - Esecuzione transizioni automatiche di stato**
Attore	Sistema
Scenario principale	Precondizione: L'applicazione viene avviata. L'archivio contiene proposte. Trigger: Inizializzazione del sistema. 1. Il sistema analizza l'archivio delle proposte. 2. Per ogni proposta APERTA con "Termine ultimo" superato, il sistema verifica se iscritti $\geq$ minimo richiesto. 3. Il sistema transita la proposta in stato CONFERMATA. 4. Il sistema genera una notifica di conferma per ogni aderente iscritto. 5. Per ogni proposta CONFERMATA con "Data conclusiva" superata, il sistema la transita in stato CONCLUSA. 6. Il sistema salva i nuovi stati e le notifiche in modo persistente. 7. Il sistema mostra a schermo il numero totale di transizioni effettuate. Postcondizione: Stati aggiornati alla data odierna e notifiche distribuite. Fine
Scenario alternativo	2a. Precondizione: la proposta APERTA è scaduta ma gli iscritti sono insufficienti (o il campo "Numero massimo" è corrotto/negativo). Il sistema transita la proposta in ANNULLATA e genera una notifica di annullamento per ogni aderente. Torna al punto 5
Scenario alternativo	6a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback ricaricando i dati precedenti dal disco per evitare la generazione di notifiche duplicate ai riavvii successivi. Fine

Nome	**UC21 - Registrazione Fruitore**
Attore	Fruitore
Scenario principale	Precondizione: L'utente non è autenticato. Trigger: Selezione "Registrazione (nuovo account)" dal menu di accesso. 1. Il sistema mostra la schermata di registrazione. 2. Il fruitore inserisce lo username desiderato. 3. Il fruitore inserisce la password. 4. Il fruitore reinserisce la password per conferma. 5. Il sistema verifica che le due password coincidano. 6. Il sistema verifica che username e password non siano vuoti. 7. Il sistema verifica l'unicità globale dello username. 8. Il sistema crea il profilo Fruitore in memoria. 9. Il sistema salva i dati in modo persistente. 10. Il sistema autentica automaticamente il fruitore e mostra il benvenuto. Postcondizione: Il fruitore è registrato, autenticato e con uno spazio notifiche vuoto. Fine
Scenario alternativo	2a. Precondizione: il fruitore digita "annulla" come username. Il sistema interrompe l'operazione e torna al menu. Fine
Scenario alternativo	3a. Precondizione: il fruitore digita "annulla" come password. Il sistema interrompe l'operazione e torna al menu. Fine
Scenario alternativo	5a. Precondizione: le password non coincidono. Il sistema mostra l'errore e interrompe l'operazione. Fine
Scenario alternativo	6a. Precondizione: campi vuoti o composti da soli spazi. Il sistema mostra l'errore e interrompe l'operazione. Fine
Scenario alternativo	7a. Precondizione: username già in uso (globale). Il sistema mostra l'errore e interrompe l'operazione. Fine
Scenario alternativo	9a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove il profilo dalla memoria (rollback), mostra l'errore e interrompe l'operazione. Fine

Nome	**UC22 - Login Fruitore**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore possiede un account e non è autenticato. Trigger: Selezione "Login (account esistente)". 1. Il sistema mostra la schermata di login. 2. Il fruitore inserisce username e password. 3. Il sistema verifica la corrispondenza delle credenziali. 4. Il sistema autentica il fruitore. 5. Il sistema conteggia le notifiche presenti nello spazio personale e mostra il benvenuto con il relativo avviso. Postcondizione: Il fruitore è autenticato nel sistema. Fine
Scenario alternativo	2a. Precondizione: il fruitore digita "annulla" nel campo username. Il sistema annulla l'operazione e torna al menu. Fine
Scenario alternativo	3a. Precondizione: le credenziali sono errate (incluso inserire "annulla" come password). Il sistema mostra l'errore e i tentativi rimasti. Torna al punto 2 (se tentativi < 3), altrimenti il sistema blocca l'accesso. Fine

Nome	**UC23 - Visualizza Bacheca (Fruitore)**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato. Trigger: Selezione "Visualizza bacheca". 1. Il fruitore richiede la visualizzazione. 2. Il sistema recupera tutte le proposte in stato APERTA. 3. Il sistema raggruppa le proposte per categoria. 4. Il sistema mostra dettagli, posti occupati, scadenze e tutti i campi informativi per ogni proposta. 5. Il sistema evidenzia con l'etichetta [ISCRITTO] le iniziative a cui l'utente loggato ha già aderito. 6. Il fruitore preme INVIO per terminare la consultazione. Postcondizione: Nessuna modifica allo stato del sistema. Fine
Scenario alternativo	2a. Precondizione: nessuna proposta APERTA trovata nell'archivio. Il sistema comunica "(Nessuna proposta aperta in bacheca)". Fine

Nome	**UC24 - Iscrizione a proposta**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato. Trigger: Selezione "Aderisci a una proposta". 1. Il fruitore avvia la procedura di iscrizione. 2. Il sistema estrae le proposte APERTE dall'archivio. 3. Il sistema verifica che ci sia almeno una proposta APERTA nella bacheca globale. 4. Il sistema filtra le proposte escludendo quelle a cui l'utente è già iscritto. 5. Il sistema verifica che la lista filtrata non sia vuota. 6. Il fruitore inserisce l'ID della proposta scelta. 7. Il sistema verifica che l'input sia numerico e diverso da "0". 8. Il sistema individua la proposta e verifica che sia in stato APERTA. 9. Il sistema verifica che la data odierna sia $\leq$ al "Termine ultimo di iscrizione". 10. Il sistema verifica che ci siano posti disponibili. 11. Il sistema aggiunge lo username del fruitore alla lista degli aderenti. 12. Il sistema salva i dati in modo persistente. 13. Il sistema conferma l'avvenuta iscrizione all'utente. Postcondizione: L'utente è iscritto e i posti disponibili scalano. Fine
Scenario alternativo	3a. Precondizione: l'archivio globale non contiene alcuna proposta in stato APERTA. Il sistema lo comunica all'utente e termina l'operazione. Fine
Scenario alternativo	5a. Precondizione: il fruitore è già iscritto a tutte le proposte APERTE (lista filtrata vuota). Il sistema lo comunica all'utente e termina l'operazione. Fine
Scenario alternativo	7a. Precondizione: il fruitore digita "0" per annullare l'inserimento. Il sistema annulla l'operazione. Fine
Scenario alternativo	7b. Precondizione: l'input inserito non è un numero intero. Il sistema mostra il messaggio "ID non valido". Fine
Scenario alternativo	8a. Precondizione: l'ID non corrisponde a nessuna proposta o la proposta non è più APERTA. Il sistema mostra un messaggio di errore. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	9a. Precondizione: il termine di iscrizione è scaduto rispetto alla data odierna. Il sistema blocca l'iscrizione e mostra un messaggio di errore. Fine
Scenario alternativo	10a. Precondizione: i posti sono esauriti (o il campo numero partecipanti è corrotto). Il sistema blocca l'iscrizione e mostra un messaggio di errore. Fine
Scenario alternativo	12a. Precondizione: Errore di salvataggio (I/O). Il sistema annulla l'iscrizione in memoria (rollback) e mostra un messaggio di errore. Fine

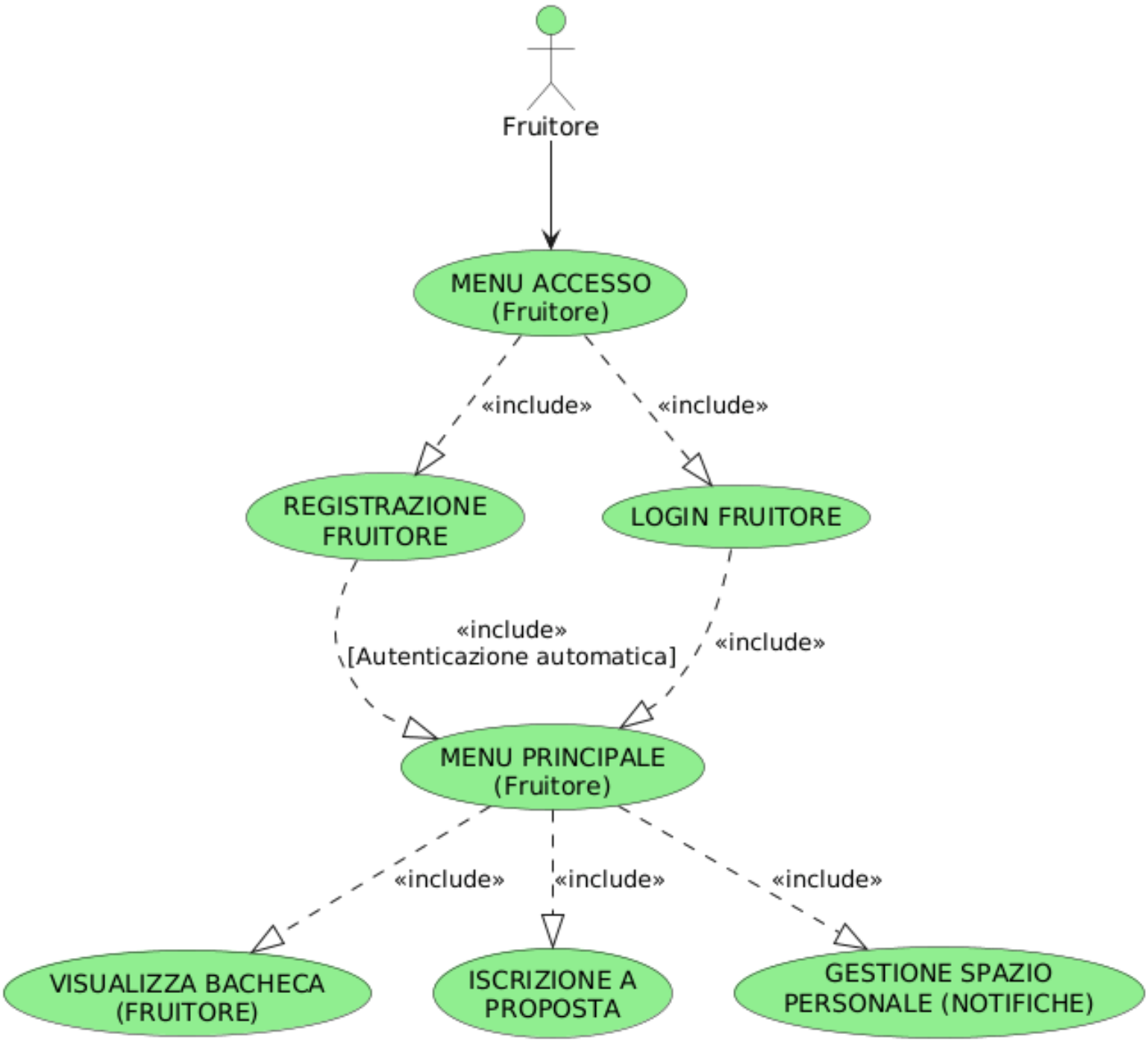
Nome	**UC25 - Gestione Spazio Personale (Notifiche)**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato. Trigger: Selezione "Spazio personale". 1. Il fruitore accede allo spazio personale. 2. Il sistema mostra l'elenco delle notifiche e le opzioni del menu ("c", "t", "0"). 3. Il fruitore inserisce l'opzione "c" (Cancella singola notifica). 4. Il sistema verifica che la lista delle notifiche non sia vuota. 5. Il sistema richiede l'inserimento dell'ID della notifica. 6. Il fruitore inserisce l'ID. 7. Il sistema individua e rimuove la notifica dalla memoria. 8. Il sistema salva i dati in modo persistente. 9. Il sistema conferma la cancellazione all'utente. Torna al punto 2
Scenario alternativo	2a. Precondizione: la lista delle notifiche dell'utente è vuota. Il sistema stampa "(Nessuna notifica)" ma mostra comunque il menu operativo. Prosegue al punto 3
Scenario alternativo	3a. Precondizione: il fruitore seleziona "t" (Cancellazione totale). Il sistema verifica che la lista non sia vuota, chiede conferma (s/n). L'utente risponde "s". Il sistema svuota la lista in memoria. Salta al punto 8
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	3b. Precondizione: il fruitore seleziona "t", ma annulla la conferma (risponde "n"). Il sistema annulla l'operazione. Torna al punto 2
Scenario alternativo	3c. Precondizione: il fruitore seleziona "0. Torna al menu principale". Il sistema chiude lo spazio personale e torna al menu precedente. Fine
Scenario alternativo	3d. Precondizione: il fruitore inserisce un'opzione non valida (es. lettera errata). Il sistema mostra l'errore "Scelta non valida". Torna al punto 2
Scenario alternativo	4a. Precondizione: il fruitore seleziona "c" o "t", ma la lista delle notifiche è vuota. Il sistema mostra l'errore "Nessuna notifica da cancellare". Torna al punto 2
Scenario alternativo	7a. Precondizione: l'ID inserito non è numerico o non corrisponde ad alcuna notifica in memoria. Il sistema mostra un errore. Torna al punto 2
Scenario alternativo	8a. Precondizione: Errore di salvataggio (I/O). Il sistema ripristina in memoria la lista originale intatta (rollback) e mostra un errore. Torna al punto 2

Nome	**UC26 - Visualizza Archivio Proposte**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Selezione "Visualizza archivio proposte". 1. Il configuratore avvia la richiesta di visualizzazione. 2. Il sistema recupera l'intero archivio (escludendo gli stati interni BOZZA e VALIDA). 3. Il sistema raggruppa dinamicamente le proposte in base allo stato (APERTA, CONFERMATA, ANNULLATA, CONCLUSA, RITIRATA). 4. Il sistema mostra i dettagli anagrafici, i posti occupati, la lista degli aderenti e lo storico completo dei passaggi di stato per ogni proposta. 5. Il configuratore preme INVIO per terminare. Postcondizione: Nessuna modifica allo stato del sistema. Fine
Scenario alternativo	2a. Precondizione: l'archivio è vuoto. Il sistema segnala "(Nessuna proposta nell'archivio)" e termina l'operazione. Fine

Casi d'uso grafici

Colore	Significato
Verde	Novità



Colore	Significato
Verde	Novità
Giallo	Modifica
Bianco	Invariato

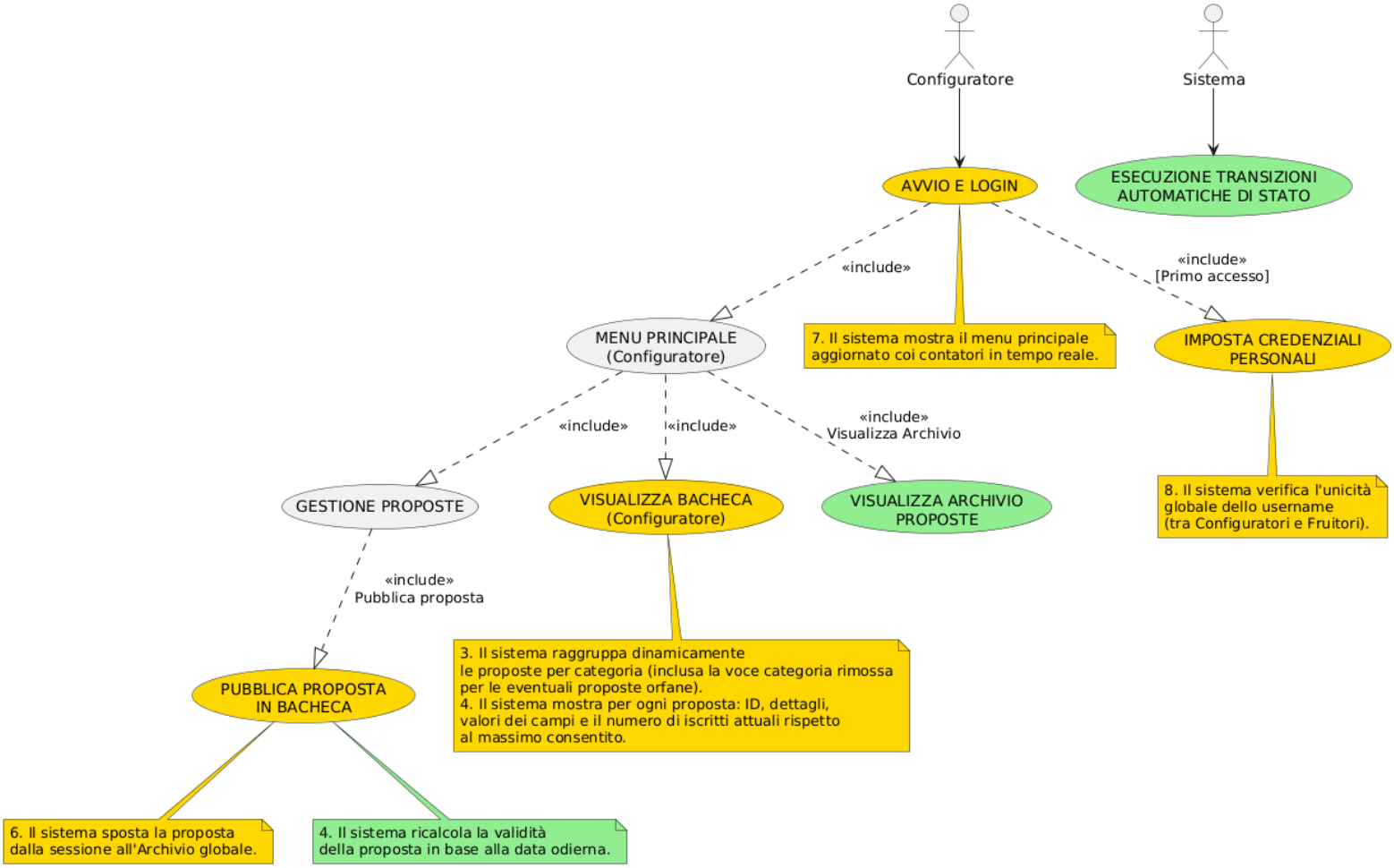
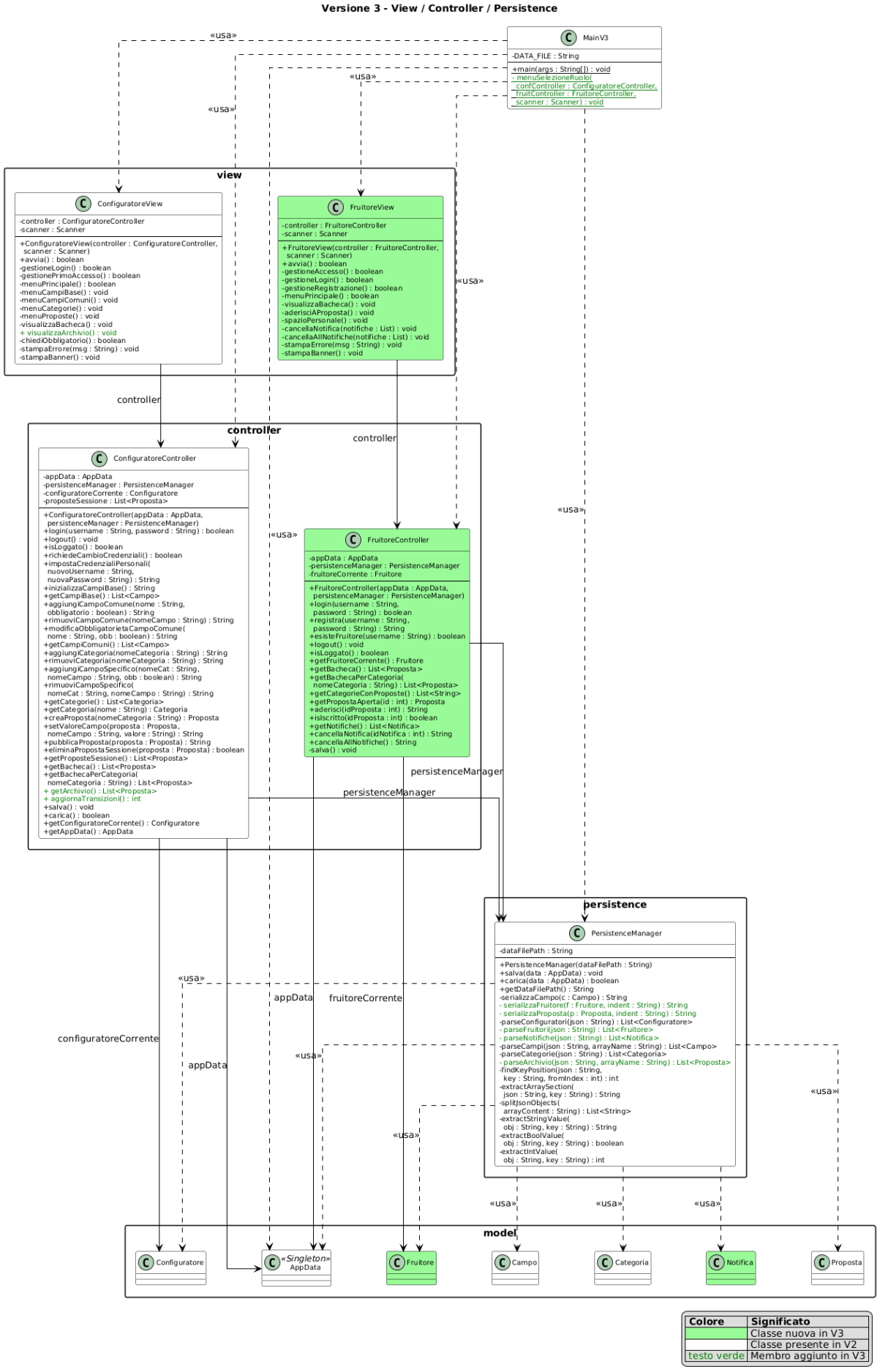
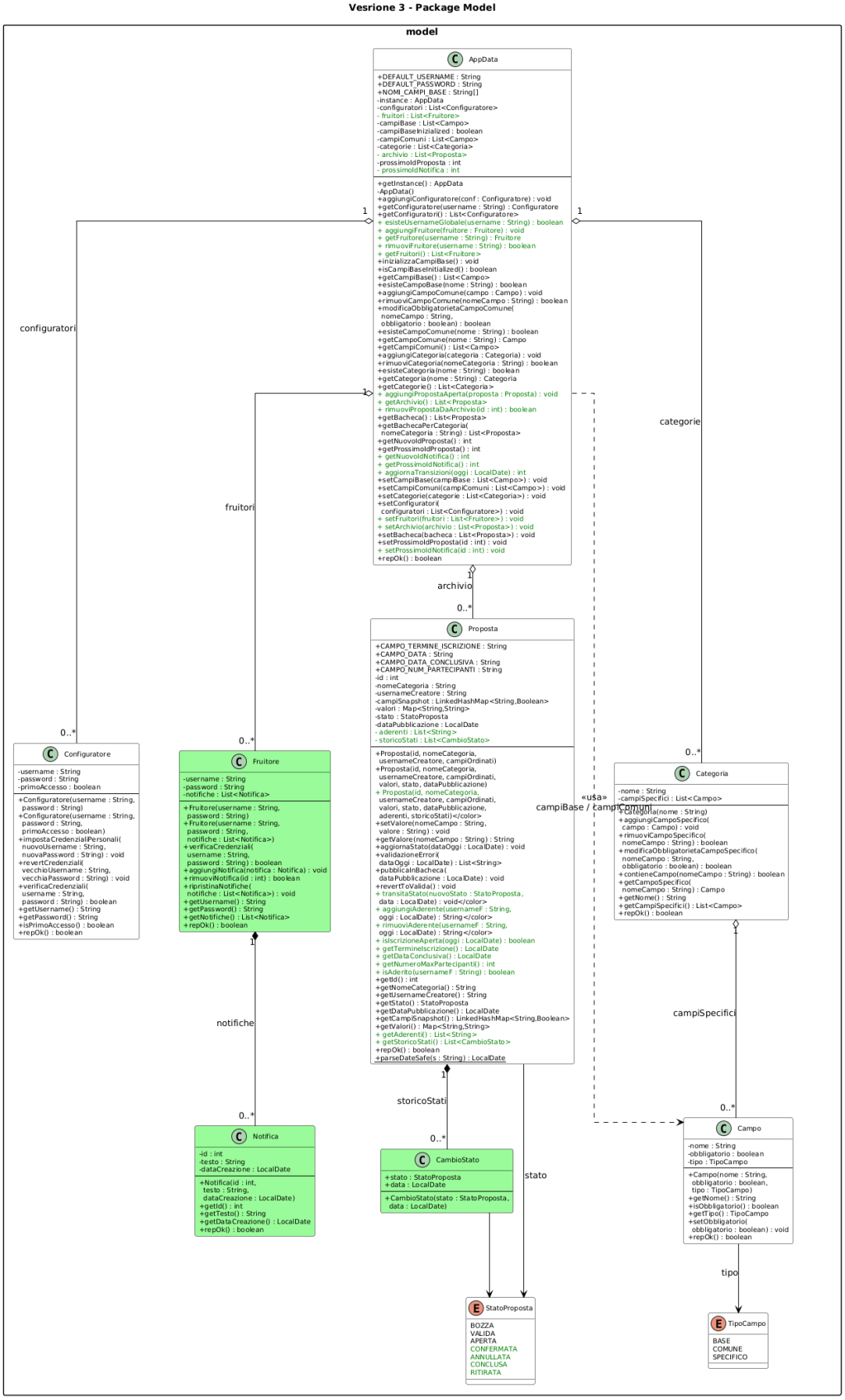


Diagramma delle classi


Versione 3 - View / Controller / Persistence
MainV3
-DATA_FILE : String
+main(args : String[]) : void
- menuSelezioneRuolo(
confController : ConfiguratoreController,
fruitController : FruitoreController,
scanner : Scanner) : void
«usa»
view
ConfiguratoreView
-controller : ConfiguratoreController
-scanner : Scanner
+ConfiguratoreView(controller : ConfiguratoreController, scanner : Scanner)
+avvia() : boolean
-gestioneLogin() : boolean
-gestionePrimoAccesso() : boolean
-menuPrincipale() : boolean
-menuCampiBase() : void
-menuCampiComuni() : void
-menuCategorie() : void
-menuProposte() : void
-visualizzaBacheca() : void
+ visualizzaArchivio() : void
-chiediObbligatorio() : boolean
-stampaErrore(msg : String) : void
-stampaBanner() : void
FruitoreView
-controller : FruitoreController
-scanner : Scanner
+FruitoreView(controller : FruitoreController, scanner : Scanner)
+avvia() : boolean
-gestioneAccesso() : boolean
-gestioneLogin() : boolean
-gestioneRegistrazione() : boolean
-menuPrincipale() : boolean
-visualizzaBacheca() : void
-aderisciAProposta() : void
-spazioPersonale() : void
-cancellaNotifica(notifiche : List) : void
-cancellaAllNotifiche(notifiche : List) : void
-stampaErrore(msg : String) : void
-stampaBanner() : void
controller
controller
ConfiguratoreController
-appData : AppData
-persistenceManager : PersistenceManager
-configuratoreCorrente : Configuratore
-proposteSessione : List<Proposta>
+ConfiguratoreController(appData : AppData, persistenceManager : PersistenceManager)
+login(username : String, password : String) : boolean
+logout() : void
+isLoggato() : boolean
+richiedeCambioCredenziali() : boolean
+impostaCredenzialiPersonali(nuovoUsername : String, nuovaPassword : String) : String
+inizializzaCampiBase() : String
+getCampiBase() : List<Campo>
+aggiungiCampoComune(nome : String, obbligatorio : boolean) : String
+rimuoviCampoComune(nomeCampo : String) : String
+modificaObbligatorietaCampoComune(nome : String, obb : boolean) : String
+getCampiComuni() : List<Campo>
+aggiungiCategoria(nomeCategoria : String) : String
+rimuoviCategoria(nomeCategoria : String) : String
+aggiungiCampoSpecifico(nomeCat : String, nomeCampo : String, obb : boolean) : String
+rimuoviCampoSpecifico(nomeCat : String, nomeCampo : String) : String
+getCategorie() : List<Categoria>
+getCategoria(nome : String) : Categoria
+creaProposta(nomeCategoria : String) : Proposta
+setValoreCampo(proposta : Proposta, nomeCampo : String, valore : String) : String
+pubblicaProposta(proposta : Proposta) : String
+eliminaPropostaSessione(proposta : Proposta) : boolean
+getProposteSessione() : List<Proposta>
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(nomeCategoria : String) : List<Proposta>
+ getArchivio() : List<Proposta>
+ aggiornaTransizioni() : int
+salva() : void
+carica() : boolean
+getConfiguratoreCorrente() : Configuratore
+getAppData() : AppData
FruitoreController
-appData : AppData
-persistenceManager : PersistenceManager
-fruitoreCorrente : Fruitore
+FruitoreController(appData : AppData, persistenceManager : PersistenceManager)
+login(username : String, password : String) : boolean
+registra(username : String, password : String) : String
+esisteFruitore(username : String) : boolean
+logout() : void
+isLoggato() : boolean
+getFruitoreCorrente() : Fruitore
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(nomeCategoria : String) : List<Proposta>
+getCategorieConProposte() : List<String>
+getPropostaAperta(id : int) : Proposta
+aderisci(idProposta : int) : String
+islscritto(idProposta : int) : boolean
+getNotifiche() : List<Notifica>
+cancellaNotifica(idNotifica : int) : String
+cancellaAllNotifiche() : String
-salva() : void
persistenceManager
persistenceManager
persistence
PersistenceManager
-dataFilePath : String
+PersistenceManager(dataFilePath : String)
+salva(data : AppData) : void
+carica(data : AppData) : boolean
+getDataFilePath() : String
-serializzaCampo(c : Campo) : String
- serializzaFruitore(f : Fruitore, indent : String) : String
- serializzaProposta(p : Proposta, indent : String) : String
-parseConfiguratori(json : String) : List<Configuratore>
- parseFruitori(json : String) : List<Fruitore>
- parseNotifiche(json : String) : List<Notifica>
-parseCampi(json : String, arrayName : String) : List<Campo>
-parseCategorie(json : String) : List<Categoria>
- parseArchivio(json : String, arrayName : String) : List<Proposta>
-findKeyPosition(json : String, key : String, fromIndex : int) : int
-extractArraySection(json : String, key : String) : String
-splitJsonObjects(arrayContent : String) : List<String>
-extractStringValue(obj : String, key : String) : String
-extractBoolValue(obj : String, key : String) : boolean
-extractIntValue(obj : String, key : String) : int
appData
fruitoreCorrente
configuratoreCorrente
appData
model
Configuratore
«Singleton» AppData
Fruitore
Campo
Categoria
Notifica
Proposta
Colore
Significato
Classe nuova in V3
Classe presente in V2
testo verde
Membro aggiunto in V3



Colore	Significato
	Classe nuova in V3
	Classe presente in V2
testo verde	Membro aggiunto in V3

CAPITOLO 4

Versione 4

Casi d'uso testuali nuovi

Nome	**UC27 - Ritira proposta in archivio**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. L'archivio contiene almeno una proposta in stato APERTA o CONFERMATA. Trigger: Selezione dell'opzione "Ritira proposta". 1. Il sistema mostra le proposte in stato APERTA o CONFERMATA presenti nell'archivio. 2. Il configuratore inserisce l'ID della proposta da ritirare. 3. Il sistema estrae la proposta e verifica che il suo stato attuale consenta il ritiro. 4. Il sistema verifica che la data odierna sia strettamente precedente alla "Data inizio" dell'evento. 5. Il sistema transita lo stato della proposta in RITIRATA e registra il passaggio nello storico. 6. Il sistema invia automaticamente una notifica di ritiro nello spazio personale di tutti i fruitori iscritti (aderenti). 7. Il sistema salva le modifiche in modo persistente. 8. Il sistema mostra la conferma dell'operazione. Fine
Scenario alternativo	2a. Precondizione: l'ID inserito non corrisponde ad alcuna proposta o la proposta non è nell'archivio. Il sistema mostra un messaggio di errore e interrompe l'operazione. Fine

Continua nella pagina successiva...

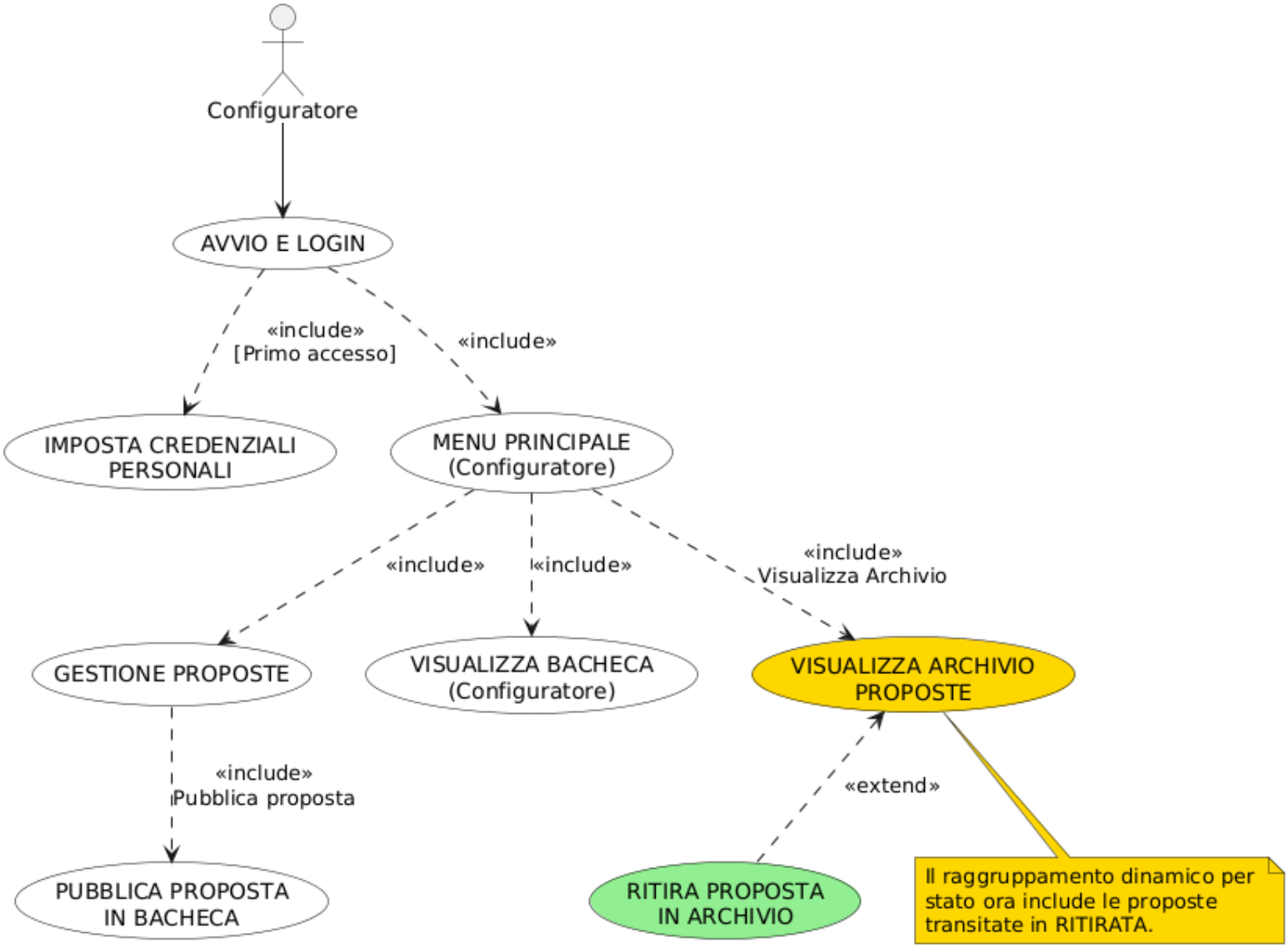
Continua dalla pagina precedente...	
Scenario alternativo	3a. Precondizione: la proposta si trova in uno stato diverso da APERTA o CONFERMATA (es. già CONCLUSA o ANNULLATA). Il sistema mostra l'errore "Il ritiro è consentito solo per proposte APERTE o CONFERMATE" e interrompe l'operazione. Fine
Scenario alternativo	4a. Precondizione: la "Data inizio" dell'evento è già trascorsa o corrisponde alla data odierna (oggi $\geq$ Data inizio). Il sistema blocca il ritiro indicando che l'evento è imminente o passato. Fine
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback ricaricando i dati precedenti dal disco per ripristinare sia lo stato della proposta che le notifiche, poi mostra un errore. Fine

Nome	**UC28 - Disdici iscrizione a proposta**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato e risulta iscritto ad almeno una proposta. Trigger: Selezione dell'opzione per disdire un'iscrizione. 1. Il sistema mostra l'elenco delle proposte APERTE a cui l'utente è iscritto. 2. Il fruitore inserisce l'ID della proposta da cui desidera disiscriversi. 3. Il sistema verifica che la proposta sia in stato APERTA. 4. Il sistema verifica che la data odierna sia minore o uguale al "Termine ultimo di iscrizione". 5. Il sistema rimuove lo username del fruitore dalla lista degli aderenti della proposta. 6. Il sistema salva le modifiche in modo persistente. 7. Il sistema mostra la conferma dell'avvenuta disdetta. Fine
Scenario alternativo	2a. Precondizione: l'ID non è valido oppure il fruitore non risulta iscritto alla proposta indicata. Il sistema mostra un errore. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	3a. Precondizione: la proposta ha cambiato stato (non è più APERTA). Il sistema mostra un errore indicando l'impossibilità di operare. Fine
Scenario alternativo	4a. Precondizione: il "Termine ultimo di iscrizione" è scaduto rispetto alla data odierna. Il sistema blocca la disdetta informando che il termine è oltrepassato. Fine
Scenario alternativo	6a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback reinserendo il fruitore tra gli aderenti in memoria e mostra un messaggio di errore. Fine

Casi d'uso grafici

Colore	Significato
Verde	Novità
Giallo	Modifica
Bianco	Invariato



Colore	Significato
Verde	Novità
Giallo	Modifica
Bianco	Invariato

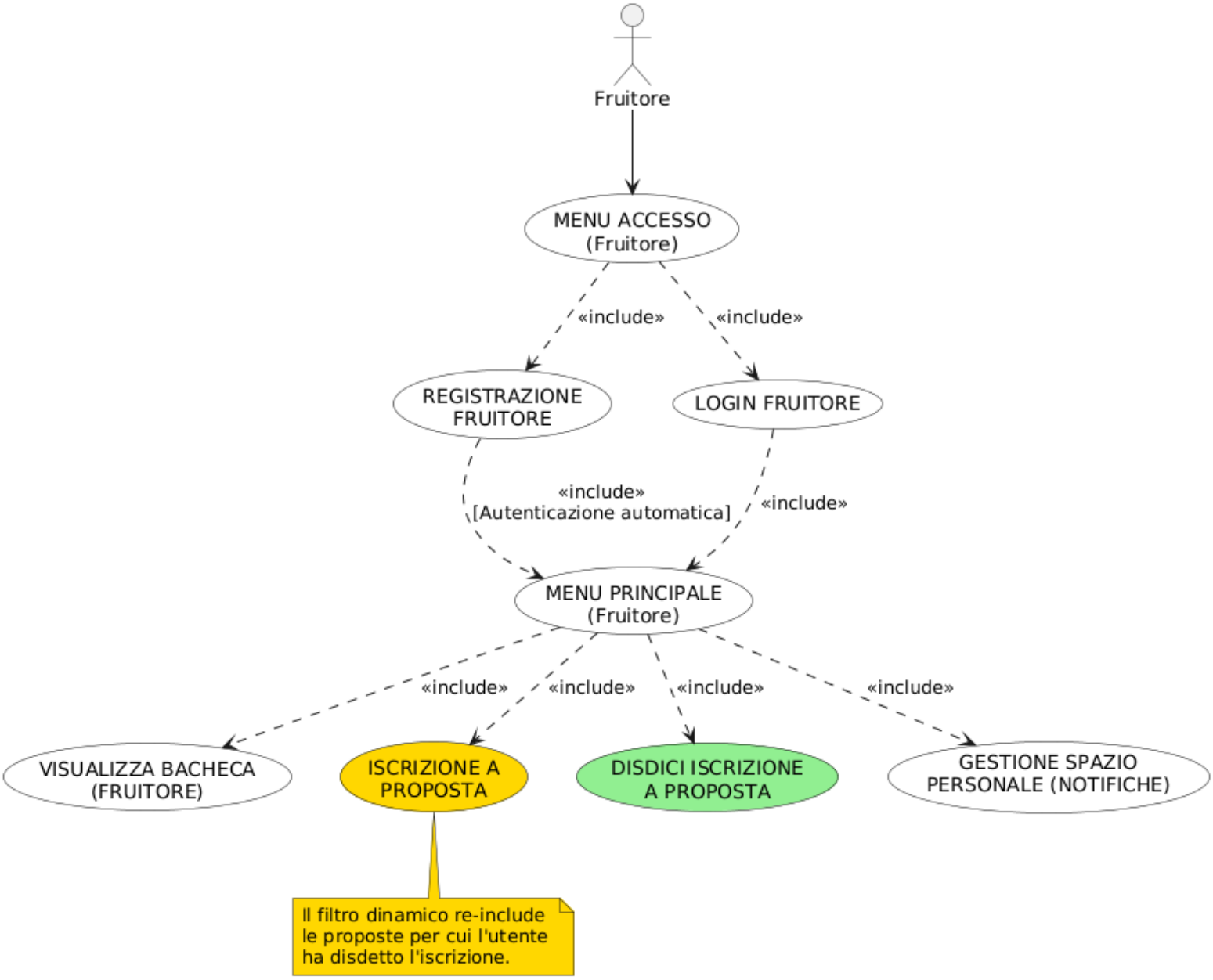
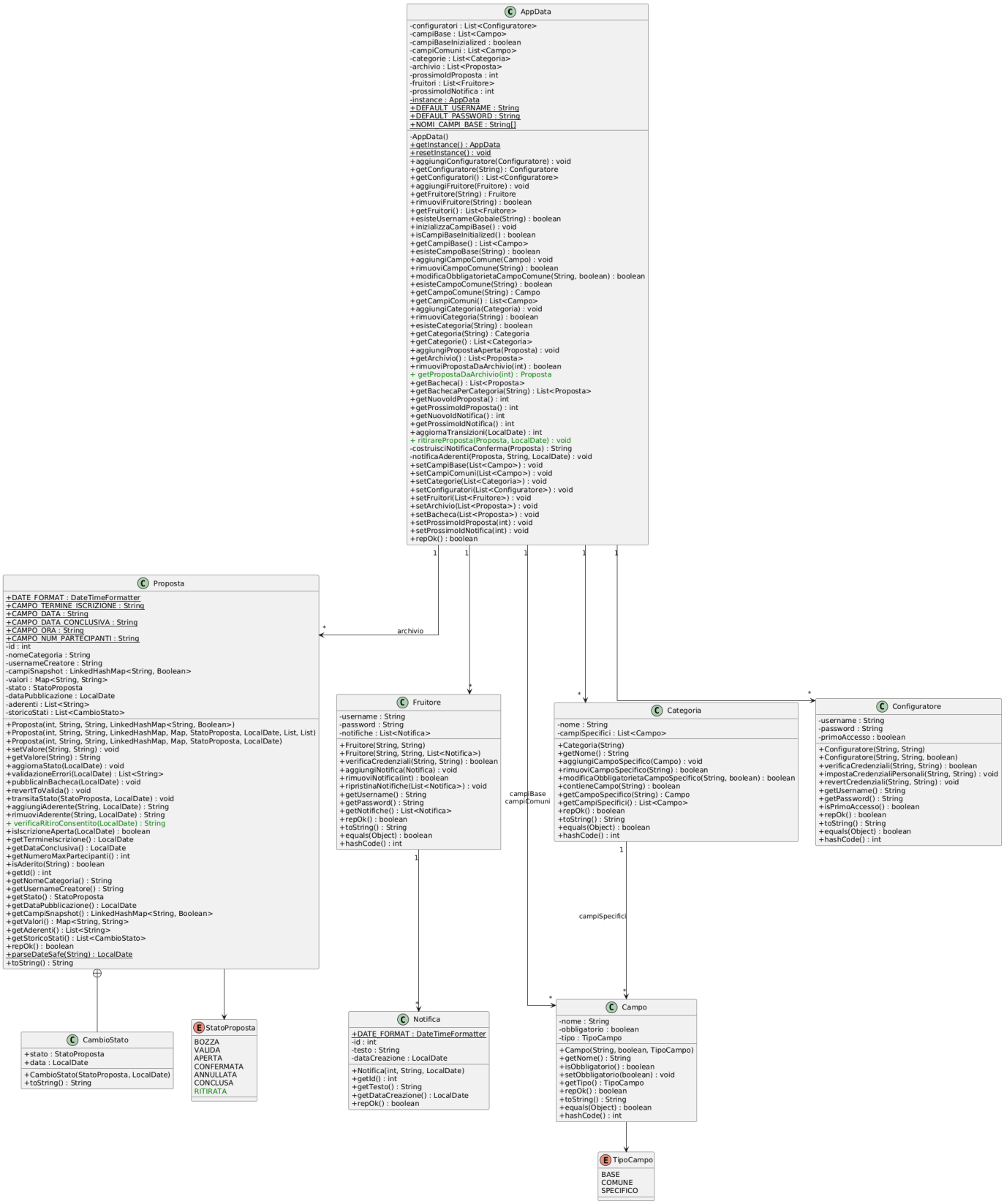


Diagramma delle classi

ConfiguratoreView
-SEPARATORE : String
-controller : ConfiguratoreController
-scanner : Scanner
+ConfiguratoreView(ConfiguratoreController, Scanner)
+avvia() : boolean
-gestioneLogin() : boolean
-gestionePrimoAccesso() : boolean
-menuPrincipale() : boolean
- menuRitiraProposta() : void
-menuCampiBase() : void
-menuCampiComuni() : void
-aggiungiCampoComune() : void
-rimuoviCampoComune() : void
-modificaObbligCampoComune() : void
-menuCategorie() : void
-aggiungiCategoria() : void
-aggiungiCampiSpecificiInterattivo(String) : void
-rimuoviCategoria() : void
-menuCampiSpecifici() : void
-visualizzaDettaglioCategoria() : void
-visualizzaRiepilogo() : void
-menuProposte() : void
-nuovaProposta() : void
-continuaCompilazione() : void
-compilaCampiProposta(Proposta) : void
-mostraRiepilogoProposta(Proposta) : void
-offriPubblicazione(Proposta) : void
-pubblicaPropostaInterattivo() : void
-eliminaPropostaSessione() : void
-selezionaPropostaSessione(String, boolean) : Proposta
-visualizzaBacheca() : void
-stampaBloccoPropostaBacheca(Proposta) : void
-visualizzaArchivio() : void
-chiediObbligatorio() : boolean
-premInvio() : void
-stampaErrore(String) : void
-stampaBanner() : void
FruitoreView
-SEPARATORE : String
-controller : FruitoreController
-scanner : Scanner
+FruitoreView(FruitoreController, Scanner)
+avvia() : boolean
-gestioneAccesso() : boolean
-gestioneLogin() : boolean
-gestioneRegistrazione() : boolean
-menuPrincipale() : boolean
-visualizzaBacheca() : void
-stampaBloccoPropostaBacheca(Proposta) : void
-aderisciAProposta() : void
- disdicilscrizione() : void
-spazioPersonale() : void
-cancellaNotifica(List<Notifica>) : void
-cancellaAllNotifiche(List<Notifica>) : void
-premInvio() : void
-stampaErrore(String) : void
-stampaBanner() : void
ConfiguratoreController
-appData : AppData
-persistenceManager : PersistenceManager
-configuratoreCorrente : Configuratore
-proposteSessione : List<Proposta>
+ConfiguratoreController(AppData, PersistenceManager)
+login(String, String) : boolean
+logout() : void
+isLoggato() : boolean
+richiedeCambioCredenziali() : boolean
+impostaCredenzialiPersonali(String, String) : String
+inizializzaCampiBase() : String
+getCampiBase() : List<Campo>
+aggiungiCampoComune(String, boolean) : String
+rimuoviCampoComune(String) : String
+modificaObbligatorietaCampoComune(String, boolean) : String
+getCampiComuni() : List<Campo>
+aggiungiCategoria(String) : String
+rimuoviCategoria(String) : String
+aggiungiCampoSpecifico(String, String, boolean) : String
+rimuoviCampoSpecifico(String, String) : String
+modificaObbligatorietaCampoSpecifico(String, String, boolean) : String
+getCategorie() : List<Categoria>
+getCategoria(String) : Categoria
+creaProposta(String) : Proposta
+setValoreCampo(Proposta, String, String) : String
+pubblicaProposta(Proposta) : String
+eliminaPropostaSessione(Proposta) : boolean
+getProposteSessione() : List<Proposta>
+ ritirareProposta(int) : String
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getArchivio() : List<Proposta>
+aggiornaTransizioni() : int
+salva() : void
+carica() : boolean
+getConfiguratoreCorrente() : Configuratore
+getAppData() : AppData
FruitoreController
-appData : AppData
-persistenceManager : PersistenceManager
-fruitoreCorrente : Fruitore
+FruitoreController(AppData, PersistenceManager)
+login(String, String) : boolean
+registra(String, String) : String
+esisteFruitore(String) : boolean
+logout() : void
+isLoggato() : boolean
+getFruitoreCorrente() : Fruitore
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getCategorieConProposte() : List<String>
+getPropostaAperta(int) : Proposta
+aderisci(int) : String
+ disdicilscrizione(int) : String
+isIscritto(int) : boolean
+getNotifiche() : List<Notifica>
+cancellaNotifica(int) : String
+cancellaAllNotifiche() : String
-salva() : void
PersistenceManager
-dataFilePath : String
+PersistenceManager(String)
+salva(AppData) : void
+carica(AppData) : boolean
-serializzaCampo(Campo) : String
-serializzaFruitore(Fruitore, String) : String
-serializzaProposta(Proposta, String) : String
-jsonStr(String) : String
-parseConfiguratori(String) : List<Configuratore>
-parseFruitori(String) : List<Fruitore>
-parseNotifiche(String) : List<Notifica>
-parseCampi(String, String) : List<Campo>
-parseCategorie(String) : List<Categoria>
-parseArchivio(String, String) : List<Proposta>
-findKeyPosition(String, String, int) : int
-extractArraySection(String, String) : String
-splitJsonObjects(String) : List<String>
-extractStringValue(String, String) : String
-extractBoolValue(String, String) : boolean
-extractIntValue(String, String) : int
+getDataFilePath() : String
AppData


AppData
-configuratori : List<Configuratore>
-campiBase : List<Campo>
-campiBaseInizialized : boolean
-campiComuni : List<Campo>
-categorie : List<Categoria>
-archivio : List<Proposta>
-prossimoIdProposta : int
-fruitori : List<Fruitore>
-prossimoIdNotifica : int
-instance : AppData
+DEFAULT_USERNAME : String
+DEFAULT_PASSWORD : String
+NOMI_CAMPI_BASE : String[]
-AppData()
+getInstance() : AppData
+resetInstance() : void
+aggiungiConfiguratore(Configuratore) : void
+getConfiguratore(String) : Configuratore
+getConfiguratori() : List<Configuratore>
+aggiungiFruitore(Fruitore) : void
+getFruitore(String) : Fruitore
+rimuoviFruitore(String) : boolean
+getFruitori() : List<Fruitore>
+esisteUsernameGlobale(String) : boolean
+inizializzaCampiBase() : void
+isCampiBaseInitialized() : boolean
+getCampiBase() : List<Campo>
+esisteCampoBase(String) : boolean
+aggiungiCampoComune(Campo) : void
+rimuoviCampoComune(String) : boolean
+modificaObbligatorietaCampoComune(String, boolean) : boolean
+esisteCampoComune(String) : boolean
+getCampoComune(String) : Campo
+getCampiComuni() : List<Campo>
+aggiungiCategoria(Categoria) : void
+rimuoviCategoria(String) : boolean
+esisteCategoria(String) : boolean
+getCategoria(String) : Categoria
+getCategorie() : List<Categoria>
+aggiungiPropostaAperta(Proposta) : void
+getArchivio() : List<Proposta>
+rimuoviPropostaDaArchivio(int) : boolean
+ getPropostaDaArchivio(int) : Proposta
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getNuovoIdProposta() : int
+getProssimoIdProposta() : int
+getNuovoIdNotifica() : int
+getProssimoIdNotifica() : int
+aggiornaTransizioni(LocalDate) : int
+ ritirareProposta(Proposta, LocalDate) : void
-costruisciNotificaConferma(Proposta) : String
-notificaAderenti(Proposta, String, LocalDate) : void
+setCampiBase(List<Campo>) : void
+setCampiComuni(List<Campo>) : void
+setCategorie(List<Categoria>) : void
+setConfiguratori(List<Configuratore>) : void
+setFruitori(List<Fruitore>) : void
+setArchivio(List<Proposta>) : void
+setBacheca(List<Proposta>) : void
+setProssimoIdProposta(int) : void
+setProssimoIdNotifica(int) : void
+repOk() : boolean
1
1
1
1
1
archivio
*
*
*
*
Proposta
+DATE_FORMAT : DateTimeFormatter
+CAMPO_TERMINE_ISCRIZIONE : String
+CAMPO_DATA : String
+CAMPO_DATA_CONCLUSIVA : String
+CAMPO_ORA : String
+CAMPO_NUM_PARTECIPANTI : String
-id : int
-nomeCategoria : String
-usernameCreatore : String
-campiSnapshot : LinkedHashMap<String, Boolean>
-valori : Map<String, String>
-stato : StatoProposta
-dataPubblicazione : LocalDate
-aderenti : List<String>
-storicoStati : List<CambioStato>
+Proposta(int, String, String, LinkedHashMap<String, Boolean>)
+Proposta(int, String, String, LinkedHashMap, Map, StatoProposta, LocalDate, List, List)
+Proposta(int, String, String, LinkedHashMap, Map, StatoProposta, LocalDate)
+setValore(String, String) : void
+getValore(String) : String
+aggiornaStato(LocalDate) : void
+validazioneErrori(LocalDate) : List<String>
+pubblicaInBacheca(LocalDate) : void
+revertToValida() : void
+transitaStato(StatoProposta, LocalDate) : void
+aggiungiAderente(String, LocalDate) : String
+rimuoviAderente(String, LocalDate) : String
+ verificaRitiroConsentito(LocalDate) : String
+isIscrizioneAperta(LocalDate) : boolean
+getTermineIscrizione() : LocalDate
+getDataConclusiva() : LocalDate
+getNumeroMaxPartecipanti() : int
+isAderito(String) : boolean
+getId() : int
+getNomeCategoria() : String
+getUsernameCreatore() : String
+getStato() : StatoProposta
+getDataPubblicazione() : LocalDate
+getCampiSnapshot() : LinkedHashMap<String, Boolean>
+getValori() : Map<String, String>
+getAderenti() : List<String>
+getStoricoStati() : List<CambioStato>
+repOk() : boolean
+parseDateSafe(String) : LocalDate
+toString() : String
CambioStato
+stato : StatoProposta
+data : LocalDate
+CambioStato(StatoProposta, LocalDate)
+toString() : String
StatoProposta
BOZZA
VALIDA
APERTA
CONFERMATA
ANNULLATA
CONCLUSA
RITIRATA
Fruitore
-username : String
-password : String
-notifiche : List<Notifica>
+Fruitore(String, String)
+Fruitore(String, String, List<Notifica>)
+verificaCredenziali(String, String) : boolean
+aggiungiNotifica(Notifica) : void
+rimuoviNotifica(int) : boolean
+ripristinaNotifiche(List<Notifica>) : void
+getUsername() : String
+getPassword() : String
+getNotifiche() : List<Notifica>
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
campiBase
campiComuni
Categoria
-nome : String
-campiSpecifici : List<Campo>
+Categoria(String)
+getNome() : String
+aggiungiCampoSpecifico(Campo) : void
+rimuoviCampoSpecifico(String) : boolean
+modificaObbligatorietaCampoSpecifico(String, boolean) : boolean
+contieneCampo(String) : boolean
+getCampoSpecifico(String) : Campo
+getCampiSpecifici() : List<Campo>
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
Configuratore
-username : String
-password : String
-primoAccesso : boolean
+Configuratore(String, String)
+Configuratore(String, String, boolean)
+verificaCredenziali(String, String) : boolean
+impostaCredenzialiPersonali(String, String) : void
+revertCredenziali(String, String) : void
+getUsername() : String
+getPassword() : String
+isPrimoAccesso() : boolean
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
1
1
campiSpecifici
Notifica
+DATE_FORMAT : DateTimeFormatter
-id : int
-testo : String
-dataCreazione : LocalDate
+Notifica(int, String, LocalDate)
+getId() : int
+getTesto() : String
+getDataCreazione() : LocalDate
+repOk() : boolean
Campo
-nome : String
-obbligatorio : boolean
-tipo : TipoCampo
+Campo(String, boolean, TipoCampo)
+getNome() : String
+isObbligatorio() : boolean
+setObbligatorio(boolean) : void
+getTipo() : TipoCampo
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
TipoCampo
BASE
COMUNE
SPECIFICO

CAPITOLO 5

Versione 5

Casi d'uso testuali nuovi

Nome	**UC29 – Importa singolo file batch**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Il file batch esiste nel filesystem ed è leggibile. Trigger: Selezione dell'opzione "Importa da file batch" → "Importa singolo file". 1. Il sistema mostra la descrizione del formato batch supportato (CAMPO_COMUNE, CATEGORIA, PROPOSTA). 2. Il configuratore inserisce il percorso del file (assoluto o relativo alla working directory). 3. Il sistema legge tutte le righe del file. 4. Il sistema scorre le righe, ignorando quelle vuote e quelle che iniziano con #. 5. Per ogni riga significativa, il sistema identifica il comando (primo token prima di |). 6. Per ogni riga CAMPO_COMUNE: il sistema crea il campo comune con nome e obbligatorietà indicati e salva su disco. 7. Per ogni riga CATEGORIA: il sistema crea la categoria; se la riga contiene token CAMPO_SPECIFICO, aggiunge i campi specifici inline prima del salvataggio, poi salva su disco. 8. Per ogni riga PROPOSTA: il sistema costruisce lo snapshot dei campi (BASE → COMUNI → SPECIFICI della categoria), assegna i valori indicati con il formato campo = valore, valida la proposta con le stesse regole della modalità interattiva e, se valida, la pubblica direttamente in bacheca salvando su disco. 9. Al termine, il sistema raccoglie il resoconto in un *BatchRisultato* (numero righe elaborate, successi, warning, errori). 10. Il sistema mostra il resoconto completo all'utente. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	2a. Precondizione: il percorso è vuoto o l'utente digita "annulla". Il sistema annulla l'operazione senza modifiche. Fine
Scenario alternativo	3a. Precondizione: il file non esiste o non è accessibile (IOException). Il sistema mostra un messaggio di errore con indicazione del percorso e interrompe l'operazione. Fine
Scenario alternativo	5a. Precondizione: il comando della riga non è riconosciuto. Il sistema registra un warning con numero di riga e prosegue con la riga successiva. Fine (riga)
Scenario alternativo	6a. Precondizione: il campo comune ha nome vuoto, oppure esiste già un campo con lo stesso nome. Il sistema registra un warning e salta la riga. Fine (riga)
Scenario alternativo	6b. Precondizione: errore di salvataggio (I/O) dopo l'aggiunta in memoria. Il sistema esegue il rollback in memoria, registra un errore e prosegue. Fine (riga)
Scenario alternativo	7a. Precondizione: la categoria esiste già. Il sistema registra un warning e salta la riga. Fine (riga)
Scenario alternativo	7b. Precondizione: un campo specifico inline non è valido. Il sistema registra un warning per quel campo specifico e lo salta; la categoria viene creata senza di esso. Fine (riga)
Scenario alternativo	7c. Precondizione: errore di salvataggio (I/O) della categoria. Il sistema esegue il rollback in memoria, registra un errore e prosegue. Fine (riga)
Scenario alternativo	8a. Precondizione: la categoria indicata nella riga PROPOSTA non esiste. Il sistema registra un errore e salta la riga. Fine (riga)
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	8b. Precondizione: token campo = valore non validi o campo non esistente. Il sistema registra un warning per ogni token non riconosciuto e prosegue con i restanti campi. Fine (riga)
Scenario alternativo	8c. Precondizione: la proposta risulta non valida dopo la validazione. Il sistema registra un errore con i problemi rilevati. La proposta non viene pubblicata. Fine (riga)
Scenario alternativo	8d. Precondizione: errore di salvataggio (I/O) della proposta pubblicata. Il sistema esegue il rollback in memoria, registra un errore e prosegue. Fine (riga)

Nome	**UC30 – Importa più file batch in sequenza**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Selezione dell'opzione "Importa da file batch" → "Importa più file in sequenza". 1. Il sistema chiede di inserire i percorsi dei file batch uno alla volta. 2. Il configuratore inserisce i percorsi (uno per invio). 3. Il configuratore preme INVIO con il campo vuoto per confermare la lista. 4. Il sistema mostra il numero di file selezionati e avvia l'elaborazione sequenziale. 5. Per ciascun file della lista, il sistema esegue l'elaborazione descritta in UC29 (passi 3–9). 6. I risultati parziali di ogni file vengono aggregati in un unico *BatchRisultato* complessivo tramite il metodo *aggiungi(parziale)*. 7. Al termine di tutti i file, il sistema mostra il resoconto aggregato con il numero totale di righe elaborate, successi, warning ed errori, e l'elenco dei file processati. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	2a. Precondizione: l'utente digita "annulla" invece di un percorso. Il sistema annulla l'intera operazione senza apportare modifiche. Fine
Scenario alternativo	3a. Precondizione: l'utente preme INVIO senza aver inserito alcun percorso. Il sistema mostra un errore "Nessun file specificato" e interrompe l'operazione. Fine
Scenario alternativo	5a. Precondizione: uno dei file indicati non esiste o non è leggibile (IOException). Il sistema registra un errore nel resoconto aggregato (indicando il file) e continua con il file successivo nella lista. Fine (file)
Scenario alternativo	5b. Precondizione: errori su singole righe di un file (comandi errati, duplicati, validazione fallita). Si applicano le stesse alternative di UC29 (5a–8d) con i relativi rollback per riga. L'elaborazione prosegue per le righe e i file successivi. Fine

Casi d'uso grafici

Colore	Significato
Verde	Novità
Bianco	Invariato

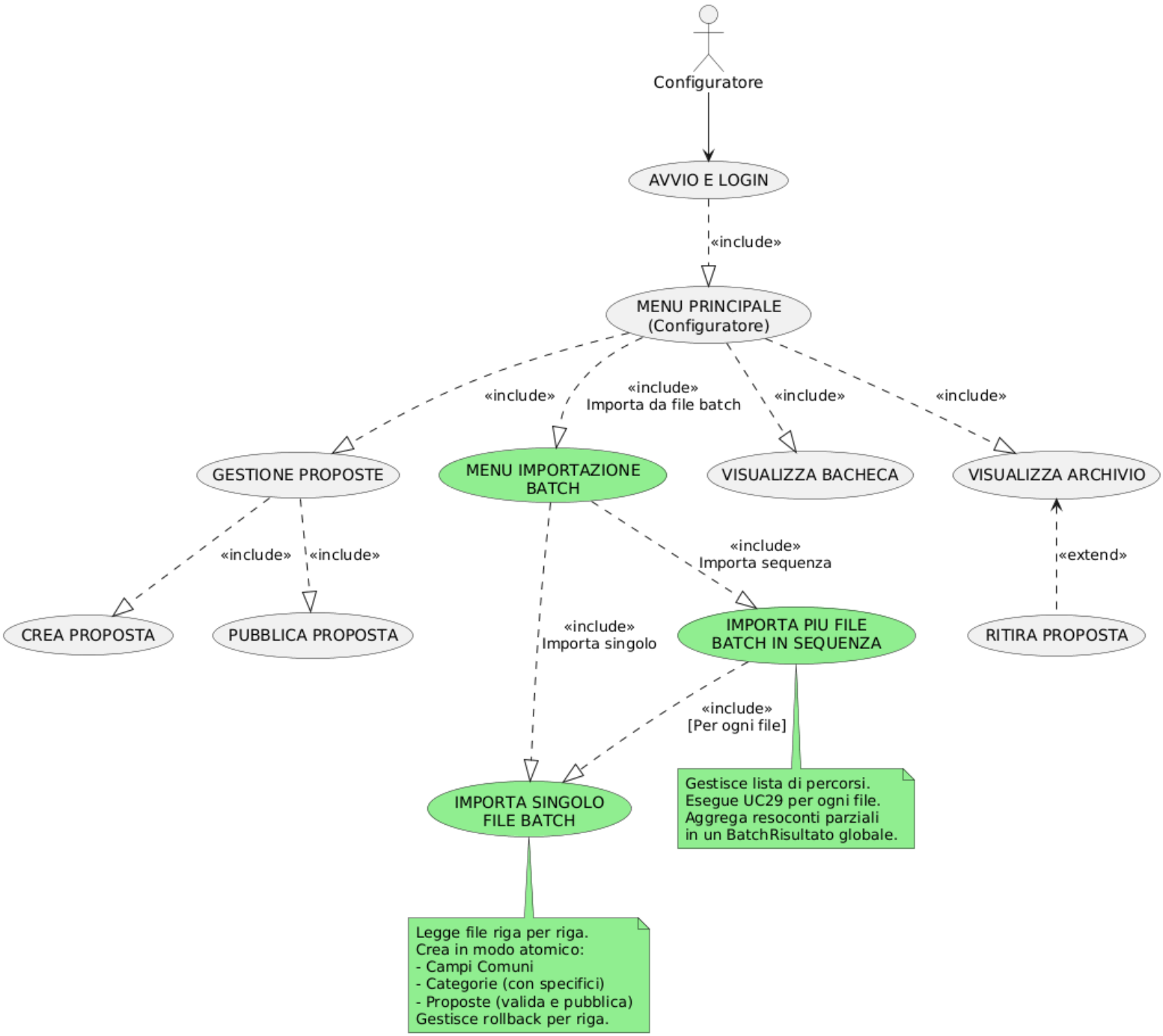
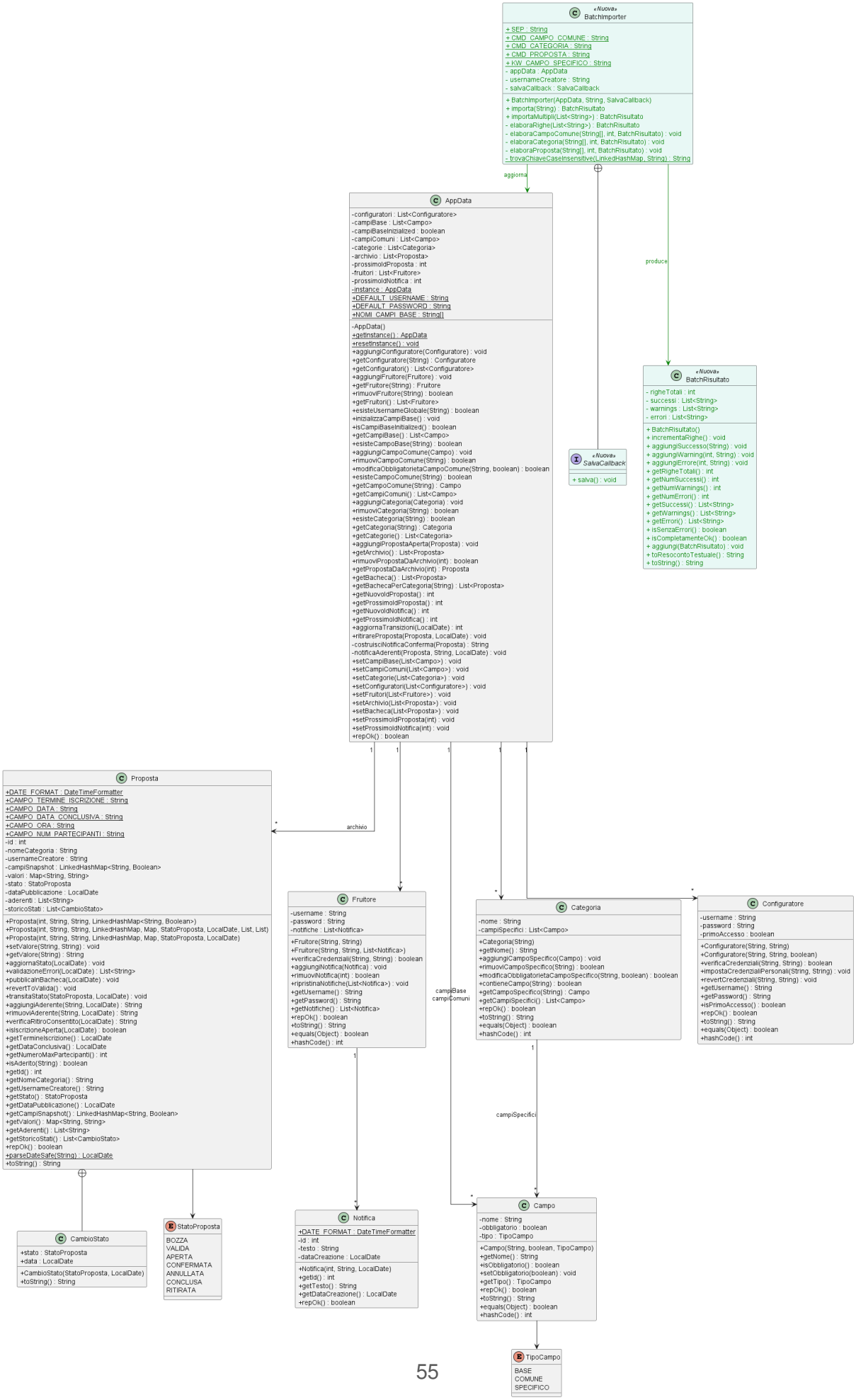
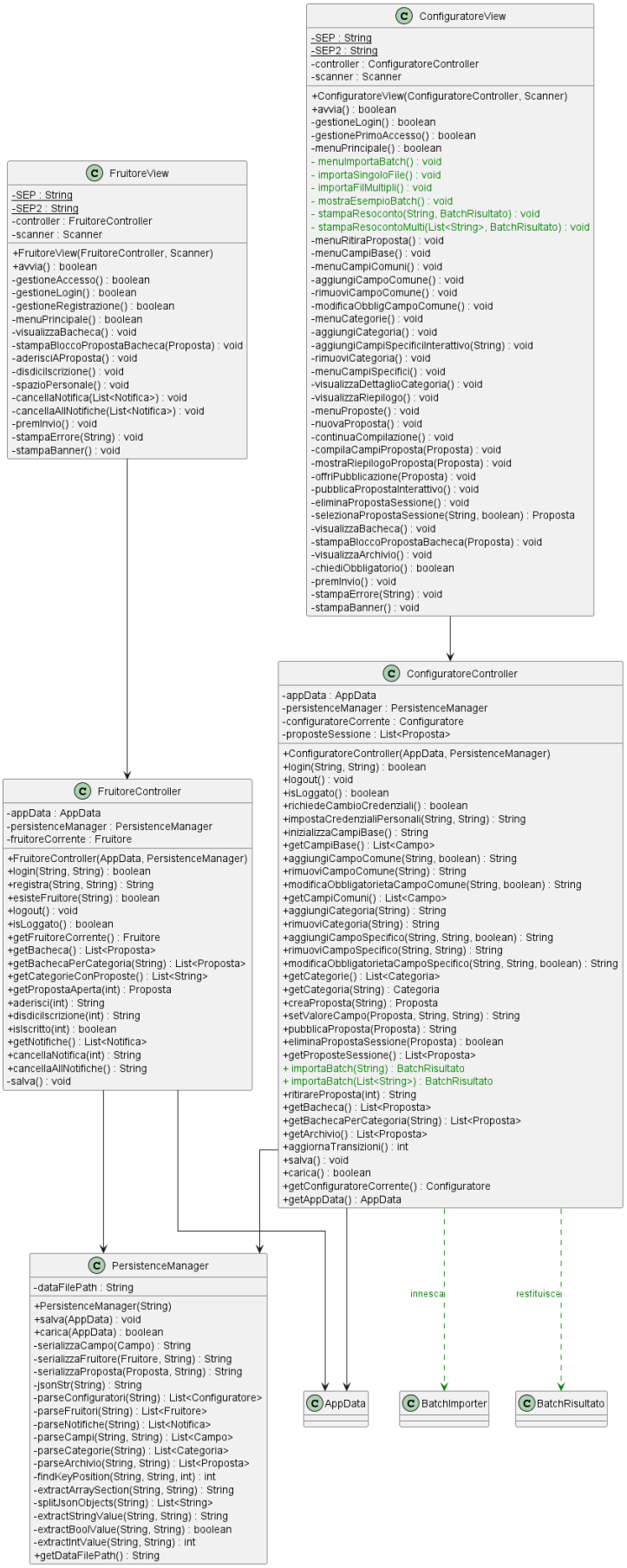


Diagramma delle classi


«Nuova»
BatchImporter
+ SEP : String
+ CMD_CAMPO_COMUNE : String
+ CMD_CATEGORIA : String
+ CMD_PROPOSTA : String
+ KW_CAMPO_SPECIFICO : String
- appData : AppData
- usernameCreatore : String
- salvaCallback : SalvaCallback
+ BatchImporter(AppData, String, SalvaCallback)
+ importa(String) : BatchRisultato
+ importaMultipli(List<String>) : BatchRisultato
- elaboraRighe(List<String>) : BatchRisultato
- elaboraCampoComune(String[], int, BatchRisultato) : void
- elaboraCategoria(String[], int, BatchRisultato) : void
- elaboraProposta(String[], int, BatchRisultato) : void
- trovaChiaveCaseInsensitive(LinkedHashMap, String) : String
aggiorna
produce
AppData
-configuratori : List<Configuratore>
-campiBase : List<Campo>
-campiBaseInizializzati : boolean
-campiComuni : List<Campo>
-categorie : List<Categoria>
-archivio : List<Proposta>
-prossimoIdProposta : int
-fruitori : List<Fruitore>
-prossimoIdNotifica : int
-instance : AppData
+DEFAULT_USERNAME : String
+DEFAULT_PASSWORD : String
+NOMI_CAMPI_BASE : String[]
-AppData()
+getInstance() : AppData
+resetInstance() : void
+aggiungiConfiguratore(Configuratore) : void
+getConfiguratore(String) : Configuratore
+getConfiguratori() : List<Configuratore>
+aggiungiFruitore(Fruitore) : void
+getFruitore(String) : Fruitore
+rimuoviFruitore(String) : boolean
+getFruitori() : List<Fruitore>
+esisteUsernameGlobale(String) : boolean
+inizializzaCampiBase() : void
+isCampiBaseInitialized() : boolean
+getCampiBase() : List<Campo>
+esisteCampoBase(String) : boolean
+aggiungiCampoComune(Campo) : void
+rimuoviCampoComune(String) : boolean
+modificaObbligatorietaCampoComune(String, boolean) : boolean
+esisteCampoComune(String) : boolean
+getCampoComune(String) : Campo
+getCampiComuni() : List<Campo>
+aggiungiCategoria(Categoria) : void
+rimuoviCategoria(String) : boolean
+esisteCategoria(String) : boolean
+getCategoria(String) : Categoria
+getCategorie() : List<Categoria>
+aggiungiPropostaAperta(Proposta) : void
+getArchivio() : List<Proposta>
+rimuoviPropostaDaArchivio(int) : boolean
+getPropostaDaArchivio(int) : Proposta
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getNuovoIdProposta() : int
+getProssimoIdProposta() : int
+getNuovoIdNotifica() : int
+getProssimoIdNotifica() : int
+aggiornaTransizioni(LocalDate) : int
+ritiraProposta(Proposta, LocalDate) : void
-costruisciNotificaConferma(Proposta) : String
-notificaAderenti(Proposta, String, LocalDate) : void
+setCampiBase(List<Campo>) : void
+setCampiComuni(List<Campo>) : void
+setCategorie(List<Categoria>) : void
+setConfiguratori(List<Configuratore>) : void
+setFruitori(List<Fruitore>) : void
+setArchivio(List<Proposta>) : void
+setBacheca(List<Proposta>) : void
+setProssimoIdProposta(int) : void
+setProssimoIdNotifica(int) : void
+repOk() : boolean
«Nuova»
SalvaCallback
+ salva() : void
«Nuova»
BatchRisultato
- righeTotali : int
- successi : List<String>
- warnings : List<String>
- errori : List<String>
+ BatchRisultato()
+ incrementaRighe() : void
+ aggiungiSuccesso(String) : void
+ aggiungiWarning(int, String) : void
+ aggiungiErrore(int, String) : void
+ getRigheTotali() : int
+ getNumSuccessi() : int
+ getNumWarnings() : int
+ getNumErrori() : int
+ getSuccessi() : List<String>
+ getWarnings() : List<String>
+ getErrori() : List<String>
+ isSenzaErrori() : boolean
+ isCompletamenteOk() : boolean
+ aggiungi(BatchRisultato) : void
+ toResocontoTestuale() : String
+ toString() : String
1
1
1
1
1
archivio
*
campiBase
campiComuni
Proposta
+DATE_FORMAT : DateTimeFormatter
+CAMPO_TERMINE_ISCRIZIONE : String
+CAMPO_DATA : String
+CAMPO_DATA_CONCLUSIVA : String
+CAMPO_ORA : String
+CAMPO_NUM_PARTECIPANTI : String
-id : int
-nomeCategoria : String
-usernameCreatore : String
-campiSnapshot : LinkedHashMap<String, Boolean>
-valori : Map<String, String>
-stato : StatoProposta
-dataPubblicazione : LocalDate
-aderenti : List<String>
-storicoStati : List<CambioStato>
+Proposta(int, String, String, LinkedHashMap<String, Boolean>)
+Proposta(int, String, String, LinkedHashMap, Map, StatoProposta, LocalDate, List, List)
+Proposta(int, String, String, LinkedHashMap, Map, StatoProposta, LocalDate)
+setValore(String, String) : void
+getValore(String) : String
+aggiornaStato(LocalDate) : void
+validazioneErrori(LocalDate) : List<String>
+pubblicaInBacheca(LocalDate) : void
+revertToValida() : void
+transitaStato(StatoProposta, LocalDate) : void
+aggiungiAderente(String, LocalDate) : String
+rimuoviAderente(String, LocalDate) : String
+verificaRitiroConsentito(LocalDate) : String
+isIscrizioneAperta(LocalDate) : boolean
+getTermineIscrizione() : LocalDate
+getDataConclusiva() : LocalDate
+getNumeroMaxPartecipanti() : int
+isAderito(String) : boolean
+getId() : int
+getNomeCategoria() : String
+getUsernameCreatore() : String
+getStato() : StatoProposta
+getDataPubblicazione() : LocalDate
+getCampiSnapshot() : LinkedHashMap<String, Boolean>
+getValori() : Map<String, String>
+getAderenti() : List<String>
+getStoricoStati() : List<CambioStato>
+repOk() : boolean
+parseDateSafe(String) : LocalDate
+toString() : String
Fruitore
-username : String
-password : String
-notifiche : List<Notifica>
+Fruitore(String, String)
+Fruitore(String, String, List<Notifica>)
+verificaCredenziali(String, String) : boolean
+aggiungiNotifica(Notifica) : void
+rimuoviNotifica(int) : boolean
+ripristinaNotifiche(List<Notifica>) : void
+getUsername() : String
+getPassword() : String
+getNotifiche() : List<Notifica>
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
Categoria
-nome : String
-campiSpecifici : List<Campo>
+Categoria(String)
+getNome() : String
+aggiungiCampoSpecifico(Campo) : void
+rimuoviCampoSpecifico(String) : boolean
+modificaObbligatorietaCampoSpecifico(String, boolean) : boolean
+contieneCampo(String) : boolean
+getCampoSpecifico(String) : Campo
+getCampiSpecifici() : List<Campo>
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
Configuratore
-username : String
-password : String
-primoAccesso : boolean
+Configuratore(String, String)
+Configuratore(String, String, boolean)
+verificaCredenziali(String, String) : boolean
+impostaCredenzialiPersonali(String, String) : void
+revertCredenziali(String, String) : void
+getUsername() : String
+getPassword() : String
+isPrimoAccesso() : boolean
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
campiSpecifici
CambioStato
+stato : StatoProposta
+data : LocalDate
+CambioStato(StatoProposta, LocalDate)
+toString() : String
StatoProposta
BOZZA
VALIDA
APERTA
CONFERMATA
ANNULLATA
CONCLUSA
RITIRATA
Notifica
+DATE_FORMAT : DateTimeFormatter
-id : int
-testo : String
-dataCreazione : LocalDate
+Notifica(int, String, LocalDate)
+getId() : int
+getTesto() : String
+getDataCreazione() : LocalDate
+repOk() : boolean
Campo
-nome : String
-obbligatorio : boolean
-tipo : TipoCampo
+Campo(String, boolean, TipoCampo)
+getNome() : String
+isObbligatorio() : boolean
+setObbligatorio(boolean) : void
+getTipo() : TipoCampo
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
TipoCampo
BASE
COMUNE
SPECIFICO


ConfiguratoreView
-SEP : String
-SEP2 : String
-controller : ConfiguratoreController
-scanner : Scanner
+ConfiguratoreView(ConfiguratoreController, Scanner)
+avvia() : boolean
-gestioneLogin() : boolean
-gestionePrimoAccesso() : boolean
-menuPrincipale() : boolean
- menuImportaBatch() : void
- importaSingoloFile() : void
- importaFilMultipli() : void
- mostraEsempioBatch() : void
- stampaResoconto(String, BatchRisultato) : void
- stampaResocontoMulti(List<String>, BatchRisultato) : void
-menuRitiraProposta() : void
-menuCampiBase() : void
-menuCampiComuni() : void
-aggiungiCampoComune() : void
-rimuoviCampoComune() : void
-modificaObbligCampoComune() : void
-menuCategorie() : void
-aggiungiCategoria() : void
-aggiungiCampiSpecificiInterattivo(String) : void
-rimuoviCategoria() : void
-menuCampiSpecifici() : void
-visualizzaDettaglioCategoria() : void
-visualizzaRiepilogo() : void
-menuProposte() : void
-nuovaProposta() : void
-continuaCompilazione() : void
-compilaCampiProposta(Proposta) : void
-mostraRiepilogoProposta(Proposta) : void
-offriPubblicazione(Proposta) : void
-pubblicaPropostaInterattivo() : void
-eliminaPropostaSessione() : void
-selezionaPropostaSessione(String, boolean) : Proposta
-visualizzaBacheca() : void
-stampaBloccoPropostaBacheca(Proposta) : void
-visualizzaArchivio() : void
-chiediObbligatorio() : boolean
-premiInvio() : void
-stampaErrore(String) : void
-stampaBanner() : void
FruitoreView
-SEP : String
-SEP2 : String
-controller : FruitoreController
-scanner : Scanner
+FruitoreView(FruitoreController, Scanner)
+avvia() : boolean
-gestioneAccesso() : boolean
-gestioneLogin() : boolean
-gestioneRegistrazione() : boolean
-menuPrincipale() : boolean
-visualizzaBacheca() : void
-stampaBloccoPropostaBacheca(Proposta) : void
-aderisciAProposta() : void
-disdicilscrizione() : void
-spazioPersonale() : void
-cancellaNotifica(List<Notifica>) : void
-cancellaAllNotifiche(List<Notifica>) : void
-premInvio() : void
-stampaErrore(String) : void
-stampaBanner() : void
ConfiguratoreController
-appData : AppData
-persistenceManager : PersistenceManager
-configuratoreCorrente : Configuratore
-proposteSessione : List<Proposta>
+ConfiguratoreController(AppData, PersistenceManager)
+login(String, String) : boolean
+logout() : void
+isLoggato() : boolean
+richiedeCambioCredenziali() : boolean
+impostaCredenzialiPersonali(String, String) : String
+inizializzaCampiBase() : String
+getCampiBase() : List<Campo>
+aggiungiCampoComune(String, boolean) : String
+rimuoviCampoComune(String) : String
+modificaObbligatorietaCampoComune(String, boolean) : String
+getCampiComuni() : List<Campo>
+aggiungiCategoria(String) : String
+rimuoviCategoria(String) : String
+aggiungiCampoSpecifico(String, String, boolean) : String
+rimuoviCampoSpecifico(String, String) : String
+modificaObbligatorietaCampoSpecifico(String, String, boolean) : String
+getCategorie() : List<Categoria>
+getCategoria(String) : Categoria
+creaProposta(String) : Proposta
+setValoreCampo(Proposta, String, String) : String
+pubblicaProposta(Proposta) : String
+eliminaPropostaSessione(Proposta) : boolean
+getProposteSessione() : List<Proposta>
+ importaBatch(String) : BatchRisultato
+ importaBatch(List<String>) : BatchRisultato
+ritirareProposta(int) : String
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getArchivio() : List<Proposta>
+aggiornaTransizioni() : int
+salva() : void
+carica() : boolean
+getConfiguratoreCorrente() : Configuratore
+getAppData() : AppData
FruitoreController
-appData : AppData
-persistenceManager : PersistenceManager
-fruitoreCorrente : Fruitore
+FruitoreController(AppData, PersistenceManager)
+login(String, String) : boolean
+registra(String, String) : String
+esisteFruitore(String) : boolean
+logout() : void
+isLoggato() : boolean
+getFruitoreCorrente() : Fruitore
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getCategorieConProposte() : List<String>
+getPropostaAperta(int) : Proposta
+aderisci(int) : String
+disdicilscrizione(int) : String
+isIscritto(int) : boolean
+getNotifiche() : List<Notifica>
+cancellaNotifica(int) : String
+cancellaAllNotifiche() : String
-salva() : void
PersistenceManager
-dataFilePath : String
+PersistenceManager(String)
+salva(AppData) : void
+carica(AppData) : boolean
-serializzaCampo(Campo) : String
-serializzaFruitore(Fruitore, String) : String
-serializzaProposta(Proposta, String) : String
-jsonStr(String) : String
-parseConfiguratori(String) : List<Configuratore>
-parseFruitori(String) : List<Fruitore>
-parseNotifiche(String) : List<Notifica>
-parseCampi(String, String) : List<Campo>
-parseCategorie(String) : List<Categoria>
-parseArchivio(String, String) : List<Proposta>
-findKeyPosition(String, String, int) : int
-extractArraySection(String, String) : String
-splitJsonObjects(String) : List<String>
-extractStringValue(String, String) : String
-extractBoolValue(String, String) : boolean
-extractIntValue(String, String) : int
+getDataFilePath() : String
innesca
restituisce
AppData
BatchImporter
BatchRisultato

CAPITOLO 6

Versione Finale

Casi d'uso testuali

Nome	**UC01 - Avvio e Login**
Attore	Configuratore
Scenario principale	Precondizione: L'applicazione è avviata. Transizioni automatiche (UC20) completate dal sistema. Trigger: L'utente seleziona "Configuratore" dal menu iniziale. 1. Il sistema mostra la schermata di login dedicata. 2. Il configuratore inserisce username e password. 3. Il sistema verifica la corrispondenza delle credenziali. 4. Il sistema autentica il configuratore. 5. Il sistema verifica se si tratta del primissimo accesso assoluto (credenziali di default). 6. «include» "UC02 - Imposta credenziali personali" (se primo accesso). 7. Il sistema mostra il menu principale aggiornato coi contatori in tempo reale. Postcondizione: Il configuratore è autenticato nel back-end. Fine
Scenario alternativo	2a. Precondizione: il configuratore digita "esci" come username. Il sistema interrompe l'operazione e torna al menu di selezione ruolo. Fine
Scenario alternativo	3a. Precondizione: le credenziali non sono valide. Il sistema mostra un errore e indica i tentativi rimasti. Torna al punto 2 (se tentativi < 3), altrimenti il sistema blocca l'accesso alla sessione. Fine

Nome	**UC02 - Imposta credenziali personali**
Attore	Configuratore
Scenario principale	Precondizione: Login effettuato con credenziali di default (primo accesso). Trigger: Il sistema rileva la necessità di cambiare le credenziali. 1. Il sistema richiede di scegliere le credenziali personali. 2. Il configuratore inserisce il nuovo username. 3. Il sistema verifica che lo username non coincida con la parola riservata "esci". 4. Il configuratore inserisce la nuova password. 5. Il configuratore inserisce la password per conferma. 6. Il sistema verifica che le due password coincidano. 7. Il sistema verifica che username e password non siano stringhe vuote/blank. 8. Il sistema verifica l'unicità globale dello username (tra Configuratori e Fruitori). 9. Il sistema aggiorna le credenziali in memoria. 10. Il sistema salva i dati in modo persistente. 11. Il sistema mostra la conferma e accede al menu. Postcondizione: Le credenziali di default sono sostituite e salvate in modo permanente. Fine
Scenario alternativo	2a. Precondizione: il configuratore digita "annulla" nel campo username. Il sistema annulla l'operazione ed esegue il logout automatico. Fine
Scenario alternativo	3a. Precondizione: lo username inserito è "esci". Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	4a. Precondizione: il configuratore digita "annulla" nel campo password. Il sistema annulla l'operazione ed esegue il logout automatico. Fine
Scenario alternativo	6a. Precondizione: le password non coincidono. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	7a. Precondizione: campi lasciati vuoti. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	8a. Precondizione: username già in uso nel sistema. Il sistema mostra un errore, annulla l'operazione ed esegue il logout. Fine
Scenario alternativo	10a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback in memoria, mostra un errore ed esegue il logout. Fine

Nome	**UC03 - Logout**
Attore	Configuratore
Scenario principale	Precondizione: Configuratore autenticato. Trigger: Selezione "Logout". 1. Il configuratore seleziona Logout. 2. Il sistema scarta le proposte non pubblicate della sessione corrente (stato BOZZA o VALIDA). 3. Se erano presenti proposte non pubblicate, il sistema avvisa il configuratore. 4. Il sistema termina la sessione corrente. 5. Il sistema chiede se accedere con un altro account. 6. Sì: ritorna a UC01. 7. No: termina l'applicazione. Fine
Scenario alternativo	5a. Precondizione: Il configuratore inserisce una risposta non valida (diversa da "s" o "n"). Il sistema mostra un messaggio di errore testuale e ripropone la domanda.

Nome	**UC04 - Visualizza campi base**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. I campi base sono stati inizializzati. Trigger: Il configuratore seleziona "Gestione campi base" dal menu principale. 1. Il configuratore seleziona la voce "Gestione campi base". 2. Il sistema mostra l'elenco dei campi base con nome e tipo, evidenziando che sono tutti obbligatori (Titolo, Numero partecipanti, ecc.). 3. Il sistema indica che i campi base sono immutabili. 4. Il configuratore preme INVIO per tornare al menu principale. Fine
Scenario alternativo	2a. Precondizione: Campi base non inizializzati. Il sistema li inizializza automaticamente al primo avvio e poi mostra l'elenco. Fine

Nome	**UC05 - Aggiungi campo comune**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Il configuratore seleziona "Aggiungi campo comune" nel menu "Gestione campi comuni". 1. Il configuratore seleziona "Aggiungi campo comune". 2. Il sistema chiede il nome del nuovo campo. 3. Il configuratore inserisce il nome. 4. Il sistema chiede se il campo è obbligatorio o facoltativo. 5. Il configuratore risponde (s/n). 6. Il sistema verifica che il nome non coincida con un campo base o comune esistente. 7. Il sistema aggiunge il campo comune e salva in modo persistente. 8. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Nome vuoto. Il sistema mostra errore e richiede nuovamente il nome.
Scenario alternativo	6a. Precondizione: Conflitto con campo base. Il sistema mostra errore e non aggiunge il campo. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	6b. Precondizione: Conflitto con campo comune esistente. Il sistema mostra errore e non aggiunge il campo. Fine
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiunge il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC06 - Rimuovi campo comune**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno un campo comune. Trigger: Il configuratore seleziona "Rimuovi campo comune". 1. Il configuratore seleziona "Rimuovi campo comune". 2. Il sistema mostra l'elenco dei campi comuni esistenti. 3. Il configuratore inserisce il nome del campo da rimuovere. 4. Il sistema rimuove il campo comune e salva in modo persistente. 5. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra errore e non esegue la rimozione. Fine
Scenario alternativo	4a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC07 - Modifica obbligatorietà campo comune**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno un campo comune. Trigger: Il configuratore seleziona "Modifica obbligatorietà campo comune". 1. Il configuratore seleziona "Modifica obbligatorietà campo comune". 2. Il sistema mostra l'elenco dei campi comuni con l'obbligatorietà attuale. 3. Il configuratore inserisce il nome del campo da modificare. 4. Il configuratore indica il nuovo valore di obbligatorietà (s/n). 5. Il sistema aggiorna il campo e salva in modo persistente. 6. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra errore e non applica modifiche. Fine
Scenario alternativo	5a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiorna il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC08 - Aggiungi categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. I campi base sono inizializzati. Trigger: Il configuratore seleziona "Aggiungi categoria". 1. Il configuratore seleziona "Aggiungi categoria". 2. Il sistema chiede il nome della nuova categoria. 3. Il configuratore inserisce il nome. 4. Il sistema verifica che il nome non sia già in uso. 5. Il sistema crea la categoria e salva in modo persistente. 6. Il sistema chiede se aggiungere subito campi specifici. 7. Sì: il caso d'uso estende UC10 (aggiunta campi specifici). 8. No: il sistema torna al menu categorie. 9. Il sistema mostra conferma dell'operazione. Fine
Scenario alternativo	3a. Precondizione: Nome vuoto. Errore e richiesta nuovo inserimento.
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	4a. Precondizione: Categoria duplicata. Errore e nessuna creazione. Fine
Scenario alternativo	5a. Precondizione: Errore di salvataggio (I/O). Il sistema crea la categoria in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC09 - Rimuovi categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una categoria. Trigger: Il configuratore seleziona "Rimuovi categoria". 1. Il configuratore seleziona "Rimuovi categoria". 2. Il sistema mostra l'elenco categorie. 3. Il configuratore inserisce il nome della categoria da rimuovere. 4. Il sistema chiede conferma esplicita prima della rimozione. 5. Il configuratore conferma. 6. Il sistema rimuove la categoria e tutti i suoi campi specifici. 7. Il sistema salva i dati in modo persistente. 8. Il sistema mostra conferma. Fine
Scenario alternativo	3a. Precondizione: Categoria non trovata. Errore, nessuna rimozione. Fine
Scenario alternativo	5a. Precondizione: Annulla. Il configuratore non conferma; operazione interrotta. Fine
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove la categoria in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC10 - Aggiungi campo specifico a categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una categoria. Trigger: Il configuratore sceglie di aggiungere un campo specifico a una categoria. 1. Il configuratore seleziona "Gestisci campi specifici" e indica la categoria. 2. Il sistema verifica che la categoria esista. 3. Il configuratore seleziona "Aggiungi campo specifico". 4. Il sistema chiede il nome del nuovo campo. 5. Il configuratore inserisce il nome. 6. Il sistema chiede se il campo è obbligatorio o facoltativo. 7. Il configuratore risponde (s/n). 8. Il sistema verifica che il nome non coincida con un campo base, comune o specifico già presente. 9. Il sistema aggiunge il campo specifico alla categoria e salva. 10. Il sistema mostra conferma e permette inserimenti successivi (INVIO per terminare). Fine
Scenario alternativo	2a. Precondizione: Categoria non esiste. Il sistema mostra un messaggio di errore. Fine
Scenario alternativo	5a. Precondizione: Nome vuoto. Il sistema mostra un errore e richiede un nuovo inserimento.
Scenario alternativo	8a. Precondizione: Nome duplicato. Il sistema mostra un errore e non procede con l'aggiunta. Fine
Scenario alternativo	9a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiunge il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC11 - Rimuovi campo specifico da categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste una categoria con almeno un campo specifico. Trigger: Il configuratore seleziona "Rimuovi campo specifico". 1. Il configuratore seleziona la categoria. 2. Il sistema mostra i campi specifici della categoria. 3. Il configuratore inserisce il nome del campo da rimuovere. 4. Il sistema rimuove il campo specifico e salva in modo persistente. 5. Il sistema mostra conferma. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra un messaggio di errore. Fine
Scenario alternativo	4a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC12 - Modifica obbligatorietà campo specifico**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste una categoria con almeno un campo specifico. Trigger: Il configuratore seleziona "Modifica obbligatorietà campo specifico". 1. Il configuratore seleziona la categoria. 2. Il sistema mostra i campi specifici con obbligatorietà attuale. 3. Il configuratore inserisce il nome del campo da modificare. 4. Il configuratore indica il nuovo valore (s/n). 5. Il sistema aggiorna il campo specifico e salva in modo persistente. 6. Il sistema mostra conferma. Fine
Scenario alternativo	3a. Precondizione: Campo non trovato. Il sistema mostra un messaggio di errore. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	5a. Precondizione: Errore di salvataggio (I/O). Il sistema aggiorna il campo in memoria ma avvisa dell'errore di salvataggio fisico. Fine

Nome	**UC13 - Visualizza categorie e campi**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Il configuratore seleziona "Visualizza riepilogo categorie e campi". 1. Il configuratore seleziona "Visualizza riepilogo". 2. Il sistema mostra i campi BASE. 3. Il sistema mostra i campi COMUNI (con indicazione di obbligatorio/facoltativo). 4. Per ogni categoria, il sistema mostra: nome categoria e campi specifici (con indicazione di obbligatorio/facoltativo). 5. Il configuratore preme INVIO per tornare al menu. Fine
Scenario alternativo	3a. Precondizione: Nessun campo comune definito. Il sistema mostra la dicitura "(nessuno definito)".
Scenario alternativo	4a. Precondizione: Nessuna categoria definita nel sistema. Il sistema mostra la dicitura "(nessuna categoria definita)".
Scenario alternativo	4b. Precondizione: Una categoria non possiede campi specifici. Il sistema mostra la dicitura "(nessun campo specifico)" sotto il nome della categoria interessata.

Nome	**UC14 - Visualizza dettaglio categoria**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una categoria. Trigger: Il configuratore seleziona "Visualizza dettaglio categoria". 1. Il configuratore seleziona "Visualizza dettaglio categoria". 2. Il sistema richiede l'inserimento del nome della categoria. 3. Il configuratore inserisce il nome richiesto. 4. Il sistema mostra, per la categoria selezionata, il set completo dei dati: tutti i campi BASE, i campi COMUNI e i campi SPECIFICI della categoria stessa. 5. Il configuratore preme INVIO per terminare la visualizzazione e tornare al menu. Fine
Scenario alternativo	3a. Precondizione: La categoria inserita non è presente nell'archivio. Il sistema mostra un messaggio di errore e l'operazione si interrompe. Fine

Nome	**UC15 - Crea proposta**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato ed esiste almeno una categoria. Trigger: Selezione "Nuova proposta" dal menu gestionale. 1. Il sistema mostra l'elenco delle categorie disponibili. 2. Il configuratore inserisce il nome della categoria desiderata. 3. Il sistema verifica l'esistenza della categoria. 4. Il sistema cattura lo snapshot dei campi (base, comuni e specifici) rendendoli indipendenti da future modifiche alla configurazione. 5. Il sistema assegna un ID univoco progressivo estratto dal contatore globale. 6. Il sistema aggiunge la proposta alla sessione corrente in stato BOZZA. 7. «include» "UC16 - Compila campi proposta". 8. Il sistema mostra il riepilogo e, se in stato VALIDA, offre la pubblicazione immediata (estende UC17). Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	1a. Precondizione: l'archivio categorie è vuoto. Il sistema mostra l'errore "Nessuna categoria disponibile" e impedisce la creazione. Fine
Scenario alternativo	3a. Precondizione: categoria non trovata in base all'input dell'utente. Il sistema mostra un errore e torna al menu proposte. Fine

Nome	**UC16 - Compila / Modifica campi proposta**
Attore	Configuratore
Scenario principale	Precondizione: Esiste almeno una proposta in sessione. Trigger: Estensione da UC15 o selezione "Continua compilazione". 1. Il sistema presenta i campi dello snapshot uno alla volta. 2. Per ogni campo, il sistema mostra l'etichetta [OBB/FAC], il formato richiesto e il valore attuale. 3. Il configuratore inserisce un valore o preme INVIO per mantenere il valore attuale. 4. Il sistema valida l'input e ricalcola automaticamente lo stato (BOZZA/VALIDA). 5. Il sistema verifica i vincoli temporali: Termine iscrizione $>$ oggi; Data inizio $\geq$ Termine + 2gg; Data fine $\geq$ Data inizio; Formato Ora (HH:MM). 6. Il sistema mostra il riepilogo finale e l'elenco degli errori se lo stato è BOZZA. Fine
Scenario alternativo	4a. Precondizione: formato data (gg/mm/aaaa) o ora (HH:MM) errato. Il sistema mostra un errore specifico e non aggiorna il valore del campo. Prosegue con il campo successivo.

Nome	**UC17 - Pubblica proposta in bacheca**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato e c'è almeno una proposta VALIDA in sessione. Trigger: Selezione "Pubblica proposta in bacheca". 1. Il sistema mostra l'elenco delle proposte in sessione attualmente VALIDE. 2. Il configuratore inserisce l'ID della proposta da pubblicare. 3. Il sistema cattura la data odierna. 4. Il sistema ricalcola la validità della proposta in base alla data odierna. 5. Il sistema imposta lo stato su APERTA e registra la data nello storico. 6. Il sistema sposta la proposta dalla sessione all'Archivio globale. 7. Il sistema salva le modifiche in modo persistente. 8. Il sistema mostra conferma dell'operazione. Postcondizione: La proposta ha stato APERTA, è salvata nell'archivio ed è visibile ai fruitori. Fine
Scenario alternativo	1a. Precondizione: nessuna proposta in sessione è in stato VALIDA. Il sistema mostra un errore. Fine
Scenario alternativo	2a. Precondizione: l'input non è numerico o non corrisponde a una proposta in elenco. Il sistema mostra un errore. Fine
Scenario alternativo	4a. Precondizione: la validità è scaduta a causa del cambio di data (es. superata mezzanotte). Il sistema nega la pubblicazione e mostra l'elenco degli errori. Fine
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback completo riportando la proposta allo stato VALIDA in sessione. Fine

Nome	**UC18 - Elimina proposta dalla sessione**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Esiste almeno una proposta in sessione (stato BOZZA o VALIDA). Trigger: Il configuratore seleziona "Elimina proposta" nel menu proposte. 1. Il sistema mostra l'elenco delle proposte in sessione. 2. Il configuratore inserisce l'ID della proposta da eliminare. 3. Il sistema chiede conferma esplicita prima dell'eliminazione. 4. Il configuratore conferma. 5. Il sistema rimuove definitivamente la proposta dalla sessione. 6. Il sistema mostra conferma dell'avvenuta operazione. Fine
Scenario alternativo	2a. Precondizione: l'ID inserito non è valido o non corrisponde a una proposta presente in sessione. Il sistema mostra un messaggio di errore e interrompe l'operazione. Fine
Scenario alternativo	4a. Precondizione: il configuratore annulla l'operazione (nega la conferma). Il sistema interrompe l'operazione senza apportare modifiche e torna al menu. Fine

Nome	**UC19 - Visualizza bacheca Configuratore**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Selezione "Visualizza bacheca". 1. Il configuratore richiede la visualizzazione della bacheca. 2. Il sistema recupera tutte le proposte con stato APERTA. 3. Il sistema raggruppa dinamicamente le proposte per categoria (inclusa la voce "(categoria rimossa)" per le eventuali proposte orfane). 4. Il sistema mostra per ogni proposta: ID, dettagli, valori dei campi e il numero di iscritti attuali rispetto al massimo consentito. 5. Il configuratore preme INVIO per terminare. Postcondizione: Nessuna modifica allo stato del sistema. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	2a. Precondizione: non ci sono proposte APERTE in archivio. Il sistema mostra il messaggio "(Nessuna proposta aperta in bacheca)". Fine
Scenario alternativo	3a. Precondizione: categoria di una proposta rimossa dopo la pubblicazione. Il sistema visualizza la proposta sotto l'etichetta "(categoria rimossa)". Fine

Nome	**UC20 - Esecuzione transizioni automatiche di stato**
Attore	Sistema
Scenario principale	Precondizione: L'applicazione viene avviata. L'archivio contiene proposte. Trigger: Inizializzazione del sistema. 1. Il sistema analizza l'archivio delle proposte. 2. Per ogni proposta APERTA con "Termine ultimo" superato, il sistema verifica se iscritti $\geq$ minimo richiesto. 3. Il sistema transita la proposta in stato CONFERMATA. 4. Il sistema genera una notifica di conferma per ogni aderente iscritto. 5. Per ogni proposta CONFERMATA con "Data conclusiva" superata, il sistema la transita in stato CONCLUSA. 6. Il sistema salva i nuovi stati e le notifiche in modo persistente. 7. Il sistema mostra a schermo il numero totale di transizioni effettuate. Postcondizione: Stati aggiornati alla data odierna e notifiche distribuite. Fine
Scenario alternativo	2a. Precondizione: la proposta APERTA è scaduta ma gli iscritti sono insufficienti (o il campo "Numero massimo" è corrotto/negativo). Il sistema transita la proposta in ANNULLATA e genera una notifica di annullamento per ogni aderente. Torna al punto 5
Scenario alternativo	6a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback ricaricando i dati precedenti dal disco per evitare la generazione di notifiche duplicate ai riavvii successivi. Fine

Nome	**UC21 - Registrazione Fruitore**
Attore	Fruitore
Scenario principale	Precondizione: L'utente non è autenticato. Trigger: Selezione "Registrazione (nuovo account)" dal menu di accesso. 1. Il sistema mostra la schermata di registrazione. 2. Il fruitore inserisce lo username desiderato. 3. Il fruitore inserisce la password. 4. Il fruitore reinserisce la password per conferma. 5. Il sistema verifica che le due password coincidano. 6. Il sistema verifica che username e password non siano vuoti. 7. Il sistema verifica l'unicità globale dello username. 8. Il sistema crea il profilo Fruitore in memoria. 9. Il sistema salva i dati in modo persistente. 10. Il sistema autentica automaticamente il fruitore e mostra il benvenuto. Postcondizione: Il fruitore è registrato, autenticato e con uno spazio notifiche vuoto. Fine
Scenario alternativo	2a. Precondizione: il fruitore digita "annulla" come username. Il sistema interrompe l'operazione e torna al menu. Fine
Scenario alternativo	3a. Precondizione: il fruitore digita "annulla" come password. Il sistema interrompe l'operazione e torna al menu. Fine
Scenario alternativo	5a. Precondizione: le password non coincidono. Il sistema mostra l'errore e interrompe l'operazione. Fine
Scenario alternativo	6a. Precondizione: campi vuoti o composti da soli spazi. Il sistema mostra l'errore e interrompe l'operazione. Fine
Scenario alternativo	7a. Precondizione: username già in uso (globale). Il sistema mostra l'errore e interrompe l'operazione. Fine
Scenario alternativo	9a. Precondizione: Errore di salvataggio (I/O). Il sistema rimuove il profilo dalla memoria (rollback), mostra l'errore e interrompe l'operazione. Fine

Nome	**UC22 - Login Fruitore**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore possiede un account e non è autenticato. Trigger: Selezione "Login (account esistente)". 1. Il sistema mostra la schermata di login. 2. Il fruitore inserisce username e password. 3. Il sistema verifica la corrispondenza delle credenziali. 4. Il sistema autentica il fruitore. 5. Il sistema conteggia le notifiche presenti nello spazio personale e mostra il benvenuto con il relativo avviso. Postcondizione: Il fruitore è autenticato nel sistema. Fine
Scenario alternativo	2a. Precondizione: il fruitore digita "annulla" nel campo username. Il sistema annulla l'operazione e torna al menu. Fine
Scenario alternativo	3a. Precondizione: le credenziali sono errate (incluso inserire "annulla" come password). Il sistema mostra l'errore e i tentativi rimasti. Torna al punto 2 (se tentativi < 3), altrimenti il sistema blocca l'accesso. Fine

Nome	**UC23 - Visualizza Bacheca (Fruitore)**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato. Trigger: Selezione "Visualizza bacheca". 1. Il fruitore richiede la visualizzazione. 2. Il sistema recupera tutte le proposte in stato APERTA. 3. Il sistema raggruppa le proposte per categoria. 4. Il sistema mostra dettagli, posti occupati, scadenze e tutti i campi informativi per ogni proposta. 5. Il sistema evidenzia con l'etichetta [ISCRITTO] le iniziative a cui l'utente loggato ha già aderito. 6. Il fruitore preme INVIO per terminare la consultazione. Postcondizione: Nessuna modifica allo stato del sistema. Fine
Scenario alternativo	2a. Precondizione: nessuna proposta APERTA trovata nell'archivio. Il sistema comunica "(Nessuna proposta aperta in bacheca)". Fine

Nome	**UC24 - Iscrizione a proposta**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato. Trigger: Selezione "Aderisci a una proposta". 1. Il fruitore avvia la procedura di iscrizione. 2. Il sistema estrae le proposte APERTE dall'archivio. 3. Il sistema verifica che ci sia almeno una proposta APERTA nella bacheca globale. 4. Il sistema filtra le proposte escludendo quelle a cui l'utente è già iscritto. 5. Il sistema verifica che la lista filtrata non sia vuota. 6. Il fruitore inserisce l'ID della proposta scelta. 7. Il sistema verifica che l'input sia numerico e diverso da "0". 8. Il sistema individua la proposta e verifica che sia in stato APERTA. 9. Il sistema verifica che la data odierna sia $\leq$ al "Termine ultimo di iscrizione". 10. Il sistema verifica che ci siano posti disponibili. 11. Il sistema aggiunge lo username del fruitore alla lista degli aderenti. 12. Il sistema salva i dati in modo persistente. 13. Il sistema conferma l'avvenuta iscrizione all'utente. Postcondizione: L'utente è iscritto e i posti disponibili scalano. Fine
Scenario alternativo	3a. Precondizione: l'archivio globale non contiene alcuna proposta in stato APERTA. Il sistema lo comunica all'utente e termina l'operazione. Fine
Scenario alternativo	5a. Precondizione: il fruitore è già iscritto a tutte le proposte APERTE (lista filtrata vuota). Il sistema lo comunica all'utente e termina l'operazione. Fine
Scenario alternativo	7a. Precondizione: il fruitore digita "0" per annullare l'inserimento. Il sistema annulla l'operazione. Fine
Scenario alternativo	7b. Precondizione: l'input inserito non è un numero intero. Il sistema mostra il messaggio "ID non valido". Fine
Scenario alternativo	8a. Precondizione: l'ID non corrisponde a nessuna proposta o la proposta non è più APERTA. Il sistema mostra un messaggio di errore. Fine

Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	9a. Precondizione: il termine di iscrizione è scaduto rispetto alla data odierna. Il sistema blocca l'iscrizione e mostra un messaggio di errore. Fine
Scenario alternativo	10a. Precondizione: i posti sono esauriti (o il campo numero partecipanti è corrotto). Il sistema blocca l'iscrizione e mostra un messaggio di errore. Fine
Scenario alternativo	12a. Precondizione: Errore di salvataggio (I/O). Il sistema annulla l'iscrizione in memoria (rollback) e mostra un messaggio di errore. Fine

Nome	**UC25 - Gestione Spazio Personale (Notifiche)**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato. Trigger: Selezione "Spazio personale". 1. Il fruitore accede allo spazio personale. 2. Il sistema mostra l'elenco delle notifiche e le opzioni del menu ("c", "t", "0"). 3. Il fruitore inserisce l'opzione "c" (Cancella singola notifica). 4. Il sistema verifica che la lista delle notifiche non sia vuota. 5. Il sistema richiede l'inserimento dell'ID della notifica. 6. Il fruitore inserisce l'ID. 7. Il sistema individua e rimuove la notifica dalla memoria. 8. Il sistema salva i dati in modo persistente. 9. Il sistema conferma la cancellazione all'utente. Torna al punto 2
Scenario alternativo	2a. Precondizione: la lista delle notifiche dell'utente è vuota. Il sistema stampa "(Nessuna notifica)" ma mostra comunque il menu operativo. Prosegue al punto 3
Scenario alternativo	3a. Precondizione: il fruitore seleziona "t" (Cancellazione totale). Il sistema verifica che la lista non sia vuota, chiede conferma (s/n). L'utente risponde "s". Il sistema svuota la lista in memoria. Salta al punto 8
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	3b. Precondizione: il fruitore seleziona "t", ma annulla la conferma (risponde "n"). Il sistema annulla l'operazione. Torna al punto 2
Scenario alternativo	3c. Precondizione: il fruitore seleziona "0. Torna al menu principale". Il sistema chiude lo spazio personale e torna al menu precedente. Fine
Scenario alternativo	3d. Precondizione: il fruitore inserisce un'opzione non valida (es. lettera errata). Il sistema mostra l'errore "Scelta non valida". Torna al punto 2
Scenario alternativo	4a. Precondizione: il fruitore seleziona "c" o "t", ma la lista delle notifiche è vuota. Il sistema mostra l'errore "Nessuna notifica da cancellare". Torna al punto 2
Scenario alternativo	7a. Precondizione: l'ID inserito non è numerico o non corrisponde ad alcuna notifica in memoria. Il sistema mostra un errore. Torna al punto 2
Scenario alternativo	8a. Precondizione: Errore di salvataggio (I/O). Il sistema ripristina in memoria la lista originale intatta (rollback) e mostra un errore. Torna al punto 2

Nome	**UC26 - Visualizza Archivio Proposte**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Selezione "Visualizza archivio proposte". 1. Il configuratore avvia la richiesta di visualizzazione. 2. Il sistema recupera l'intero archivio (escludendo gli stati interni BOZZA e VALIDA). 3. Il sistema raggruppa dinamicamente le proposte in base allo stato (APERTA, CONFERMATA, ANNULLATA, CONCLUSA, RITIRATA). 4. Il sistema mostra i dettagli anagrafici, i posti occupati, la lista degli aderenti e lo storico completo dei passaggi di stato per ogni proposta. 5. Il configuratore preme INVIO per terminare. Postcondizione: Nessuna modifica allo stato del sistema. Fine
Scenario alternativo	2a. Precondizione: l'archivio è vuoto. Il sistema segnala "(Nessuna proposta nell'archivio)" e termina l'operazione. Fine

Nome	**UC27 - Ritira proposta in archivio**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. L'archivio contiene almeno una proposta in stato APERTA o CONFERMATA. Trigger: Selezione dell'opzione "Ritira proposta". 1. Il sistema mostra le proposte in stato APERTA o CONFERMATA presenti nell'archivio. 2. Il configuratore inserisce l'ID della proposta da ritirare. 3. Il sistema estrae la proposta e verifica che il suo stato attuale consenta il ritiro. 4. Il sistema verifica che la data odierna sia strettamente precedente alla "Data inizio" dell'evento. 5. Il sistema transita lo stato della proposta in RITIRATA e registra il passaggio nello storico. 6. Il sistema invia automaticamente una notifica di ritiro nello spazio personale di tutti i fruitori iscritti (aderenti). 7. Il sistema salva le modifiche in modo persistente. 8. Il sistema mostra la conferma dell'operazione. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	2a. Precondizione: l'ID inserito non corrisponde ad alcuna proposta o la proposta non è nell'archivio. Il sistema mostra un messaggio di errore e interrompe l'operazione. Fine
Scenario alternativo	3a. Precondizione: la proposta si trova in uno stato diverso da APERTA o CONFERMATA (es. già CONCLUSA o ANNULLATA). Il sistema mostra l'errore "Il ritiro è consentito solo per proposte APERTE o CONFERMATE" e interrompe l'operazione. Fine
Scenario alternativo	4a. Precondizione: la "Data inizio" dell'evento è già trascorsa o corrisponde alla data odierna (oggi $\geq$ Data inizio). Il sistema blocca il ritiro indicando che l'evento è imminente o passato. Fine
Scenario alternativo	7a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback ricaricando i dati precedenti dal disco per ripristinare sia lo stato della proposta che le notifiche, poi mostra un errore. Fine

Nome	**UC28 - Disdici iscrizione a proposta**
Attore	Fruitore
Scenario principale	Precondizione: Il fruitore è autenticato e risulta iscritto ad almeno una proposta. Trigger: Selezione dell'opzione per disdire un'iscrizione. 1. Il sistema mostra l'elenco delle proposte APERTE a cui l'utente è iscritto. 2. Il fruitore inserisce l'ID della proposta da cui desidera disiscriversi. 3. Il sistema verifica che la proposta sia in stato APERTA. 4. Il sistema verifica che la data odierna sia minore o uguale al "Termine ultimo di iscrizione". 5. Il sistema rimuove lo username del fruitore dalla lista degli aderenti della proposta. 6. Il sistema salva le modifiche in modo persistente. 7. Il sistema mostra la conferma dell'avvenuta disdetta. Fine
	Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	2a. Precondizione: l'ID non è valido oppure il fruitore non risulta iscritto alla proposta indicata. Il sistema mostra un errore. Fine
Scenario alternativo	3a. Precondizione: la proposta ha cambiato stato (non è più APERTA). Il sistema mostra un errore indicando l'impossibilità di operare. Fine
Scenario alternativo	4a. Precondizione: il "Termine ultimo di iscrizione" è scaduto rispetto alla data odierna. Il sistema blocca la disdetta informando che il termine è oltrepassato. Fine
Scenario alternativo	6a. Precondizione: Errore di salvataggio (I/O). Il sistema esegue un rollback reinserendo il fruitore tra gli aderenti in memoria e mostra un messaggio di errore. Fine

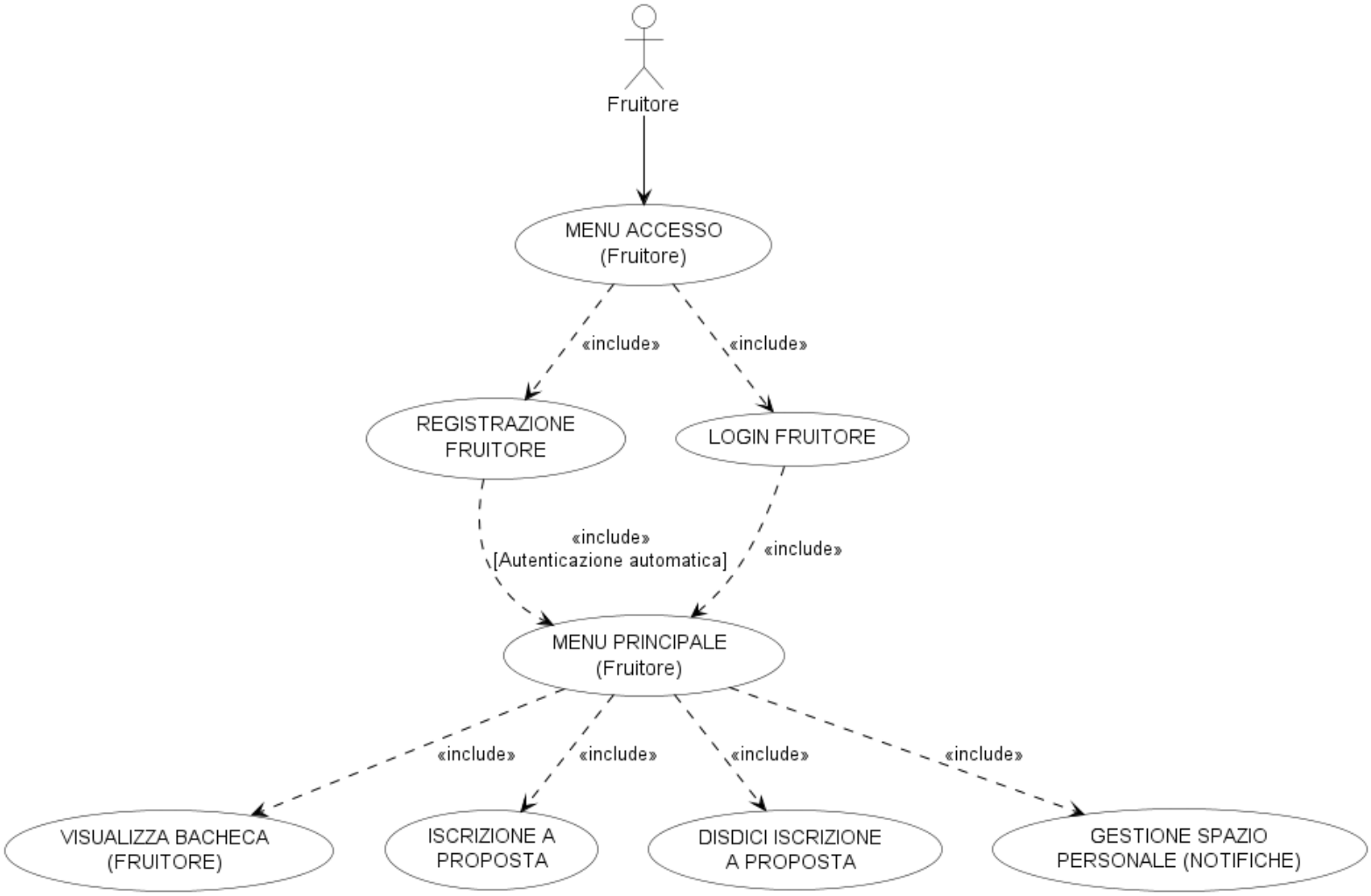
Nome	**UC29 – Importa singolo file batch**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Il file batch esiste nel filesystem ed è leggibile. Trigger: Selezione dell'opzione "Importa da file batch" → "Importa singolo file". 1. Il sistema mostra la descrizione del formato batch supportato (CAMPO_COMUNE, CATEGORIA, PROPOSTA). 2. Il configuratore inserisce il percorso del file (assoluto o relativo alla working directory). 3. Il sistema legge tutte le righe del file. 4. Il sistema scorre le righe, ignorando quelle vuote e quelle che iniziano con #. 5. Per ogni riga significativa, il sistema identifica il comando (primo token prima di |). 6. Per ogni riga CAMPO_COMUNE: il sistema crea il campo comune con nome e obbligatorietà indicati e salva su disco. 7. Per ogni riga CATEGORIA: il sistema crea la categoria; se la riga contiene token CAMPO_SPECIFICO, aggiunge i campi specifici inline prima del salvataggio, poi salva su disco. 8. Per ogni riga PROPOSTA: il sistema costruisce lo snapshot dei campi (BASE → COMUNI → SPECIFICI della categoria), assegna i valori indicati con il formato campo = valore, valida la proposta con le stesse regole della modalità interattiva e, se valida, la pubblica direttamente in bacheca salvando su disco. 9. Al termine, il sistema raccoglie il resoconto in un *BatchRisultato* (numero righe elaborate, successi, warning, errori). 10. Il sistema mostra il resoconto completo all'utente. Fine
Scenario alternativo	2a. Precondizione: il percorso è vuoto o l'utente digita "annulla". Il sistema annulla l'operazione senza modifiche. Fine
Scenario alternativo	3a. Precondizione: il file non esiste o non è accessibile (IOException). Il sistema mostra un messaggio di errore con indicazione del percorso e interrompe l'operazione. Fine
Scenario alternativo	5a. Precondizione: il comando della riga non è riconosciuto. Il sistema registra un warning con numero di riga e prosegue con la riga successiva. Fine (riga)

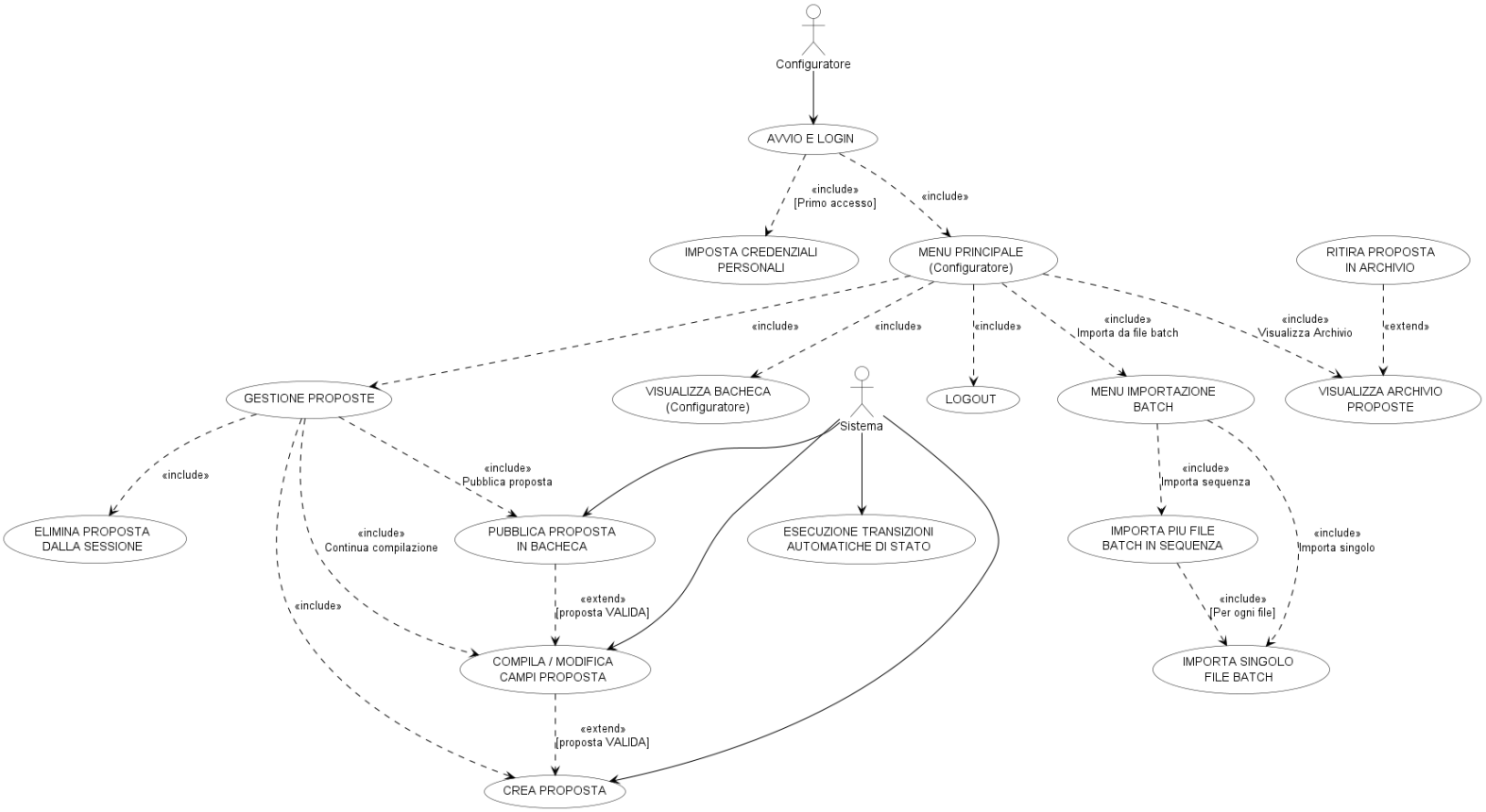
Continua nella pagina successiva...

Continua dalla pagina precedente...	
Scenario alternativo	6a. Precondizione: il campo comune ha nome vuoto, oppure esiste già un campo con lo stesso nome. Il sistema registra un warning e salta la riga. Fine (riga)
Scenario alternativo	6b. Precondizione: errore di salvataggio (I/O) dopo l'aggiunta in memoria. Il sistema esegue il rollback in memoria, registra un errore e prosegue. Fine (riga)
Scenario alternativo	7a. Precondizione: la categoria esiste già. Il sistema registra un warning e salta la riga. Fine (riga)
Scenario alternativo	7b. Precondizione: un campo specifico inline non è valido. Il sistema registra un warning per quel campo specifico e lo salta; la categoria viene creata senza di esso. Fine (riga)
Scenario alternativo	7c. Precondizione: errore di salvataggio (I/O) della categoria. Il sistema esegue il rollback in memoria, registra un errore e prosegue. Fine (riga)
Scenario alternativo	8a. Precondizione: la categoria indicata nella riga PROPOSTA non esiste. Il sistema registra un errore e salta la riga. Fine (riga)
Scenario alternativo	8b. Precondizione: token campo = valore non validi o campo non esistente. Il sistema registra un warning per ogni token non riconosciuto e prosegue con i restanti campi. Fine (riga)
Scenario alternativo	8c. Precondizione: la proposta risulta non valida dopo la validazione. Il sistema registra un errore con i problemi rilevati. La proposta non viene pubblicata. Fine (riga)
Scenario alternativo	8d. Precondizione: errore di salvataggio (I/O) della proposta pubblicata. Il sistema esegue il rollback in memoria, registra un errore e prosegue. Fine (riga)

Nome	**UC30 – Importa più file batch in sequenza**
Attore	Configuratore
Scenario principale	Precondizione: Il configuratore è autenticato. Trigger: Selezione dell'opzione "Importa da file batch" → "Importa più file in sequenza". 1. Il sistema chiede di inserire i percorsi dei file batch uno alla volta. 2. Il configuratore inserisce i percorsi (uno per invio). 3. Il configuratore preme INVIO con il campo vuoto per confermare la lista. 4. Il sistema mostra il numero di file selezionati e avvia l'elaborazione sequenziale. 5. Per ciascun file della lista, il sistema esegue l'elaborazione descritta in UC29 (passi 3–9). 6. I risultati parziali di ogni file vengono aggregati in un unico *BatchRisultato* complessivo tramite il metodo *aggiungi(parziale)*. 7. Al termine di tutti i file, il sistema mostra il resoconto aggregato con il numero totale di righe elaborate, successi, warning ed errori, e l'elenco dei file processati. Fine
Scenario alternativo	2a. Precondizione: l'utente digita "annulla" invece di un percorso. Il sistema annulla l'intera operazione senza apportare modifiche. Fine
Scenario alternativo	3a. Precondizione: l'utente preme INVIO senza aver inserito alcun percorso. Il sistema mostra un errore "Nessun file specificato" e interrompe l'operazione. Fine
Scenario alternativo	5a. Precondizione: uno dei file indicati non esiste o non è leggibile (IOException). Il sistema registra un errore nel resoconto aggregato (indicando il file) e continua con il file successivo nella lista. Fine (file)
Scenario alternativo	5b. Precondizione: errori su singole righe di un file (comandi errati, duplicati, validazione fallita). Si applicano le stesse alternative di UC29 (5a–8d) con i relativi rollback per riga. L'elaborazione prosegue per le righe e i file successivi. Fine

Casi d'uso grafici




Configuratore
AVVIO E LOGIN
«include»
[Primo accesso]
«include»
IMPOSTA CREDENZIALI PERSONALI
MENU PRINCIPALE (Configuratore)
RITIRA PROPOSTA IN ARCHIVIO
«include»
«include»
«include»
«include»
Importa da file batch
«include»
Visualizza Archivio
«extend»
GESTIONE PROPOSTE
VISUALIZZA BACHECA (Configuratore)
Sistema
LOGOUT
MENU IMPORTAZIONE BATCH
VISUALIZZA ARCHIVIO PROPOSTE
«include»
«include»
Pubblica proposta
«include»
Importa sequenza
ELIMINA PROPOSTA DALLA SESSIONE
«include»
Continua compilazione
PUBBLICA PROPOSTA IN BACHECA
ESECUZIONE TRANSIZIONI AUTOMATICHE DI STATO
IMPORTA PIU FILE BATCH IN SEQUENZA
«include»
Importa singolo
«include»
«extend»
[proposta VALIDA]
«include»
[Per ogni file]
COMPILA / MODIFICA CAMPI PROPOSTA
IMPORTA SINGOLO FILE BATCH
«extend»
[proposta VALIDA]
CREA PROPOSTA

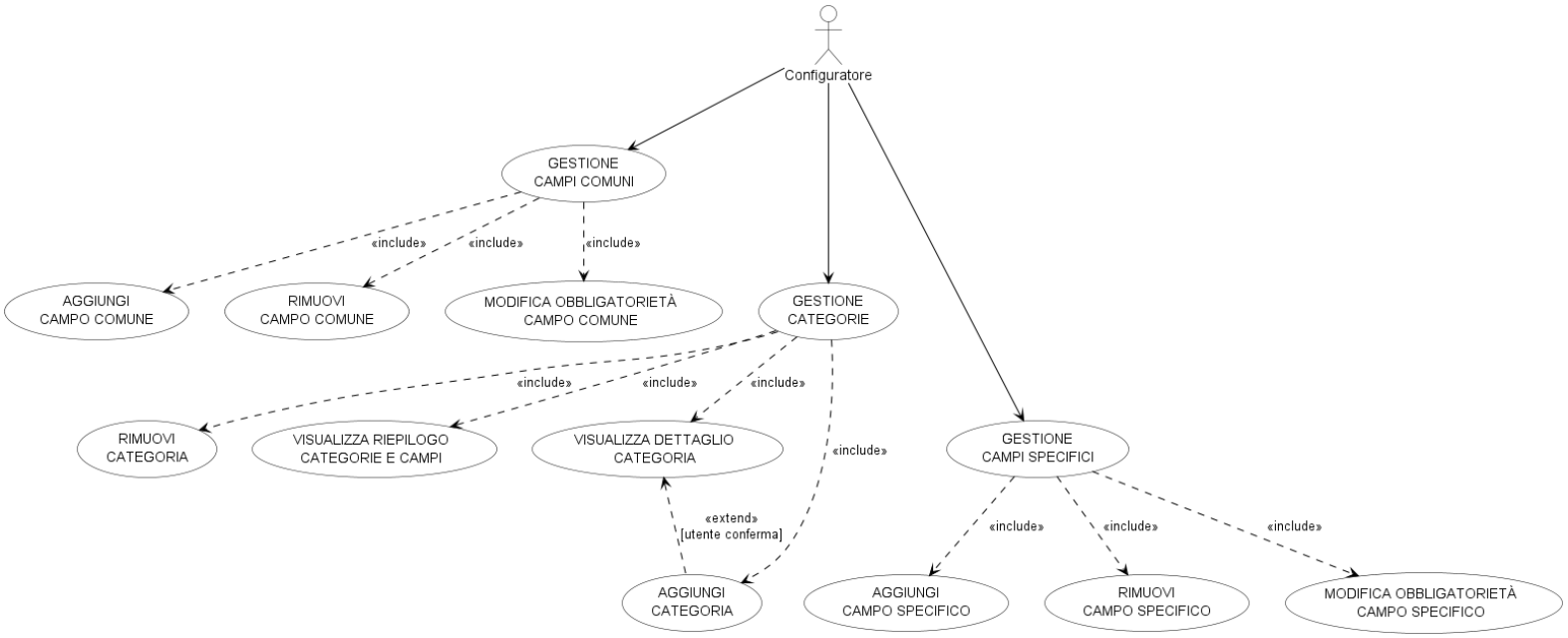
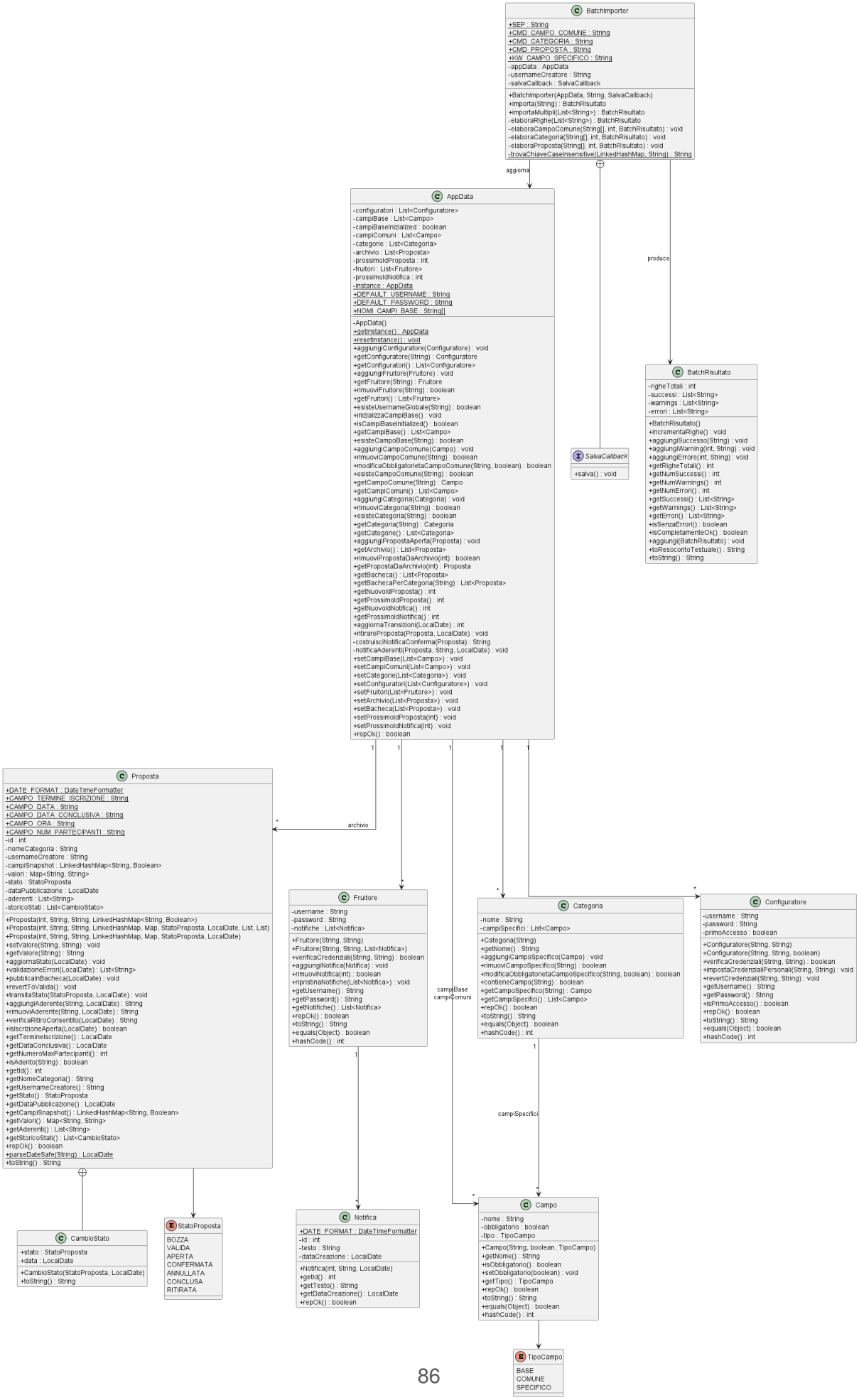
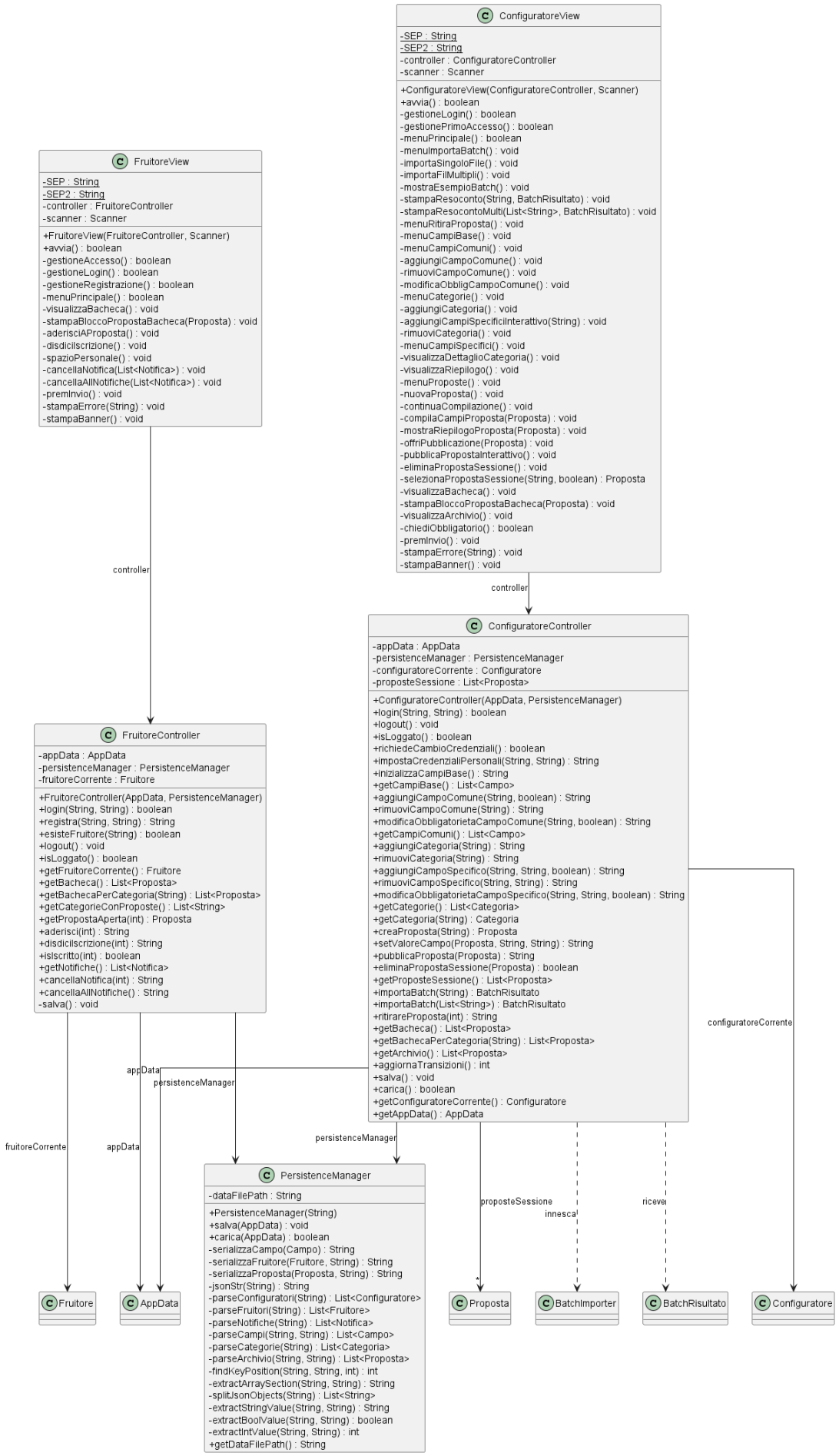

Configuratore
GESTIONE CAMPI COMUNI
«include»
«include»
«include»
AGGIUNGI CAMPO COMUNE
RIMUOVI CAMPO COMUNE
MODIFICA OBBLIGATORIETÀ CAMPO COMUNE
GESTIONE CATEGORIE
«include»
«include»
«include»
RIMUOVI CATEGORIA
VISUALIZZA RIEPILOGO CATEGORIE E CAMPI
VISUALIZZA DETTAGLIO CATEGORIA
«include»
GESTIONE CAMPI SPECIFICI
«extend»
[utente conferma]
«include»
«include»
«include»
AGGIUNGI CATEGORIA
AGGIUNGI CAMPO SPECIFICO
RIMUOVI CAMPO SPECIFICO
MODIFICA OBBLIGATORIETÀ CAMPO SPECIFICO

Diagramma delle classi


BatchImporter
+SEP : String
+CMD_CAMPO_COMUNE : String
+CMD_CATEGORIA : String
+CMD_PROPOSTA : String
+kW_CAMPO_SPECIFICO : String
-appData : AppData
-usernameCreatore : String
-salvaCallback : SalvaCallback
+BatchImporter(AppData, String, SalvaCallback)
+importa(String) : BatchRisultato
+importaMultipli(List<String>) : BatchRisultato
-elaboraRighe(List<String>) : BatchRisultato
-elaboraCampoComune(String[], int, BatchRisultato) : void
-elaboraCategoria(String[], int, BatchRisultato) : void
-elaboraProposta(String[], int, BatchRisultato) : void
-trovaChiaveCaseInsensitive(LinkedHashMap, String) : String
aggiorna
produce
AppData
-configuratori : List<Configuratore>
-campiBase : List<Campo>
-campiBaseInizializzed : boolean
-campiComuni : List<Campo>
-categorie : List<Categoria>
-archivio : List<Proposta>
-prossimoIdProposta : int
-fruitori : List<Fruitore>
-prossimoIdNotifica : int
-instance : AppData
+DEFAULT_USERNAME : String
+DEFAULT_PASSWORD : String
+NOMI_CAMPI_BASE : String[]
-AppData()
+getInstance() : AppData
+resetInstance() : void
+aggiungiConfiguratore(Configuratore) : void
+getConfiguratore(String) : Configuratore
+getConfiguratori() : List<Configuratore>
+aggiungiFruitore(Fruitore) : void
+getFruitore(String) : Fruitore
+rimuoviFruitore(String) : boolean
+getFruitori() : List<Fruitore>
+esisteUsernameGlobale(String) : boolean
+inizializzaCampiBase() : void
+isCampiBaseInitialized() : boolean
+getCampiBase() : List<Campo>
+esisteCampoBase(String) : boolean
+aggiungiCampoComune(Campo) : void
+rimuoviCampoComune(String) : boolean
+modificaObbligatorietaCampoComune(String, boolean) : boolean
+esisteCampoComune(String) : boolean
+getCampoComune(String) : Campo
+getCampiComuni() : List<Campo>
+aggiungiCategoria(Categoria) : void
+rimuoviCategoria(String) : boolean
+esisteCategoria(String) : boolean
+getCategoria(String) : Categoria
+getCategorie() : List<Categoria>
+aggiungiPropostaAperta(Proposta) : void
+getArchivio() : List<Proposta>
+rimuoviPropostaDaArchivio(int) : boolean
+getPropostaDaArchivio(int) : Proposta
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getNuovoIdProposta() : int
+getProssimoIdProposta() : int
+getNuovoIdNotifica() : int
+getProssimoIdNotifica() : int
+aggiornaTransizioni(LocalDate) : int
+ritirareProposta(Proposta, LocalDate) : void
-costruisciNotificaConferma(Proposta) : String
-notificaAderenti(Proposta, String, LocalDate) : void
+setCampiBase(List<Campo>) : void
+setCampiComuni(List<Campo>) : void
+setCategorie(List<Categoria>) : void
+setConfiguratori(List<Configuratore>) : void
+setFruitori(List<Fruitore>) : void
+setArchivio(List<Proposta>) : void
+setBacheca(List<Proposta>) : void
+setProssimoIdProposta(int) : void
+setProssimoIdNotifica(int) : void
+repOk() : boolean
SalvaCallback
+salva() : void
BatchRisultato
-righeTotali : int
-successi : List<String>
-warnings : List<String>
-errori : List<String>
+BatchRisultato()
+incrementaRighe() : void
+aggiungiSuccesso(String) : void
+aggiungiWarning(int, String) : void
+aggiungiErrore(int, String) : void
+getRigheTotali() : int
+getNumSuccessi() : int
+getNumWarnings() : int
+getNumErrori() : int
+getSuccessi() : List<String>
+getWarnings() : List<String>
+getErrori() : List<String>
+isSenzaErrori() : boolean
+isCompletamenteOk() : boolean
+aggiungi(BatchRisultato) : void
+toResocontoTestuale() : String
+toString() : String
1
1
1
1
1
archivio
*
campiBase
campiComuni
Proposta
+DATE_FORMAT : DateTimeFormatter
+CAMPO_TERMINE_ISCRIZIONE : String
+CAMPO_DATA : String
+CAMPO_DATA_CONCLUSIVA : String
+CAMPO_ORA : String
+CAMPO_NUM_PARTECIPANTI : String
-id : int
-nomeCategoria : String
-usernameCreatore : String
-campiSnapshot : LinkedHashMap<String, Boolean>
-valori : Map<String, String>
-stato : StatoProposta
-dataPubblicazione : LocalDate
-aderenti : List<String>
-storicoStati : List<CambioStato>
+Proposta(int, String, String, LinkedHashMap<String, Boolean>)
+Proposta(int, String, String, LinkedHashMap, Map, StatoProposta, LocalDate, List, List)
+Proposta(int, String, String, LinkedHashMap, Map, StatoProposta, LocalDate)
+setValore(String, String) : void
+getValore(String) : String
+aggiornaStato(LocalDate) : void
+validazioneErrori(LocalDate) : List<String>
+pubblicaInBacheca(LocalDate) : void
+revertToValida() : void
+transitaStato(StatoProposta, LocalDate) : void
+aggiungiAderente(String, LocalDate) : String
+rimuoviAderente(String, LocalDate) : String
+verificaRitiroConsentito(LocalDate) : String
+isIscrizioneAperta(LocalDate) : boolean
+getTermineIscrizione() : LocalDate
+getDataConclusiva() : LocalDate
+getNumeroMaxPartecipanti() : int
+isAderito(String) : boolean
+getId() : int
+getNomeCategoria() : String
+getUsernameCreatore() : String
+getStato() : StatoProposta
+getDataPubblicazione() : LocalDate
+getCampiSnapshot() : LinkedHashMap<String, Boolean>
+getValori() : Map<String, String>
+getAderenti() : List<String>
+getStoricoStati() : List<CambioStato>
+repOk() : boolean
+parseDateSafe(String) : LocalDate
+toString() : String
Fruitore
-username : String
-password : String
-notifiche : List<Notifica>
+Fruitore(String, String)
+Fruitore(String, String, List<Notifica>)
+verificaCredenziali(String, String) : boolean
+aggiungiNotifica(Notifica) : void
+rimuoviNotifica(int) : boolean
+ripristinaNotifiche(List<Notifica>) : void
+getUsername() : String
+getPassword() : String
+getNotifiche() : List<Notifica>
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
Categoria
-nome : String
-campiSpecifici : List<Campo>
+Categoria(String)
+getNome() : String
+aggiungiCampoSpecifico(Campo) : void
+rimuoviCampoSpecifico(String) : boolean
+modificaObbligatorietaCampoSpecifico(String, boolean) : boolean
+contieneCampo(String) : boolean
+getCampoSpecifico(String) : Campo
+getCampiSpecifici() : List<Campo>
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
Configuratore
-username : String
-password : String
-primoAccesso : boolean
+Configuratore(String, String)
+Configuratore(String, String, boolean)
+verificaCredenziali(String, String) : boolean
+impostaCredenzialiPersonali(String, String) : void
+revertCredenziali(String, String) : void
+getUsername() : String
+getPassword() : String
+isPrimoAccesso() : boolean
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
campiSpecifici
CambioStato
+stato : StatoProposta
+data : LocalDate
+CambioStato(StatoProposta, LocalDate)
+toString() : String
StatoProposta
BOZZA
VALIDA
APERTA
CONFERMATA
ANNULLATA
CONCLUSA
RITIRATA
Notifica
+DATE_FORMAT : DateTimeFormatter
-id : int
-testo : String
-dataCreazione : LocalDate
+Notifica(int, String, LocalDate)
+getId() : int
+getTesto() : String
+getDataCreazione() : LocalDate
+repOk() : boolean
Campo
-nome : String
-obbligatorio : boolean
-tipo : TipoCampo
+Campo(String, boolean, TipoCampo)
+getNome() : String
+isObbligatorio() : boolean
+setObbligatorio(boolean) : void
+getTipo() : TipoCampo
+repOk() : boolean
+toString() : String
+equals(Object) : boolean
+hashCode() : int
TipoCampo
BASE
COMUNE
SPECIFICO


ConfiguratoreView
-SEP : String
-SEP2 : String
-controller : ConfiguratoreController
-scanner : Scanner
+ConfiguratoreView(ConfiguratoreController, Scanner)
+avvia() : boolean
-gestioneLogin() : boolean
-gestionePrimoAccesso() : boolean
-menuPrincipale() : boolean
-menuImportaBatch() : void
-importaSingoloFile() : void
-importaFilMultipli() : void
-mostraEsempioBatch() : void
-stampaResoconto(String, BatchRisultato) : void
-stampaResocontoMulti(List<String>, BatchRisultato) : void
-menuRitiraProposta() : void
-menuCampiBase() : void
-menuCampiComuni() : void
-aggiungiCampoComune() : void
-rimuoviCampoComune() : void
-modificaObbligCampoComune() : void
-menuCategorie() : void
-aggiungiCategoria() : void
-aggiungiCampiSpecificiInterattivo(String) : void
-rimuoviCategoria() : void
-menuCampiSpecifici() : void
-visualizzaDettaglioCategoria() : void
-visualizzaRiepilogo() : void
-menuProposte() : void
-nuovaProposta() : void
-continuaCompilazione() : void
-compilaCampiProposta(Proposta) : void
-mostraRiepilogoProposta(Proposta) : void
-offriPubblicazione(Proposta) : void
-pubblicaPropostaInterattivo() : void
-eliminaPropostaSessione() : void
-selezionaPropostaSessione(String, boolean) : Proposta
-visualizzaBacheca() : void
-stampaBloccoPropostaBacheca(Proposta) : void
-visualizzaArchivio() : void
-chiediObbligatorio() : boolean
-premInvio() : void
-stampaErrore(String) : void
-stampaBanner() : void
FruitoreView
-SEP : String
-SEP2 : String
-controller : FruitoreController
-scanner : Scanner
+FruitoreView(FruitoreController, Scanner)
+avvia() : boolean
-gestioneAccesso() : boolean
-gestioneLogin() : boolean
-gestioneRegistrazione() : boolean
-menuPrincipale() : boolean
-visualizzaBacheca() : void
-stampaBloccoPropostaBacheca(Proposta) : void
-aderisciAProposta() : void
-disdicilscrizione() : void
-spazioPersonale() : void
-cancellaNotifica(List<Notifica>) : void
-cancellaAllNotifiche(List<Notifica>) : void
-premInvio() : void
-stampaErrore(String) : void
-stampaBanner() : void
controller
controller
ConfiguratoreController
-appData : AppData
-persistenceManager : PersistenceManager
-configuratoreCorrente : Configuratore
-proposteSessione : List<Proposta>
+ConfiguratoreController(AppData, PersistenceManager)
+login(String, String) : boolean
+logout() : void
+isLoggato() : boolean
+richiedeCambioCredenziali() : boolean
+impostaCredenzialiPersonali(String, String) : String
+inizializzaCampiBase() : String
+getCampiBase() : List<Campo>
+aggiungiCampoComune(String, boolean) : String
+rimuoviCampoComune(String) : String
+modificaObbligatorietaCampoComune(String, boolean) : String
+getCampiComuni() : List<Campo>
+aggiungiCategoria(String) : String
+rimuoviCategoria(String) : String
+aggiungiCampoSpecifico(String, String, boolean) : String
+rimuoviCampoSpecifico(String, String) : String
+modificaObbligatorietaCampoSpecifico(String, String, boolean) : String
+getCategorie() : List<Categoria>
+getCategoria(String) : Categoria
+creaProposta(String) : Proposta
+setValoreCampo(Proposta, String, String) : String
+pubblicaProposta(Proposta) : String
+eliminaPropostaSessione(Proposta) : boolean
+getProposteSessione() : List<Proposta>
+importaBatch(String) : BatchRisultato
+importaBatch(List<String>) : BatchRisultato
+ritirareProposta(int) : String
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getArchivio() : List<Proposta>
+aggiornaTransizioni() : int
+salva() : void
+carica() : boolean
+getConfiguratoreCorrente() : Configuratore
+getAppData() : AppData
FruitoreController
-appData : AppData
-persistenceManager : PersistenceManager
-fruitoreCorrente : Fruitore
+FruitoreController(AppData, PersistenceManager)
+login(String, String) : boolean
+registra(String, String) : String
+esisteFruitore(String) : boolean
+logout() : void
+isLoggato() : boolean
+getFruitoreCorrente() : Fruitore
+getBacheca() : List<Proposta>
+getBachecaPerCategoria(String) : List<Proposta>
+getCategorieConProposte() : List<String>
+getPropostaAperta(int) : Proposta
+aderisci(int) : String
+disdicilscrizione(int) : String
+isIscritto(int) : boolean
+getNotifiche() : List<Notifica>
+cancellaNotifica(int) : String
+cancellaAllNotifiche() : String
-salva() : void
configuratoreCorrente
appData
persistenceManager
fruitoreCorrente
appData
persistenceManager
PersistenceManager
-dataFilePath : String
+PersistenceManager(String)
+salva(AppData) : void
+carica(AppData) : boolean
-serializzaCampo(Campo) : String
-serializzaFruitore(Fruitore, String) : String
-serializzaProposta(Proposta, String) : String
-jsonStr(String) : String
-parseConfiguratori(String) : List<Configuratore>
-parseFruitori(String) : List<Fruitore>
-parseNotifiche(String) : List<Notifica>
-parseCampi(String, String) : List<Campo>
-parseCategorie(String) : List<Categoria>
-parseArchivio(String, String) : List<Proposta>
-findKeyPosition(String, String, int) : int
-extractArraySection(String, String) : String
-splitJsonObjects(String) : List<String>
-extractStringValue(String, String) : String
-extractBoolValue(String, String) : boolean
-extractIntValue(String, String) : int
+getDataFilePath() : String
proposteSessione
*
innesca
ricever
Fruitore
AppData
Proposta
BatchImporter
BatchRisultato
Configuratore

EasyEvent

Manuale di Installazione e Uso

Versione 5

Sistema software per la gestione di iniziative ricreative,
amatoriali e dilettantistiche

Back-end per configuratori · Front-end per fruitori · Importazione batch

Insegnamento	Ingegneria del Software (9 CFU)
Anno accademico	2025-2026
Corso	III anno – Ingegneria Informatica
Docente	Marina Zanella
Ateneo	Università degli Studi di Brescia

Autori	Riccardo Marini	matr. 746230
	Raffaele Principe	matr. 749438

Sommario

Il presente documento costituisce il **Manuale di Installazione e Uso** dell'applicazione **EasyEvent**, sviluppata nell'ambito del progetto di *Ingegneria del Software* (A.A. 2025-2026). Il manuale è relativo alla **Versione 5**, che rappresenta la versione finale e completa del sistema.

Scopo dell'applicazione. EasyEvent è un sistema stand-alone per l'organizzazione di iniziative ricreative, amatoriali e dilettantistiche. Permette a enti senza scopo di lucro (circoli aziendali, uffici del tempo libero comunali, ecc.) di proporre eventi ai propri utenti, gestire le iscrizioni e comunicare automaticamente l'esito di ciascuna iniziativa.

Due tipologie di utente:

- **Configuratore**: accede al back-end, definisce categorie e campi, crea e pubblica proposte, gestisce l'archivio e può ritirare iniziative già confermate.
- **Fruitore**: accede al front-end, visualizza le proposte in bacheca, si iscrive alle iniziative di suo interesse e riceve notifiche sull'esito.

Novità della Versione 5. Rispetto alla versione precedente, la Versione 5 introduce la **modalità batch**: il configuratore può importare categorie, campi e proposte di iniziative leggendole da uno o più file di testo strutturati, senza dover inserire ogni dato manualmente tramite la CLI. La modalità interattiva rimane invariata e pienamente operativa.

Struttura del manuale.

- Capitolo 1: requisiti di sistema e istruzioni di installazione.
- Capitolo 2: avvio dell'applicazione e primo accesso.
- Capitolo 3: guida completa al back-end del configuratore.
- Capitolo 4: guida completa al front-end del fruitore.
- Capitolo 5: importazione batch (funzionalità Versione 5).
- Capitolo 6: formato dei dati e gestione del file di persistenza.
- Capitolo 7: messaggi di errore e risoluzione dei problemi.

- Appendice A: credenziali demo precaricate.
- Appendice B: file batch di esempio completo.

Indice

CAPITOLO 1

Installazione

1.1 Requisiti di sistema

1.1.1 Requisiti hardware

EasyEvent è un'applicazione leggera; non è richiesta alcuna configurazione hardware specifica.

- **Processore**: qualsiasi CPU moderna a 64 bit.
- **RAM**: almeno 128 MB disponibili per la JVM (512 MB consigliati).
- **Spazio su disco**: circa 2 MB per il codice compilato; ulteriore spazio per il file di dati `easyevent_data.json` (dimensione variabile in base al numero di proposte e fruitori).

1.1.2 Requisiti software

- **Sistema operativo**: Windows 10/11, Linux (qualsiasi distribuzione recente), macOS 11+.
- **Java Development Kit (JDK) versione 17 o superiore** (richiesto per la compilazione). Download: https://adoptium.net oppure https://www.oracle.com/java/technologies/downloads/
- **Java Runtime Environment (JRE) versione 17 o superiore** (sufficiente per la sola esecuzione del codice pre-compilato).

> **Nota.** Il progetto non richiede alcun framework, database, server web o libreria esterna. L'unica dipendenza è il JDK/JRE standard.

1.1.3 Verifica della versione Java installata

Aprire un terminale (o il Prompt dei comandi su Windows) e digitare:

```
java -version
```

L'output atteso è simile a:

```
openjdk version "21.0.x" ...
```

Se il comando non viene riconosciuto oppure la versione è inferiore a 17, installare il JDK come indicato alla sezione precedente.

Per compilare, è necessario anche `javac`:

```
javac -version
# output atteso: javac 21.0.x
```

1.2 Ottenere i file del progetto

Il progetto è distribuito come cartella `Versione 5/` contenente:

- `src/` — sorgenti Java.
- `data/` — file di persistenza JSON (creato automaticamente al primo salvataggio).
- `batch_examples/` — file batch di esempio per la funzionalità V5.
- `run.sh` / `run.bat` — script di compilazione ed esecuzione.
- `sources.txt` — elenco dei file sorgente per `javac`.

> **Attenzione.** Non spostare singoli file dalla cartella di progetto: gli script e il programma fanno riferimento a percorsi relativi. Spostare sempre l'intera cartella `Versione 5/`.

1.3 Compilazione

1.3.1 Linux e macOS — script automatico

```
cd "Versione 5"
chmod +x run.sh
./run.sh
```

Lo script compila automaticamente tutti i file Java nella cartella `out/` e avvia l'applicazione.

1.3.2 Windows — script automatico

```
cd "Versione 5"
run.bat
```

Lo script è equivalente a quello per Linux/macOS.

1.3.3 Compilazione manuale (qualsiasi sistema operativo)

Eseguire i comandi seguenti dalla cartella radice del progetto (`Versione 5/`):

```
# Linux / macOS
mkdir -p out
```

```
find src -name "*.java" > sources.txt
javac -encoding UTF-8 -d out @sources.txt

# Windows (Prompt dei comandi)
mkdir out
dir /s /b src\*.java > sources.txt
javac -encoding UTF-8 -d out @sources.txt
```

Compilazione riuscita. Se non appaiono messaggi di errore, il codice compilato si trova nella cartella `out/` ed è pronto per l'esecuzione.

1.4 Prima esecuzione

Dopo la compilazione, avviare il programma con:

```
# Linux / macOS
java -ea -cp out it.easyevent.v5.MainV5

# Windows
java -ea -cp out it.easyevent.v5.MainV5
```

Il flag `-ea` attiva le asserzioni Java (*assertions*), utili per verificare le precondizioni e le postcondizioni durante lo sviluppo e i test.

Nota. Gli script `run.sh` e `run.bat` includono già il flag `-ea` e il percorso corretto; si raccomanda di usarli per l'avvio ordinario.

CAPITOLO 2

Avvio e navigazione generale

2.1 Menu di selezione del ruolo

All'avvio, l'applicazione esegue automaticamente:

1. Caricamento dei dati dal file `data/easyevent_data.json` (se esistente).
2. Inizializzazione dei campi base (solo al primo avvio assoluto).
3. Aggiornamento automatico degli stati delle proposte scadute.

Successivamente viene mostrato il menu di selezione del ruolo:

```
------------------------------------------------------------
  EasyEvent  -  Versione 5
------------------------------------------------------------
  Con quale ruolo vuoi accedere?
  1. Configuratore  (back-end + importazione batch)
  2. Fruitore       (front-end)
  0. Esci
```

Digitare il numero corrispondente al ruolo desiderato e premere `INVIO`.

2.2 Convenzioni di interfaccia

- **Navigazione**: ogni schermata mostra un elenco di voci numerate o con lettere. Digitare il numero/lettera corrispondente e premere `INVIO`.
- **Conferma**: dove richiesto, digitare `s` per confermare o `n` per annullare.
- **Uscita**: digitare `0` in qualsiasi menu per tornare al livello superiore o effettuare il logout.
- **Annullamento**: in alcune schermate è possibile digitare `annulla` per interrompere un'operazione in corso.
- **Percorsi file**: possono essere assoluti (es. `/home/utente/file.batch`) o relativi alla directory di lavoro corrente (es. `batch_examples/setup.batch`).
- **Formato date**: tutte le date vanno inserite nel formato **gg/mm/aaaa** (es. `25/06/2026`).

- **Formato ore**: nel formato **HH:MM** (es. `09:30`, `14:00`).

2.3 Transizioni automatiche di stato

A ogni avvio dell'applicazione il sistema verifica automaticamente le proposte in bacheca e aggiorna i loro stati:

Da	A	Condizione
APERTA	CONFERMATA	Oggi > Termine iscrizione **e** aderenti ≥ N. partecipanti
APERTA	ANNULLATA	Oggi > Termine iscrizione **e** aderenti < N. partecipanti
CONFERMATA	CONCLUSA	Oggi > Data conclusiva

Alla transizione APERTA→CONFERMATA o APERTA→ANNULLATA vengono generate automaticamente notifiche per tutti i fruitori iscritti.

CAPITOLO 3

Guida al back-end del configuratore

3.1 Accesso e gestione delle credenziali

3.1.1 Primo accesso

Al primo avvio assoluto dell'applicazione, le credenziali predefinite sono:

- **Username**: `admin`
- **Password**: `admin123`

Il sistema riconosce automaticamente il primo accesso e richiede obbligatoriamente la scelta di credenziali personali prima di consentire qualsiasi altra operazione. Sono valide qualsiasi combinazione di username e password non vuote, purché lo username non sia già in uso.

> **Importante.** Lo username scelto deve essere univoco in tutto il sistema (non deve coincidere con quello di altri configuratori né di alcun fruitore già registrato). La parola `esci` è riservata e non può essere usata come username.

3.1.2 Accessi successivi

Inserire username e password scelti al primo accesso. Il sistema concede massimo 3 tentativi prima di bloccare l'accesso per la sessione corrente.

3.1.3 Multi-configuratore

Il sistema supporta più configuratori. Il secondo configuratore (e quelli successivi) devono ricevere le credenziali predefinite da un configuratore già registrato. L'interpretazione adottata è quella semplice: le credenziali predefinite (`admin` / `admin123`) sono condivise e fisse.

3.2 Menu principale del configuratore

```
MENU PRINCIPALE  [mario]
1. Gestione campi base
2. Gestione campi comuni
3. Gestione categorie
4. Visualizza riepilogo categorie e campi
```

```
5. Gestione proposte          [N in sessione]
6. Visualizza bacheca         [N proposte aperte]
7. Visualizza archivio proposte
8. Ritira proposta            [N ritirabili]
9. Importa da file batch      [NUOVO V5]
0. Logout
```

3.3 Gestione dei campi

3.3.1 Campi base (voce 1)

I campi base sono definiti una volta sola al primo avvio e non possono essere modificati per tutta la vita operativa dell'applicazione. Questa schermata li visualizza a titolo informativo.

I campi base predefiniti sono:

Nome campo	Descrizione
Titolo	Nome descrittivo dell'iniziativa.
Numero di partecipanti	Numero esatto di persone richieste.
Termine ultimo di iscrizione	Ultimo giorno per iscriversi (ore 23:59).
Luogo	Indirizzo o luogo di ritrovo.
Data inizio	Data di inizio (o data unica) dell'iniziativa.
Ora	Ora di inizio.
Quota individuale	Spesa a carico di ciascun partecipante (può essere 0).
Data conclusiva	Data di fine (coincide con Data inizio per eventi di un giorno solo).

3.3.2 Campi comuni (voce 2)

I campi comuni sono condivisi da tutte le categorie e possono essere aggiunti, rimossi o modificati in qualsiasi momento. Le modifiche non hanno effetto sulle proposte già pubblicate; si applicano solo alle nuove.

Opzione	Comando	Descrizione
Aggiungi	`a`	Inserire nome e obbligatorietà (s/n).
Rimuovi	`r`	Inserire il nome del campo da eliminare.
Modifica	`m`	Cambiare l'obbligatorietà di un campo esistente.

Esempi di campi comuni utili: *Durata*, *Note*, *Compreso nella quota*, *Ora conclusiva*.

3.3.3 Campi specifici (voce 3 — sotto-menu campi specifici)

I campi specifici sono propri di una singola categoria e si gestiscono all'interno del menu categorie (§3.4).

3.4 Gestione delle categorie

3.4.1 Aggiunta di una categoria (voce 3 – a)

1. Selezionare `a` dal menu categorie.
2. Inserire il nome della nuova categoria (es. *Sport*, *Arte*, *Gite*).
3. Il sistema chiede se aggiungere subito campi specifici; rispondere `s` per farlo o `n` per farlo in un secondo momento.

3.4.2 Gestione dei campi specifici (voce 3 – c)

1. Selezionare `c` e inserire il nome della categoria.
2. Dal sotto-menu scegliere: `a` (aggiungi), `r` (rimuovi), `m` (modifica obbligatorietà).

> **Nota.** Un campo specifico non può avere lo stesso nome di un campo base o comune. I nomi sono confrontati senza distinzione tra maiuscole e minuscole.

3.4.3 Rimozione di una categoria (voce 3 – r)

Rimuovere una categoria elimina anche tutti i suoi campi specifici. La rimozione non ha effetto sulle proposte già pubblicate; si applica solo alle proposte future.

3.4.4 Visualizzazione (voce 4)

La voce 4 mostra un riepilogo completo di tutti i campi base, comuni e specifici di ogni categoria.

3.5 Gestione delle proposte

3.5.1 Ciclo di vita di una proposta

Stato	Significato
BOZZA	Proposta in creazione; uno o più campi obbligatori mancanti o vincoli di data non rispettati. Non viene salvata se non pubblicata.
VALIDA	Tutti i campi obbligatori compilati e vincoli rispettati. Non ancora pubblicata. Non viene salvata se non pubblicata.
APERTA	Pubblicata in bacheca; iscrizioni attive. Salvata su file.
CONFERMATA	Termine scaduto, aderenti $\geq$ N. partecipanti. Notifica inviata.
ANNULLATA	Termine scaduto, aderenti $<$ N. partecipanti. Notifica inviata.
CONCLUSA	Giorno successivo alla Data conclusiva (solo da Confermata).
RITIRATA	Ritirata dal configuratore (da Aperta o Confermata). Notifica inviata.

3.5.2 Creazione di una proposta (voce 5 – n)

1. Selezionare la categoria tra quelle disponibili.
2. Compilare i campi presentati uno per uno. I campi **[OBB]**ligatòri devono essere valorizzati; i **[FAC]**oltativi possono essere saltati premendo `INVIO`.
3. Al termine il sistema mostra un riepilogo e lo stato (BOZZA o VALIDA).
4. Se la proposta è VALIDA, il sistema offre di pubblicarla immediatamente.

3.5.2.1 Vincoli di data obbligatori

- **Termine ultimo di iscrizione** deve essere successivo alla data odierna.
- **Data inizio** deve essere almeno 2 giorni dopo il Termine ultimo di iscrizione.
- **Data conclusiva** non può essere precedente alla Data inizio.
- **Numero di partecipanti** deve essere un intero strettamente positivo.

3.5.3 Continuare la compilazione (voce 5 – c)

Permette di riprendere la compilazione di una proposta già creata nella sessione corrente, inserendo l'ID della proposta desiderata.

3.5.4 Pubblicazione (voce 5 – p)

Elenca le proposte VALIDE in sessione e chiede quale pubblicare. La proposta passa allo stato APERTA e viene salvata su file; la proposta BOZZA o VALIDA non pubblicata viene scartata alla fine della sessione.

3.5.5 Eliminazione dalla sessione (voce 5 – e)

Rimuove una proposta non ancora pubblicata dalla sessione corrente. L'operazione è irreversibile.

3.6 Visualizzazione della bacheca (voce 6)

Mostra le proposte in stato APERTA, organizzate per categoria. Per ogni proposta sono visibili: ID, data di pubblicazione, numero di iscritti, tutti i campi compilati.

3.7 Visualizzazione dell'archivio (voce 7)

Mostra tutte le proposte pubblicate in qualsiasi stato (APERTA, CONFERMATA, ANNULLATA, CONCLUSA, RITIRATA), raggruppate per stato. Per ogni proposta sono visibili: ID, titolo, categoria, storico degli stati con date, elenco degli aderenti.

3.8 Ritiro di una proposta (voce 8)

Il ritiro è una misura eccezionale per cause di forza maggiore (es. chiusura improvvisa del luogo dell'evento).

1. La voce 8 mostra solo le proposte ritirabili: quelle in stato APERTA o CONFERMATA la cui Data inizio è ancora futura rispetto a oggi.
2. Selezionare l'ID della proposta da ritirare.
3. Confermare con `s`.
4. Il sistema transita la proposta in stato RITIRATA e invia automaticamente una notifica a tutti i fruitori iscritti.

> **Attenzione.** Il ritiro è irreversibile. Non è possibile riportare una proposta dallo stato RITIRATA a qualsiasi altro stato.

CAPITOLO 4

Guida al front-end del fruitore

4.1 Registrazione e accesso

4.1.1 Registrazione (primo accesso)

1. Dal menu di selezione ruolo scegliere **2. Fruitore**.
2. Scegliere **2. Registrazione**.
3. Inserire lo username desiderato (deve essere univoco in tutto il sistema).
4. Inserire e confermare la password.

Dopo la registrazione l'accesso avviene automaticamente.

4.1.2 Login

1. Dal menu fruitore scegliere **1. Login**.
2. Inserire username e password. Massimo 3 tentativi.

4.2 Menu principale del fruitore

```
MENU FRUITORE  [alice]
1. Visualizza bacheca
2. Aderisci a una proposta
3. Disdici iscrizione         [N iscrizioni attive]
4. Spazio personale           [N notifiche]
0. Logout
```

4.3 Visualizzazione della bacheca (voce 1)

Mostra le proposte aperte, organizzate per categoria, con tutte le informazioni rilevanti: titolo, posti disponibili, termine di iscrizione, data, ora, luogo, quota, campi specifici.

4.4 Adesione a una proposta (voce 2)

1. Il sistema mostra le proposte aperte a cui non si è ancora iscritti, con il numero di posti rimasti.
2. Inserire l'ID della proposta desiderata (o `0` per annullare).

4.4.0.1 Condizioni per l'iscrizione

- La proposta deve essere in stato APERTA.
- La data odierna deve essere $\leq$ Termine ultimo di iscrizione.
- Non si è già iscritti alla stessa proposta.
- Non è ancora stato raggiunto il numero massimo di partecipanti.

4.5 Disdetta dell'iscrizione (voce 3)

1. Il sistema mostra le proposte aperte a cui si è iscritti.
2. Selezionare la proposta e confermare con `s`.
3. La disdetta è possibile solo entro il Termine ultimo di iscrizione.
4. Dopo la disdetta è possibile re-iscriversi alla stessa proposta (sempre entro il termine).

4.6 Spazio personale e notifiche (voce 4)

Lo spazio personale raccoglie tutte le notifiche automatiche generate dal sistema:

- **Proposta confermata**: con promemoria di data, ora, luogo e quota.
- **Proposta annullata**: avviso di cancellazione per iscritti insufficienti.
- **Proposta ritirata**: avviso di ritiro da parte del configuratore.

Dal menu spazio personale:

- `c` — cancellare una singola notifica (inserire l'ID).
- `t` — cancellare tutte le notifiche.

Le notifiche persistono tra una sessione e l'altra; una notifica cancellata non è recuperabile.

CAPITOLO 5

Importazione batch NUOVO V5

5.1 Panoramica

La modalità batch consente al configuratore di importare **campi comuni, categorie e proposte** leggendo uno o più file di testo strutturati, invece di inserirli manualmente tramite la CLI. È utile per:

- Configurare rapidamente il sistema al primo utilizzo.
- Importare stagionalmente un gran numero di proposte.
- Automatizzare l'inserimento di dati ricorrenti.

La modalità interattiva (voci 1–8 del menu) rimane disponibile e invariata in parallelo alla modalità batch.

5.2 Accesso alla funzionalità

Dal menu principale del configuratore selezionare «**9. Importa da file batch**»:

```
IMPORTAZIONE BATCH  [NUOVO V5]
s. Importa singolo file
m. Importa piu' file in sequenza
e. Mostra esempio del formato batch
0. Torna al menu principale
```

5.3 Formato dei file batch

I file batch sono file di testo in codifica **UTF-8**. Il separatore tra i token di ogni riga è « | » (spazio-pipe-spazio).

5.3.1 Commenti e righe vuote

Le righe che iniziano con # (dopo il trim degli spazi) vengono ignorate, così come le righe vuote.

```
# Questo e' un commento -- viene completamente ignorato

# Le righe vuote sopra e sotto sono anch'esse ignorate
```

5.3.2 Comando CAMPO_COMUNE

```
CAMPO_COMUNE | <nome> | <obbligatorio>
```

- `<obbligatorio>`: `si`, `sì`, `yes` o `true` per obbligatorio; qualsiasi altro valore per facoltativo.

Esempi:

```
CAMPO_COMUNE | Durata | no
CAMPO_COMUNE | Compreso nella quota | no
CAMPO_COMUNE | Note | no
CAMPO_COMUNE | Ora conclusiva | no
```

5.3.3 Comando CATEGORIA

```
# Categoria senza campi specifici:
CATEGORIA | <nome>

# Categoria con campi specifici inline:
CATEGORIA | <nome> | CAMPO_SPECIFICO | <nome_cs> | <obbligatorio>
   [| CAMPO_SPECIFICO | ...]
```

Esempi:

```
CATEGORIA | Cinema
CATEGORIA | Sport | CAMPO_SPECIFICO | Certificato medico | si
CATEGORIA | Gite | CAMPO_SPECIFICO | Mezzo di trasporto | no |
   CAMPO_SPECIFICO | Guida | si
```

5.3.4 Comando PROPOSTA

```
PROPOSTA | <categoria> | <campo1> = <valore1> [| <campo2> = <
   valore2>] ...
```

- I nomi dei campi sono confrontati senza distinzione tra maiuscole e minuscole.
- Le date seguono il formato `gg/mm/aaaa`; le ore il formato `HH:MM`.
- La proposta è validata con le stesse regole della modalità interattiva. Se valida, viene pubblicata in bacheca; altrimenti viene segnalata come errore e non salvata.

Esempio:

```
PROPOSTA | Sport | Titolo = Torneo di Padel | Numero di
   partecipanti = 8 |
```

```
Termine ultimo di iscrizione = 30/06/2026 | Luogo = Palestra
 Centrale |
Data inizio = 10/07/2026 | Ora = 09:00 | Quota individuale = 15 |
Data conclusiva = 10/07/2026 | Certificato medico = Si | Durata =
  6 ore
```

> **Nota.** Nella realtà ogni riga PROPOSTA deve essere scritta su una sola riga del file (nessun a-capo all'interno). L'esempio sopra usa interruzioni di riga solo per ragioni tipografiche.

5.4 Importazione di un singolo file

1. Selezionare `s` dal menu batch.
2. Inserire il percorso del file (assoluto o relativo alla directory di lavoro).
3. Il sistema elabora il file e mostra il resoconto al termine.

Esempio di percorso:

```
batch_examples/setup_completo.batch
/home/mario/mio_batch.txt
C:\Utenti\Mario\batch.txt
```

5.5 Importazione di più file in sequenza

1. Selezionare `m` dal menu batch.
2. Inserire i percorsi dei file uno alla volta, premendo `INVIO` dopo ciascuno.
3. Premere `INVIO` senza testo per avviare l'importazione.
4. Il sistema elabora i file nell'ordine inserito e mostra un resoconto aggregato.

> **Suggerimento.** Conviene importare prima il file di campi e categorie, poi quello delle proposte: le proposte batch richiedono che le categorie esistano già nel sistema.

5.6 Il resoconto di importazione

Al termine di ogni importazione il sistema mostra un resoconto strutturato:

```
Righe elaborate: 12
Successi:        9
Warning:         2
Errori:          1

--- OPERAZIONI COMPLETATE ---
v Campo comune aggiunto: 'Durata' [facoltativo]
```

```
v Categoria aggiunta: 'Fotografia' -- (senza campi specifici)
v Proposta pubblicata in bacheca: [ID 9] 'Uscita fotografica' --
 categoria: Fotografia
...

--- WARNING (operazioni saltate) ---
Riga 7: Campo comune 'Durata' gia' presente. Riga saltata.
Riga 11: Categoria 'Sport' gia' presente. Riga saltata.

--- ERRORI ---
Riga 15: Proposta non valida per la categoria 'Letteratura':
   'Termine ultimo di iscrizione' deve essere successivo alla
 data odierna.
```

5.6.0.1 Comportamento in caso di errori

- Ogni riga è elaborata **indipendentemente**: un errore su una riga non blocca le successive.
- Le operazioni andate a buon fine vengono salvate; quelle fallite non lasciano traccia (rollback automatico in memoria).
- Al termine è possibile correggere le righe errate nel file e reimportare.

5.7 File batch di esempio

Nella cartella `batch_examples/` sono disponibili tre file pronti all'uso:

File	Contenuto
`campi_e_categorie.batch`	Aggiunge campi comuni e nuove categorie.
`proposte.batch`	Pubblica proposte (include casi di errore intenzionali per test).
`setup_completo.batch`	File unico con campi, categorie e proposte. Ideale per la demo.

CAPITOLO 6

Formato dei dati e persistenza

6.1 File di persistenza

L'applicazione salva tutti i dati in un unico file JSON:

```
Versione 5/data/easyevent_data.json
```

Il file viene creato automaticamente al primo salvataggio. Non è necessario crearlo manualmente.

6.2 Cosa viene salvato

Dato	Note
Configuratori	Username, password (in chiaro[1]), stato primo accesso.
Fruitori	Username, password (in chiaro), elenco notifiche.
Campi base	Definiti una volta sola; immutabili.
Campi comuni	Con nome e obbligatorietà.
Categorie	Con nome e campi specifici.
Archivio proposte	Tutti gli stati (Aperta, Confermata, Annullata, Conclusa, Ritirata) con storico stati, aderenti e valori dei campi.

6.3 Cosa *non* viene salvato

- Proposte in stato BOZZA o VALIDA (esistono solo in memoria durante la sessione).
- Sessioni di lavoro (ogni avvio ricomincia dal file salvato).

6.4 Compatibilità retroattiva

Il formato del file JSON è compatibile con tutte le versioni precedenti:

[1] La specifica non richiede requisiti di sicurezza per questo progetto didattico.

- File prodotti dalla V4: letti correttamente dalla V5.
- File prodotti dalla V3 (campo `"archivio"`): letti correttamente.
- File prodotti dalla V2 (campo `"bacheca"`): letti correttamente (le proposte vengono migrate nell'archivio V3/V4/V5).

6.5 Backup e ripristino

Per eseguire un backup dei dati è sufficiente copiare il file `easyevent_data.json`. Per ripristinare un backup, sostituire il file corrente con quello di backup prima di avviare l'applicazione.

> **Attenzione.** Non modificare manualmente il file JSON durante l'esecuzione dell'applicazione; le modifiche verrebbero sovrascritte al successivo salvataggio.

CAPITOLO 7

Messaggi di errore e risoluzione dei problemi

7.1 Messaggi dell'applicazione

Messaggio	Causa e rimedio
Credenziali non valide.	Username o password errati. Verificare le credenziali e riprovare.
Username già in uso.	Lo username scelto è già registrato. Scegliere un username diverso.
Il nome non può essere vuoto.	È stato premuto INVIO senza inserire testo. Inserire un valore.
Esiste già un campo base/comune con nome...	Non è possibile creare un campo con lo stesso nome di un campo già esistente. Scegliere un nome diverso.
Esiste già una categoria con nome...	Nome categoria già in uso. Scegliere un nome diverso.
'Termine ultimo di iscrizione' deve essere successivo alla data odierna.	La data inserita è nel passato o è oggi. Inserire una data futura.
'Data inizio' deve essere almeno 2 giorni dopo 'Termine ultimo di iscrizione'.	La Data inizio è troppo vicina al termine di iscrizione. Aumentare l'intervallo.
'Data conclusiva' non può essere precedente a 'Data inizio'.	La data di fine è precedente alla data di inizio. Correggere le date.
'Numero di partecipanti' deve essere un numero intero positivo.	Il valore inserito è 0, negativo o non numerico. Inserire un numero intero ≥ 1.
Il termine ultimo di iscrizione è scaduto.	Non è possibile iscriversi o disdire dopo la scadenza.
La proposta ha raggiunto il numero massimo di partecipanti.	Non ci sono più posti disponibili per questa proposta.

Messaggio	Causa e rimedio
Sei già iscritto a questa proposta.	Un fruitore può avere al più un'iscrizione per proposta.
Il ritiro è consentito solo per proposte APERTE o CONFERMATE.	Non è possibile ritirare proposte in altri stati.
Non è più possibile ritirare la proposta: la data dell'evento è già oggi o passata.	Il ritiro deve avvenire almeno il giorno prima della Data inizio.
Notifica non trovata con ID…	L'ID inserito non corrisponde a nessuna notifica. Verificare l'ID nel riepilogo notifiche.

7.2 Messaggi dell'importazione batch

Messaggio	Causa e rimedio
File non trovato: …	Il percorso inserito non corrisponde a nessun file. Verificare il percorso (maiuscole/minuscole su Linux sono significative).
Comando non riconosciuto: '…'	Il primo token della riga non è CAMPO_COMUNE, CATEGORIA o PROPOSTA. Correggere il file batch.
CAMPO_COMUNE richiede almeno 3 token.	La riga ha meno di 3 sezioni separate da `|` . Verificare la sintassi.
Campo comune '…' già presente.	Warning: il campo esiste già. L'operazione viene saltata; le successive continuano.
Categoria '…' già presente.	Warning: la categoria esiste già. I campi specifici inline vengono ignorati.
Categoria '…' non trovata.	Una riga PROPOSTA fa riferimento a una categoria inesistente. Aggiungere prima la categoria (interattivamente o in batch).
Proposta non valida…	I valori inseriti non superano la validazione. Controllare date, numero di partecipanti e campi obbligatori.

Messaggio	Causa e rimedio
Campo aggiunto in memoria ma errore nel salvataggio (rollback eseguito).	Errore di I/O sul file di persistenza. Verificare i permessi sulla cartella `data/`. Il rollback garantisce la coerenza.

7.3 Problemi di avvio

`java: command not found` Java non è installato o non è nel PATH. Installare il JDK (§1.1) e aggiungere la cartella `bin/` al PATH di sistema.

`javac: command not found` È installato solo il JRE, non il JDK completo. Installare il JDK.

`Error: Could not find or load main class it.easyevent.v5.MainV5` Il codice non è stato compilato o la compilazione è fallita. Rieseguire lo script di compilazione e verificare l'assenza di errori.

`Attenzione: impossibile caricare i dati salvati.` Il file `easyevent_data.json` è corrotto o scritto da una versione incompatibile. L'applicazione parte con dati vuoti; il file corrotto può essere sostituito con un backup.

CAPITOLO A

Credenziali e dati demo precaricati

Il file `data/easyevent_data.json` distribuito con il progetto contiene dati di esempio pronti per la demo orale.

A.1 Credenziali

Tipo	Username	Password	Note
Configuratore	`mario`	`mario123`	—
Configuratore	`laura`	`laura456`	—
Fruitore	`alice`	`alice123`	2 notifiche
Fruitore	`bob`	`bob456`	—
Fruitore	`clara`	`clara789`	—
Fruitore	`marini`	`marini123`	—

A.2 Proposte precaricate

ID	Titolo	Categoria	Stato	Iscritti
1	Gita al Lago d'Iseo	Gite	Aperta	alice, clara
2	Gita a Montisola	Gite	Confermata	alice, bob, clara
3	Serata Jazz	Musica	Annullata	bob
4	Mostra Crepax	Arte	Ritirata	alice
5	Visita Pinacoteca T.M.	Arte	Conclusa	alice, bob
6	Torneo di Padel	Sport	Aperta	—
7	Concerto Orch. Cherubini	Musica	Aperta	bob, clara
8	Serata alla Specola	Arte	Confermata	alice, clara

CAPITOLO B

File batch di esempio completo

Di seguito il contenuto completo del file `batch_examples/setup_completo.batch`, utilizzabile per la demo orale della funzionalità batch (Capitolo 5).

```
# ============================================================
# EasyEvent V5 - File batch di esempio 3
# Oggetto: setup completo - campi, categorie e proposte
# ============================================================

# --- SEZIONE 1: CAMPI COMUNI ---

CAMPO_COMUNE | Compreso nella quota | no
CAMPO_COMUNE | Ora conclusiva | no

# --- SEZIONE 2: CATEGORIE ---

CATEGORIA | Fotografia
CATEGORIA | Cinema
CATEGORIA | Letteratura
CATEGORIA | Gastronomia | CAMPO_SPECIFICO | Intolleranze alimentari
    | no
CATEGORIA | Giochi | CAMPO_SPECIFICO | Esperienza richiesta | no |
   CAMPO_SPECIFICO | Materiale incluso | si
CATEGORIA | Escursionismo | CAMPO_SPECIFICO | Difficolta' percorso
   | si | CAMPO_SPECIFICO | Equipaggiamento consigliato | no

# --- SEZIONE 3: PROPOSTE ---

PROPOSTA | Fotografia | Titolo = Uscita fotografica al Vittoriale |
    Numero di partecipanti = 12 | Termine ultimo di iscrizione =
   31/05/2026 | Luogo = Gardone Riviera, Brescia | Data inizio =
   07/06/2026 | Ora = 09:30 | Quota individuale = 0 | Data
   conclusiva = 07/06/2026 | Compreso nella quota = Ingresso al
   Vittoriale | Durata = 4 ore | Note = Portare macchina
   fotografica

PROPOSTA | Cinema | Titolo = Rassegna Kubrick | Numero di
   partecipanti = 20 | Termine ultimo di iscrizione = 15/06/2026 |
   Luogo = Cinema Nuovo Eden, Brescia | Data inizio = 20/06/2026 |
   Ora = 20:00 | Quota individuale = 5 | Data conclusiva =
```

```
   20/06/2026 | Durata = 3 ore

PROPOSTA | Gastronomia | Titolo = Visita alla Franciacorta | Numero
    di partecipanti = 15 | Termine ultimo di iscrizione =
   10/07/2026 | Luogo = Erbusco, Brescia | Data inizio = 19/07/2026
    | Ora = 10:00 | Quota individuale = 35 | Data conclusiva =
   19/07/2026 | Compreso nella quota = Degustazione vini e pranzo |
    Durata = 6 ore | Intolleranze alimentari = Comunicare entro il
   termine di iscrizione

PROPOSTA | Giochi | Titolo = Torneo di Scacchi | Numero di
   partecipanti = 16 | Termine ultimo di iscrizione = 20/05/2026 |
   Luogo = Biblioteca Queriniana, Brescia | Data inizio =
   25/05/2026 | Ora = 14:00 | Quota individuale = 0 | Data
   conclusiva = 25/05/2026 | Esperienza richiesta = Base |
   Materiale incluso = Si

PROPOSTA | Letteratura | Titolo = Gruppo di lettura - Primo Levi |
   Numero di partecipanti = 8 | Termine ultimo di iscrizione =
   05/06/2026 | Luogo = Libreria Rinascita, Brescia | Data inizio =
    15/06/2026 | Ora = 17:30 | Quota individuale = 0 | Data
   conclusiva = 15/06/2026 | Note = Leggere preventivamente "Se
   questo e' un uomo"
```

Listing B.1: setup_completo.batch